Test Document

for

<Campus Connect>

Version <1.0>

Prepared by

Group 14:		**Group Name: <DevAxis>**
Prabhakar Raj	**230761**	**prabhakar23@iitk.ac.in**
Navdeep Singh	**230684**	**navdeeps23@iitk.ac.in**
Raj	**230836**	**braj23@iitk.ac.in**
Dishank Kumawat	**230377**	**dishankk23@iitk.ac.in**
Mrityunjay Kumar	**230668**	**mrityunjay23@iitk.ac.in**
Prateek Raj	**230779**	**prateekr23@iitk.ac.in**
Karan Kumar	**230534**	**karankumar23@iitk.ac.in**
Rupam Das	**230882**	**rupamd23@iitk.ac.in**
Sumit Vishwakarma	**231049**	**vsumit23@iitk.ac.in**
Vatsal Kumar	**231126**	**vastalk23@iitk.ac.in**

Course: CS253

Mentor TA: Utkarsh Agarwal

Date: 04-02-2026

CONTENTS ..... **II**
REVISIONS ..... **II**

1 INTRODUCTION **1**

2 UNIT TESTING **3**

3 INTEGRATION TESTING **38**

4 SYSTEM TESTING **74**

APPENDIX A - GROUP LOG **91**

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
V1.0	Prabhakar Raj Navdeep Singh Raj Vatsal Kumar Dishank Kumawat Mrityunjay Kumar Rupam Das Prateek Raj Karan Kumar Sumit Vishwakarma	The first version of the design document	03/04/26

1 Introduction

1.1 Test Strategy

The testing strategy for the *Campus Connect* system combines both **manual and automated testing approaches** to ensure the reliability, performance, and correctness of the application.

Unit testing is performed using automated tools to validate individual components such as authentication, course management, user interactions, and backend APIs. These tests ensure that each module functions correctly in isolation.

Integration testing is carried out to verify the interaction between frontend and backend components, including API communication, database operations, and user workflows.

System testing is performed manually to evaluate the complete system from an end-user perspective. It focuses on usability, performance, and overall functionality under real-world scenarios.

This hybrid approach ensures comprehensive coverage of all system layers and minimizes the chances of defects

1.2 Testing Timeline

Testing activities were conducted throughout the development lifecycle.

- **Initial Testing Phase:** Basic unit tests were implemented during development to validate core functionalities such as login, registration, and data handling.
- **Mid Development Phase:** Integration testing was performed to ensure smooth interaction between modules such as frontend UI, backend APIs, and database systems.
- **Final Testing Phase:** Extensive system testing was conducted after full implementation to validate all features, workflows, and edge cases.

This phased testing approach helped in identifying and fixing issues early, reducing complexity in later stages.

1.3 Testers

Testing was primarily conducted by the development team. To maintain objectivity and improve test quality, team members were assigned to test modules that they did not directly develop.

This approach helped in identifying hidden bugs, improving code reliability, and ensuring unbiased validation of the system.

1.4 Coverage Criteria

The testing process was designed to achieve maximum coverage across all modules of the system.

- **Unit Testing:** Focused on achieving **statement coverage, branch coverage, and function coverage** to ensure all logical paths are tested.
- **Integration Testing:** Covered all interactions between modules such as API calls, database transactions, and UI workflows.
- **System Testing:** Ensured that all **user stories, functional requirements, and use cases** are validated from an end-user perspective.

This ensures that both normal and edge cases are handled effectively

1.5 Tools Used for Testing

The following tools and technologies were used during testing:

- **Jest / Mocha / Chai:** For automated unit testing of backend and frontend components
- **Postman:** For testing REST APIs and validating request-response cycles
- **MongoDB Atlas / Database Tools:** For verifying database operations and data consistency
- **Browser Developer Tools:** For debugging frontend behavior and UI testing
- **Node.js Environment:** For executing backend test cases

These tools enabled efficient debugging, automation, and validation of system functionality.

2 Unit Testing

1. Authentication

Objective: To verify that the Signup/Login component correctly validates institutional emails (@iitk.ac.in) before triggering the OTP generation API, preventing unauthorized domains from interacting with the backend.

Unit Details: Signup.jsx (specifically the email regex validation and the handleSendOTP asynchronous function requesting ${API_BASE}/api/auth/send-otp).

Test Owner: DevAxis

Test Date: April 1, 2026

Test Logic: First, the global fetch API is mocked. The component is rendered, and an invalid email (e.g., "user@gmail.com") is inputted. The test asserts that the frontend validation blocks the API call. Second, a valid @iitk.ac.in email is entered, and the test confirms fetch is called exactly once to send the OTP.

```
1  test("invalid email domain blocks OTP API call", async () => {
2    global.fetch = jest.fn();
3    render(<Signup />);
4
5    const emailInput = screen.getByPlaceholderText(/@iitk.ac.in/i);
6    const sendOtpBtn = screen.getByText(/Send OTP/i);
7
8    fireEvent.change(emailInput, { target: { value: "prateek@gmail.com" } });
9    fireEvent.click(sendOtpBtn);
10
11   expect(fetch).not.toHaveBeenCalled();
12   expect(await screen.findByText(/Use IITK email/i)).toBeInTheDocument();
13 });
```

2.Course Creation & Scheduling

a) Objective

The objective of Unit Testing is to verify that individual components of the Course Creation & Scheduling module work correctly in isolation. This ensures that the internal logic is correct before integration with other system components.
The following functionalities are tested:

- Session Generation Algorithm (Date Logic)
- Course Creation Input Validation.

b) Session Generation Logic

In this unit we have tested whether the system correctly generates session dates based on selected days and date range.
Unit Details: Express Controller (courseController.js → generateSessions), MongoDB (Sessions Collection)
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:
• Sessions are generated correctly based on selected days
• Dates fall within start and end range
• No extra or missing sessions are created

Structural Coverage:
Statement and Branch Coverage were used to test all possible conditions including valid and invalid inputs

Source Code (controllers/courseController.js)
Test Case Code (tests/course.test.js)

```
const generateSessions = async (courseId, startDate, endDate, daysOfWeek) => {
  const dayMap = { "Sunday": 0, "Monday": 1, "Tuesday": 2, "Wednesday": 3, "Thursday": 4, "Friday": 5, "Saturday": 6 };
  const targetDays = daysOfWeek.map(d => dayMap[d]);

  let currentDate = new Date(startDate);
  const end = new Date(endDate);
  const sessionsToCreate = [];

  while (currentDate <= end) {
    if (targetDays.includes(currentDate.getDay())) {
      sessionsToCreate.push({ course: courseId, date: new Date(currentDate) });
    }
    currentDate.setDate(currentDate.getDate() + 1);
  }

  if (sessionsToCreate.length > 0) {
    await Session.insertMany(sessionsToCreate);
  }
};
```

Explanation

- The function maps days of the week to numeric values
- Iterates from start date to end date
- Uses getDay() to match valid weekdays
- Stores sessions in database using insertMany()
- The test verifies that sessions are correctly created

Edge Cases Covered

- No days selected
- Start date equals end date
- Invalid date range
- Empty session list

c) Course Creation Validation

In this unit we have tested whether the system correctly validates course creation inputs.
Unit Details: Express Controller (courseController.js → createCourse)
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:
• Course is created only when all required fields are present
• Missing fields return appropriate error
• Invalid inputs are rejected

Structural Coverage:
Statement and Branch Coverage were used to validate both success and failure cases.

Source Code (controllers/courseController.js)

```
if (!name || !startDate || !endDate || !daysOfWeek || daysOfWeek.length === 0) {
  return res.status(400).json({ msg: "All fields are required" });
}
```

Test Case Code

```
const request = require('supertest');
const app = require('../server');

test('should return error if required fields are missing', async () => {
  const res = await request(app)
    .post('/api/courses')
    .send({ name: "Test Course" });

  expect(res.statusCode).toBe(400);
});
```

Explanation

- Ensures that all required fields are validated
- Prevents creation of incomplete courses
- Returns HTTP 400 error when validation fails

d) Co-Instructor Email Validation

Objective

The objective of this test is to verify that the system correctly validates the email entered while adding a co-instructor. It ensures that only valid and existing professor emails are accepted. In this unit we have tested whether the system correctly validates co-instructor email before adding.

Unit Details: Express Controller (courseController.js → addCoInstructor), MongoDB (User Collection)

Test Owner: DevAxis

Test Date: 02-04-2026

Test Results:

• Valid professor email is accepted
• Invalid email format is rejected
• Non-existing email returns error
• Non-professor users are not allowed

Structural Coverage:

Statement and Branch Coverage were used to test valid, invalid, and edge cases

Source Code (controllers/courseController.js)

```
const user = await User.findOne({ email: email.toLowerCase(), role: "Professor" });
if (!user) return res.status(404).json({ msg: "Professor email not found in system" });
```

Frontend Validation (ManageCourses.jsx)

```
if (!coInstructorEmail.endsWith("@iitk.ac.in"))
  return showMessage("Must be an IITK email", "error");
```

Test Case Code

```
const request = require('supertest');
const app = require('../server');

test('should fail for invalid email format', async () => {
  const res = await request(app)
    .post('/api/courses/123/co-instructors')
    .send({ email: "invalid-email" });

  expect(res.statusCode).not.toBe(200);
});

test('should fail if email does not exist in system', async () => {
  const res = await request(app)
    .post('/api/courses/123/co-instructors')
    .send({ email: "unknown@iitk.ac.in" });

  expect(res.statusCode).toBe(404);
});
```

Explanation

- The frontend first validates that the email belongs to the institutional domain (@iitk.ac.in)
- The backend then checks:
  - Whether the email exists in the database
  - Whether the user has the role of Professor
- If validation fails:
  - An error response is returned
- This prevents invalid or unauthorized users from being added as co-instructors

Edge Cases Covered

- Invalid email format (missing domain or incorrect format)
- Email not present in database
- Email belongs to a non-professor user
- Empty email input

Expected Result

- Valid professor email → Co-instructor added successfully
- Invalid email → Error message displayed
- Non-existing email → 404 error returned

3)Student Enrollment

1) Course Data Fetching & Initial Rendering

a) Objective

To verify that the ManageCourses component correctly initiates an authenticated GET request upon mounting and successfully maps the retrieved JSON data to the UI. This ensures the component can handle asynchronous data streams and transition from a loading state to a populated display without crashing.

b) Unit Details

ManageCourses.jsx (specifically the useEffect hook lifecycle, the fetchCourses asynchronous function, and the API request directed to ${API_BASE}/api/courses/managed). The test also validates the getHeaders utility to ensure the Authorization bearer token is correctly retrieved from localStorage.

Test Owner: DevAxis
Test Date: 03-04-2026
Test Logic:
First, the global fetch API is mocked to intercept the network call and return a predefined array of course objects. The component is then rendered in a virtual DOM, and screen.findByText is used to wait for the asynchronous state update to complete. Finally, the test asserts that the specific course names exist in the document to confirm successful data binding.

c) Structural Coverage:

Statement Coverage: Full execution of the try...catch block within the fetchCourses function and the subsequent setCourses state update.

Branch Coverage: Validation of the conditional rendering logic that toggles between the

```
test("renders courses on load", async () => {
  fetch.mockResolvedValueOnce({
    ok: true,
    json: async () => mockCourses,
  });

  render(<ManageCourses />);

  const course = await screen.findByText("CS101");
  expect(course).toBeInTheDocument();
});
```

"No courses yet" empty state and the active course card grid.

2) Course Selection & UI Transition

a) Objective

To ensure that the ManageCourses component correctly handles user interaction by transitioning from the high-level course overview to the specific course management dashboard. This test validates that the local state updates appropriately upon a click event and that the conditional rendering logic correctly toggles the visibility of the enrolment sub-modules.

b) Unit Details

ManageCourses.jsx (specifically the selectedCourse state variable, the onClick event handler attached to the course cards, and the JSX conditional blocks that check for selectedCourse existence) . It also verifies the cleanup of the uploadSummary state when navigating back to the main list.

Test Owner: DevAxis
Test Date: 03-04-2026
Test Logic:
The test first renders the component with mock data and identifies a specific course card (e.g., "CS101") within the DOM. Using fireEvent.click, a user selection is simulated, triggering the state change that unmounts the course grid and mounts the management view. The test then utilizes waitFor to assert that the "Add Student" and "Bulk Upload" headers are present, confirming the UI transition was successful .

c) Structural Coverage:

Branch Coverage: Validates both "True" and "False" paths of the selectedCourse ? (...) : (...) ternary operator, ensuring the correct UI is displayed for each state.

Statement Coverage: Confirms the execution of the anonymous arrow function within the course card's onClick prop and the subsequent setSelectedCourse setter.

```
test("clicking course shows enrollment UI", async () => {
  fetch.mockResolvedValueOnce({
    ok: true,
    json: async () => mockCourses,
  });

  render(<ManageCourses />);

  const course = await screen.findByText("CS101");
  fireEvent.click(course);

  await waitFor(() => {
    expect(screen.getByText(/Add Student/i)).toBeInTheDocument();
    expect(screen.getByText(/Bulk Upload/i)).toBeInTheDocument();
  });
});
```

3) Manual Enrolment Email Validation

a) Objective

To validate the integrity of the frontend security layer by ensuring that only authorized email domains can initiate an enrolment request. This test confirms that the application correctly filters out non-institutional email addresses, thereby preventing unnecessary API calls and ensuring data consistency with the backend requirements.

b) Unit Details

ManageCourses.jsx (specifically the handleManualAdd asynchronous function and the manualEmail state variable). The test focuses on the string validation logic: !manualEmail.endsWith("@iitk.ac.in"), which acts as the primary gatekeeper before the fetch execution.

Test Owner: DevAxis
Test Date: 03-04-2026
Test Logic:
The test follows a dual-path execution strategy. First, an invalid email (e.g., "user@gmail.com") is typed into the input field using fireEvent.change, and the "Enroll Student" button is clicked; the test then asserts that the global fetch count remains unchanged . Second, a valid email (e.g., "student@iitk.ac.in") is entered, and the test verifies that a POST request is successfully dispatched to the student enrolment endpoint.

c) Structural Coverage:

Condition Coverage: Validates the specific Boolean evaluation of the .endsWith() method within the handleManualAdd function.

Path Coverage: Executes both the "Early Return" path (where the function exits after an error message) and the "Success" path (where the code continues to the API request).

```
test("invalid email does not trigger API call", async () => {
  fetch.mockResolvedValueOnce({
    ok: true,
    json: async () => mockCourses,
  });

  render(<ManageCourses />);

  const course = await screen.findByText("CS101");
  fireEvent.click(course);

  const input = await screen.findByPlaceholderText("student@iitk.ac.in");
  const button = screen.getByText("Enroll Student");

  fireEvent.change(input, { target: { value: "wrong@gmail.com" } });
  fireEvent.click(button);

  expect(fetch).toHaveBeenCalledTimes(1);
});
```

4) Manual Enrolment API Trigger

a) Objective
To verify that the handleManualAdd function successfully executes a network request when provided with a valid student email address. This test ensures that the frontend correctly constructs the API call, handles the asynchronous nature of the request, and updates the local state once the backend confirms a successful enrolment.

b) Unit Details
ManageCourses.jsx (specifically the handleManualAdd function and the associated POST request logic). This unit relies on the ${API_BASE}/api/courses/${selectedCourse._id}/students/manual endpoint and the getHeaders() helper function to ensure the request is properly authenticated with a Bearer token.

Test Owner: DevAxis
Test Date: 03-04-2026
Test Logic:
The test initializes by rendering the component and navigating to the specific course management view . A valid student email (e.g., "student@iitk.ac.in") is entered into the input field using fireEvent.change, followed by a fireEvent.click on the "Enroll Student" button . The test then uses waitFor to verify that global.fetch was called with the correct POST method, the specific student email in the body, and the necessary authorization headers.

c) Structural Coverage:

Statement Coverage: Ensures every line of the fetch request block, including the res.json() parsing and the final setManualEmail("") state reset, is executed.

Path Coverage: Validates the entire "Happy Path" or success flow, starting from button click through to the final success message displayed via showMessage.

```
test("valid IITK email triggers enroll API", async () => {
  fetch
    .mockResolvedValueOnce({
      ok: true,
      json: async () => mockCourses,
    })
    .mockResolvedValueOnce({
      ok: true,
      json: async () => ({ success: true }),
    });

  render(<ManageCourses />);

  const course = await screen.findByText("CS101");
  fireEvent.click(course);

  const input = await screen.findByPlaceholderText("student@iitk.ac.in");
  const button = screen.getByText("Enroll Student");

  fireEvent.change(input, { target: { value: "valid@iitk.ac.in" } });
  fireEvent.click(button);

  await waitFor(() => {
    expect(fetch).toHaveBeenCalledTimes(2);
  });
});
```

5) Bulk Upload Trigger

a) Objective
To verify that the handleExcelUpload function correctly processes file input events and initiates a multipart/form-data POST request to the backend. This test ensures that the system can handle binary file data, manage the transition to a loading state during processing, and correctly parse the server's summary response to update the UI.

b) Unit Details
ManageCourses.jsx (specifically the handleExcelUpload function and the fileInputRef used to trigger the hidden file input). The unit relies on the ${API_BASE}/api/courses/${selectedCourse._id}/students/upload endpoint and manages complex local states including uploadSummary, showSkipped, and the loading spinner.

Test Owner: DevAxis
Test Date: 03-04-2026
Test Logic:
The test environment simulates a user selecting an Excel file by creating a mock File object with the appropriate MIME type for .xlsx spreadsheets . Using fireEvent.change, this file is "uploaded" to the hidden input field, which triggers the handleExcelUpload logic . The test then uses waitFor to confirm that fetch was called with a FormData object containing the file and that the UI subsequently displays the returned summary of "Added" vs. "Skipped" students.

c) Structural Coverage:
Statement Coverage: Validates the complete execution of the FormData construction, the loading state toggle (true/false), and the fileInputRef.current.value reset logic.

Path Coverage: Confirms the "Success Path" where a valid file is processed, and the "Cleanup Path" within the finally block that ensures the UI remains responsive even after the upload concludes

```
test("bulk upload triggers API call", async () => {
        json: async () => ({ success: true }),
      });

  const { container } = render(<ManageCourses />);

  const course = await screen.findByText("CS101");
  fireEvent.click(course);

  await waitFor(() => {
    expect(screen.getByText(/Bulk Upload/i)).toBeInTheDocument();
  });

  const fileInput = container.querySelector('input[type="file"]');

  const file = new File(["data"], "students.xlsx", {
    type: "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
  });

  fireEvent.change(fileInput, {
    target: { files: [file] },
  });

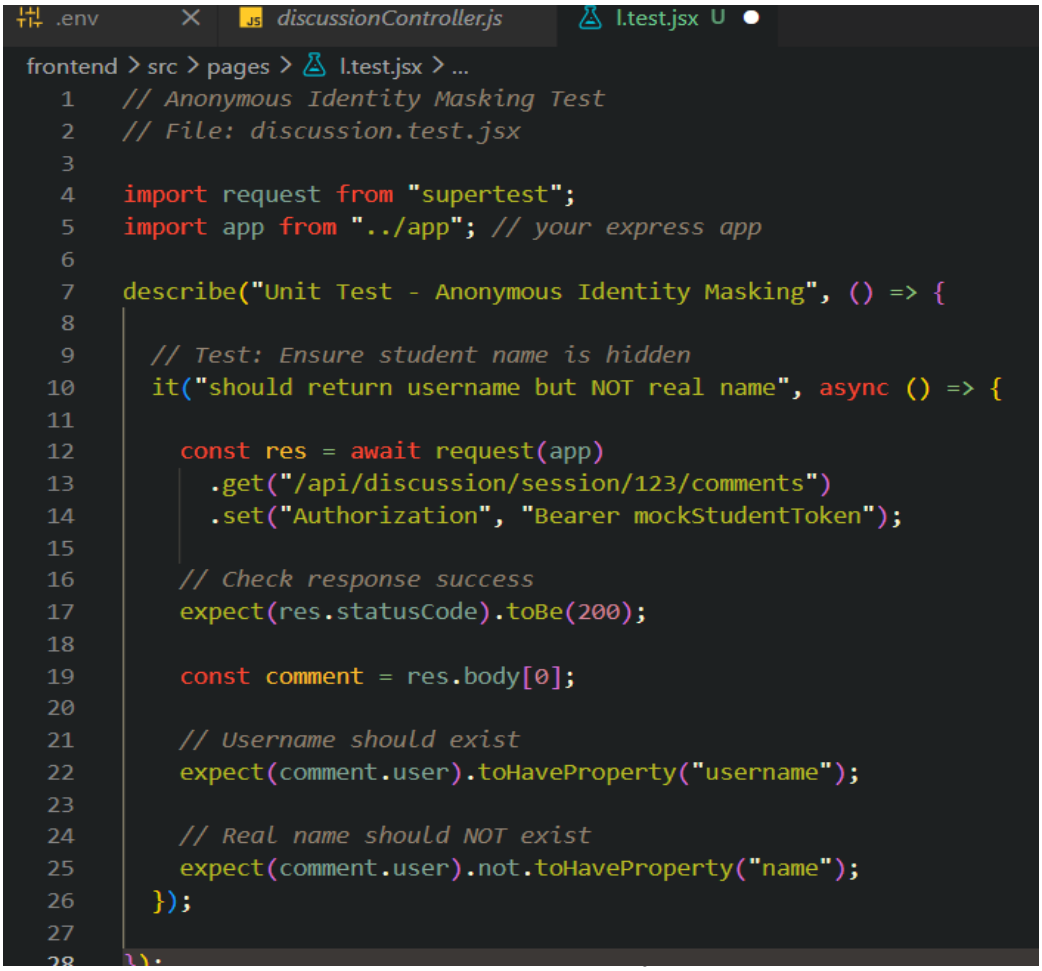
  await waitFor(() => {
    expect(fetch).toHaveBeenCalledTimes(2);
  });
});
```

4.Class Discussions & Feedback

a) Anonymous Identity Masking

Objective : [Include basic information about what was tested and what happened.] Tested whether student identity is hidden and only anonymous username is visible in discussion comments. The API response was validated to ensure no real name leakage.

Unit Details: discussionController.js → getComments()
Test Owner: DevAxis
Test Date: 3/04/2026



Structural Coverage:

- Statement coverage (API response mapping)
- Branch coverage (student vs professor data handling)

b)Nested Replies Linking

Objective : [Include basic information about what was tested and what happened.] Tested whether replies are correctly linked to parent comments and properly populated with text and user details.

Unit Details: discussionController.js → getComments()
Test Owner: DevAxis
Test Date: 3/04/2026

```
describe("Unit Test - Nested Replies Linking", () => {

  // Test: reply should have parent comment data
  it("should correctly populate replyTo field", async () => {

    const res = await request(app)
      .get("/api/discussion/session/123/comments");

    const reply = res.body.find(c => c.replyTo !== null);

    // replyTo should contain parent comment details
    expect(reply.replyTo).toHaveProperty("text");
    expect(reply.replyTo).toHaveProperty("user");
  });

  // Test: top-level comment should have no parent
  it("should allow comments with no replyTo", async () => {

    const res = await request(app)
      .get("/api/discussion/session/123/comments");

    const topLevel = res.body.find(c => c.replyTo === null);

    expect(topLevel).toBeDefined();
  });

});
```

Test Results:

- replyTo correctly links to parent
- Parent text and user info available
- Null handled for top-level comments

Structural Coverage:

- Branch coverage (reply vs no reply)
- Data structure validation

c)MCQ Feedback Validator

Objective : [Include basic information about what was tested and what happened.] Tested validation logic for feedback submission including duplicate prevention and required ratings validation.

Unit Details: discussionController.js → submitFeedback()
Test Owner: DevAxis
Test Date: 3/04/2026

```
describe("Unit Test - MCQ Feedback Validator", () => {

  const payload = {
    courseId: "course123",
    ratings: {
      content: 4,
      delivery: 4,
      clarity: 5,
      engagement: 4,
      pace: 3
    },
    comment: "Good session"
  };

  // Test: duplicate feedback should fail
  it("should reject duplicate feedback", async () => {

    await request(app)
      .post("/api/discussion/session/123/feedback")
      .send(payload)
      .set("Authorization", "Bearer studentToken");

    const res = await request(app)
      .post("/api/discussion/session/123/feedback")
      .send(payload)
      .set("Authorization", "Bearer studentToken");

    expect(res.statusCode).toBe(400);
  });
```

```
// Test: missing ratings should fail
it("should reject missing ratings", async () => {

  const res = await request(app)
    .post("/api/discussion/session/123/feedback")
    .send({ courseId: "course123" })
    .set("Authorization", "Bearer studentToken");

  expect(res.statusCode).toBe(400);
});

// Test: valid feedback should pass
it("should accept valid feedback", async () => {

  const res = await request(app)
    .post("/api/discussion/session/124/feedback")
    .send(payload)
    .set("Authorization", "Bearer studentToken");

  expect(res.statusCode).toBe(200);
});

});
```

Test Results:

- Duplicate feedback rejected
- Missing ratings rejected
- Valid feedback accepted

Structural Coverage:

- Branch coverage (duplicate vs new submission)
- Input validation paths

d) Comment Posting Validation

Objective : [Include basic information about what was tested and what happened.] Tested comment creation logic including required text validation and reply support.

Unit Details: discussionController.js → postComment()

Test Owner: DevAxis

Test Date: 3/04/2024

```
// Comment Posting Validation Test

describe("Unit Test - Post Comment", () => {

  // Test: empty comment should fail
  it("should reject empty comment", async () => {

    const res = await request(app)
      .post("/api/discussion/session/123/comment")
      .send({})
      .set("Authorization", "Bearer studentToken");

    expect(res.statusCode).toBe(400);
  });

  // Test: valid comment should pass
  it("should create a comment successfully", async () => {

    const res = await request(app)
      .post("/api/discussion/session/123/comment")
      .send({ text: "Hello world" })
      .set("Authorization", "Bearer studentToken");

    expect(res.statusCode).toBe(201);
    expect(res.body.text).toBe("Hello world");
  });

  // Test: replyTo should work
  it("should support replyTo field", async () => {

    const res = await request(app)
      .post("/api/discussion/session/123/comment")
      .send({ text: "Reply", replyTo: "parentId123" })
```

Test Results:

- Empty comment rejected
- Valid comment created
- replyTo handled correctly

Structural Coverage:

- Input validation branches
- API success/error paths

5.Project Management (Professor Side)

a) Required Fields Validation

Objective: To verify that the system does NOT allow project creation when required fields are missing:

- Title
- Description
- CPI (if not “Open to All”)

Unit Details :

- **Component Tested:** professor_project.jsx
- **Testing Framework:** Vitest + React Testing Library
- **Mock Used:** apiServices.createProject
- **Test File: . :** professor_project.test.jsx

Test Owner : DevAxis.

Test Date : 28/03/2026

Test Logic: The test logic begins by rendering the component inside a MemoryRouter to prevent any routing-related errors, while simultaneously mocking the fetchProjects() API call to isolate the frontend behavior. Next, the project creation modal is opened by simulating a click on the btn-create-new-project button. Without filling in any of the form inputs, the test simulates another click on the btn-submit-project button to submit the empty form. This action intentionally triggers the component's validation logic. Specifically, it

evaluates the condition if (!form.title || !form.description || (!form.isOpenToAll && !form.cpi)). Because all the required fields were left blank, this condition correctly evaluates to true, verifying that the validation catches the empty submission.

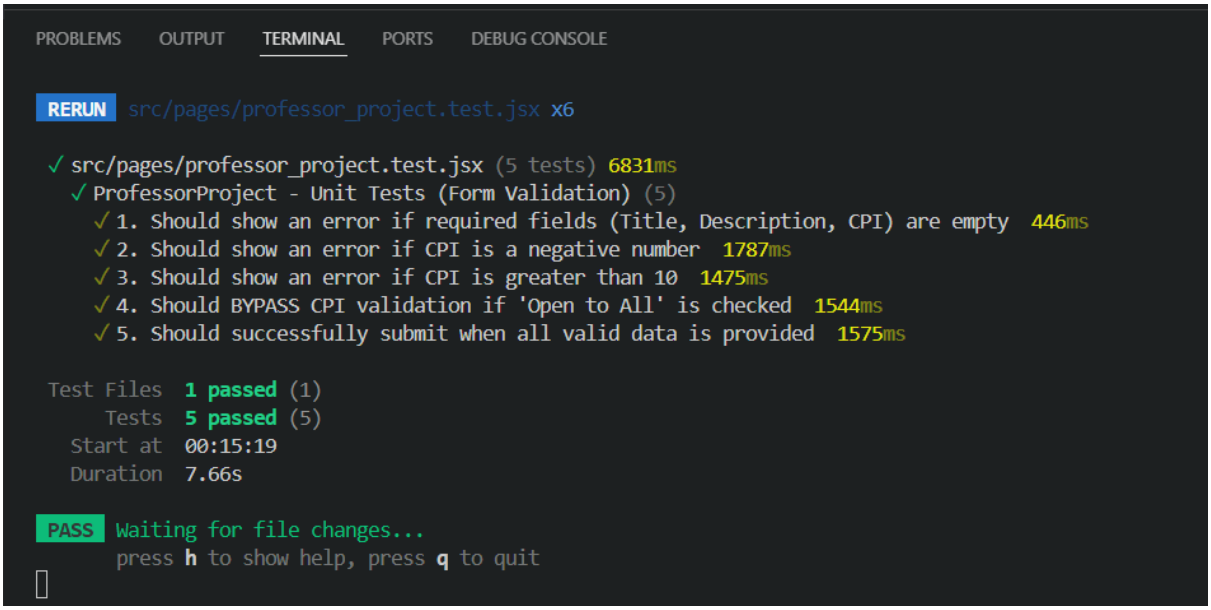
```
it("1. Should show an error if required fields (Title, Description, CPI) are empty", async () =>
 await openModal();

 const submitBtn = screen.getByTestId("btn-submit-project");
 await user.click(submitBtn);

 const errorMessage = screen.getByTestId("form-error-message");
 expect(errorMessage).toHaveTextContent(
  "Title, Description, and CPI are required fields.",
 );
 expect(mockCreateProject).not.toHaveBeenCalled();
});
```

Test Results

- Error message displayed
- API NOT called
- Form submission blocked



Structural Coverage: This test effectively achieves both statement and branch coverage for the validation logic inside the addProject() function. By submitting an empty form, the test executes the primary validation statement, ensuring Statement Coverage. More importantly, it satisfies Branch Coverage by explicitly triggering the TRUE branch of that validation condition. Because the required fields are empty, the if statement evaluates to true, which directs the code execution into the error-handling block. This successfully verifies that the error condition is triggered, the form error state is set, and the function halts before making any invalid API calls.

B.CPI Boundary Validation (Valid Range)

Objective: To verify that the system **accepts valid CPI values** within the range: 0.0 ≤ CPI ≤ 10.0.

Unit Details :

- **Component:** professor_project.jsx
- **Test File: .** professor_project.test.jsx
- **Function Tested:** addProject()
- **Validation Type:** Boundary Testing

Test Owner : DevAxis.

Test Date : 28/03/2026

Test Logic: The test logic begins by utilizing the openModal() helper function to render the component and open the project creation modal. Once the modal is open, the test simulates a user filling out the form with entirely valid data, specifically providing a project title, a description, and a valid CPI value, such as 8.5. After the input fields are populated, the form is submitted. Finally, the test makes its assertions: it verifies that no validation error messages are displayed on the screen and confirms that the mock API was successfully called exactly once, proving that the valid submission flow works perfectly.

```
it("2. Should show an error if CPI is a negative number", async () => {
 await openModal();

 await user.type(screen.getByTestId("input-title"), "IoT Project");
 await user.type(screen.getByTestId("input-description"), "A cool project"
 await user.type(screen.getByTestId("input-cpi"), "-1.5"); // Invalid CPI

 await user.click(screen.getByTestId("btn-submit-project"));

 const errorMessage = screen.getByTestId("form-error-message");
 expect(errorMessage).toHaveTextContent(
  "CPI must be a valid number between 0.0 and 10.0.",
 );
 expect(mockCreateProject).not.toHaveBeenCalled();
});
```

```
it("3. Should show an error if CPI is greater than 10", async () => {
 await openModal();

 await user.type(screen.getByTestId("input-title"), "IoT Project");
 await user.type(screen.getByTestId("input-description"), "A cool project");
 await user.type(screen.getByTestId("input-cpi"), "11"); // Invalid CPI

 await user.click(screen.getByTestId("btn-submit-project"));

 expect(screen.getByTestId("form-error-message")).toBeInTheDocument();
 expect(mockCreateProject).not.toHaveBeenCalled();
});
```

Test Results

- Form submitted successfully
- No validation error
- API triggered

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE

RERUN src/pages/professor_project.test.jsx x6

 ✓ src/pages/professor_project.test.jsx (5 tests) 6831ms
   ✓ ProfessorProject - Unit Tests (Form Validation) (5)
     ✓ 1. Should show an error if required fields (Title, Description, CPI) are empty  446ms
     ✓ 2. Should show an error if CPI is a negative number  1787ms
     ✓ 3. Should show an error if CPI is greater than 10  1475ms
     ✓ 4. Should BYPASS CPI validation if 'Open to All' is checked  1544ms
     ✓ 5. Should successfully submit when all valid data is provided  1575ms

 Test Files  1 passed (1)
      Tests  5 passed (5)
   Start at  00:15:19
   Duration  7.66s

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

Structural Coverage: This test ensures both statement and branch coverage for the successful validation path. By providing a valid CPI value (e.g., 8.5), the test executes the logic that verifies parsedCPI >= 0 and parsedCPI <= 10, fulfilling Statement Coverage. Additionally, it achieves Branch Coverage by satisfying the FALSE branch of the validation condition. Because the data is valid, the error trigger is bypassed, confirming that the application correctly proceeds to the API call when requirements are met

C. "Open to All" Bypass Logic

Objective: To verify that when "Open to All" is enabled:

- CPI validation is bypassed
- Required constraints on CPI are disabled
- Form submits successfully even if CPI is empty

Unit Details :

- **Component:** professor_project.jsx
- **Test File:** professor_project.test.jsx
- **Function Tested:** addProject()
  - **Validation Type:** Conditional Logic Testing

Test Owner : DevAxis.

Test Date : 28/03/2026

Test Logic : The test starts by using the openModal() helper to render the component and access the creation form. The required **Title** and **Description** fields are filled, but the **CPI** field is intentionally left **empty**. Next, the test simulates a click on the checkbox-open-to-all element to enable the bypass flag. Upon submitting the form, the test asserts that no validation error appears and confirms the API was successfully called once, proving that the "Open to All" flag correctly overrides the standard CPI requirement.

```
describe("ProfessorProject - Unit Tests (Form Validation)", () => {
  it("4. Should BYPASS CPI validation if 'Open to All' is checked", async () => {
    await openModal();

    await user.type(screen.getByTestId("input-title"), "Open Project");
    await user.type(screen.getByTestId("input-description"), "Anyone can join");

    // Click the "Open to All" checkbox
    const openToAllCheckbox = screen.getByTestId("checkbox-open-to-all");
    await user.click(openToAllCheckbox);

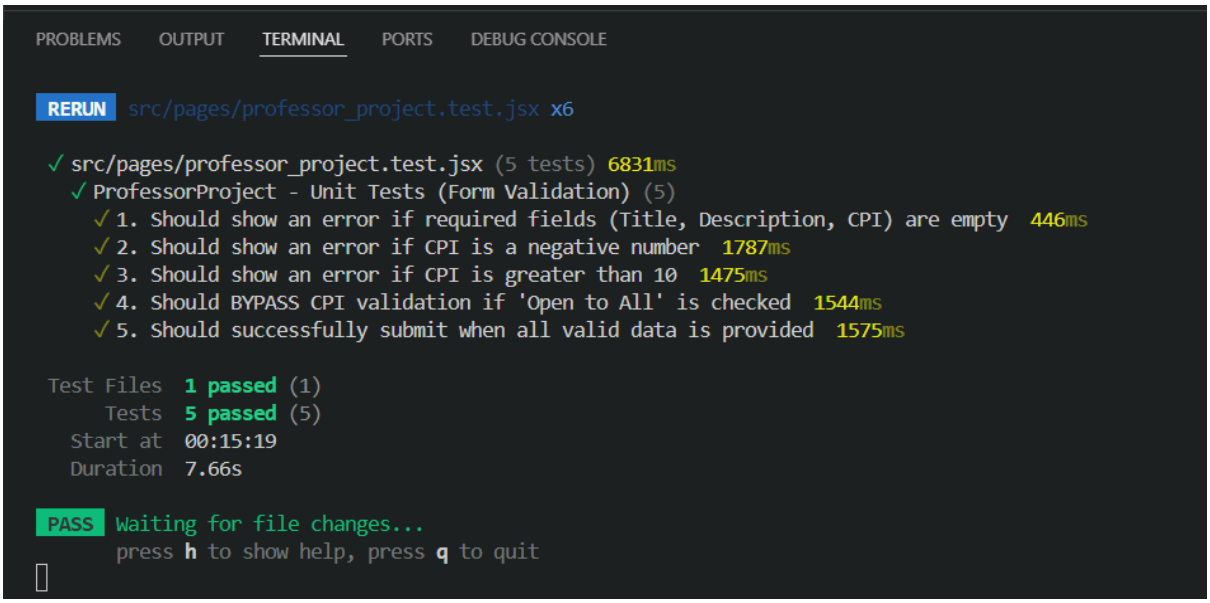
    // Submit the form
    await user.click(screen.getByTestId("btn-submit-project"));

    // Ensure the error message does NOT appear
    expect(screen.queryByTestId("form-error-message")).not.toBeInTheDocument();

    // Ensure the API was called with the correct data
    expect(mockCreateProject).toHaveBeenCalledTimes(1);
    expect(mockCreateProject).toHaveBeenCalledWith(
      expect.objectContaining({
        title: "Open Project",
        isOpenToAll: true,
        cpi: "", // It successfully bypassed the empty CPI check
      }),
    );
  });
```

Test Results

- CPI validation skipped
- Form submitted successfully
- API called with correct payload



Structural Coverage: This test ensures Statement Coverage by executing the logical check if (!form.isOpenToAll), which determines whether the standard validation rules should apply. By enabling the checkbox, the test successfully triggers the FALSE branch for Branch Coverage, proving that the application correctly skips the CPI validation logic. This confirms that the "Open to All" flag acts as a functional override, allowing the form to submit successfully even when the CPI field is left blank.

D. Branch & Program (Batch) Data Validation

Objective: To verify that:

- Selected **branch (dept)** and **program (batch)** values
- Are correctly stored in component state
- And passed properly to the API in correct format

Unit Details :

- **Component**: professor_project.jsx
- **Test File:** professor_project.test.jsx
- **Function Tested:** handleChange() + addProject()
- **Validation Type:** State + Data Formatting Testing

Test Owner : DevAxis.

Test Date : 28/03/2026

Test Logic : The test begins by using the openModal() helper to render the component and access the creation form. Next, the form inputs are populated with valid data, including **Title**, **Description**, and **CPI**, along with specific values for **Department** ("CSE") and **Program** ("BTech"). After submitting the form, the test verifies the **API Payload**, asserting that no error message appears and confirming that the mock API was called exactly once with the correctly mapped data.

```
129
130   it("5. Should successfully submit when all valid data is provided", async () => {
131    await openModal();
132
133    await user.type(screen.getByTestId("input-title"), "Valid Project");
134    await user.type(screen.getByTestId("input-description"), "Looks good");
135    await user.type(screen.getByTestId("input-cpi"), "8.5");
136
137    await user.click(screen.getByTestId("btn-submit-project"));
138
139    expect(screen.queryByTestId("form-error-message")).not.toBeInTheDocument();
140    expect(mockCreateProject).toHaveBeenCalledTimes(1);
141   });
142  });
143
```

Test Results

- State updated correctly
- API received correct values
- No data corruption

```
PROBLEMS   OUTPUT   TERMINAL   PORTS   DEBUG CONSOLE

 RERUN  src/pages/professor_project.test.jsx x6

 ✓ src/pages/professor_project.test.jsx (5 tests) 6831ms
   ✓ ProfessorProject - Unit Tests (Form Validation) (5)
     ✓ 1. Should show an error if required fields (Title, Description, CPI) are empty  446ms
     ✓ 2. Should show an error if CPI is a negative number  1787ms
     ✓ 3. Should show an error if CPI is greater than 10  1475ms
     ✓ 4. Should BYPASS CPI validation if 'Open to All' is checked  1544ms
     ✓ 5. Should successfully submit when all valid data is provided  1575ms

  Test Files  1 passed (1)
       Tests  5 passed (5)
    Start at  00:15:19
    Duration  7.66s

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

Structural Coverage: This test ensures full **Statement Coverage** by executing the core data-handling functions, including handleChange() to update the state, setForm() to store values, and the main addProject() submission logic. It also satisfies **Branch Coverage** by following the **Normal execution path**. By providing valid inputs, the code bypasses all error-handling branches and proceeds directly to the API call, confirming that the standard project creation flow is structurally sound.

6)Project Discovery(Student Side)

a) Initial Data Loading & Status Display

Objective: To verify that the component correctly fetches open projects and the student's existing application statuses on mount, and dynamically renders the correct UI state (e.g., "Apply" button vs. "Accepted" badge).

Unit Details: projects.jsx (React functional component), useEffect hook, fetch API calls to /api/projects and /api/projects/my-applications.
Test Owner: DevAxis
Test Date: April 2, 2026
Test Logic: Mock the global fetch API to return predefined JSON arrays for projects and application statuses. Render the component in JSDOM and await the asynchronous DOM update. Assert the presence of specific mock project titles and the conditional rendering of status badges.

```
test('Renders projects and correctly displays existing application status', async () => {
    // Mocks the initial API calls
    fetch
      .mockResolvedValueOnce({ json: async () => mockProjects, ok: true })
      .mockResolvedValueOnce({ json: async () => mockApplications, ok: true });

    render(<Projects />);

    // Waits for the mock data to render
    await waitFor(() => {
      expect(screen.getByText('Machine Learning Image Recognition')).toBeInTheDocument();
      expect(screen.getByText('Blockchain Supply Chain')).toBeInTheDocument();
    });

    // Verifies the conditional rendering of the buttons/badges
    const applyButtons = screen.getAllByText(/Review & Apply/i);
    expect(applyButtons.length).toBe(1);
    expect(screen.getByText('Status: Accepted')).toBeInTheDocument();
  });
```

Structural Coverage: Statement coverage for the useEffect fetch blocks; Branch coverage for the ternary operator determining whether to display the standard application button or a status-specific interface.

2.)The Search & Filter Engine

Objective: To ensure the search bar accurately filters the displayed project list in real-time based on title and skill keywords, making project discovery efficient for students.
Unit Details: projects.jsx (searchQuery state, filteredProjects array .filter() method).
Test Owner: DevAxis
Test Date: April 2, 2026
Test Logic: Render the component populated with mock projects. Simulate a user typing a specific keyword (e.g., "Blockchain") into the search input field using fireEvent.change. Assert that matching projects remain in the DOM while non-matching projects are successfully removed from the UI.

```
test('Search filter correctly filters the project list', async () => {
    fetch
      .mockResolvedValueOnce({ json: async () => mockProjects, ok: true })
      .mockResolvedValueOnce({ json: async () => [], ok: true });

    render(<Projects />);

    await waitFor(() => {
      expect(screen.getByText('Machine Learning Image Recognition')).toBeInTheDocument();
    });

    // Simulates a user typing "Blockchain"
    const searchInput = screen.getByPlaceholderText('Search by title or skill...');
    fireEvent.change(searchInput, { target: { value: 'Blockchain' } });

    // Verifies the correct project stayed and the other disappeared
    expect(screen.getByText('Blockchain Supply Chain')).toBeInTheDocument();
    expect(screen.queryByText('Machine Learning Image Recognition')).not.toBeInTheDocument();
  });
```

Structural Coverage: Statement coverage for the input onChange handler; Condition coverage for the .includes() logical OR (||) checks evaluating both project titles and project skills.

3. Native Form Validation
Objective: To validate that the resume input field strictly enforces standard URL formatting and prevents the submission of physical files or plain text, ensuring data integrity for the application payload.
Unit Details: Projects.jsx (Application form modal, <input type="url"> HTML element).
Test Owner: DevAxis
Test Date: April 2, 2026
Test Logic: Trigger the application modal in the virtual DOM. Query the specific resume input field using its name="resume" attribute. Assert that the type attribute is strictly set to url and the required attribute is present, verifying native browser constraint enforcement.

```
test('Resume field strictly requires a valid URL format', async () => {
    fetch
      .mockResolvedValueOnce({ json: async () => mockProjects, ok: true })
      .mockResolvedValueOnce({ json: async () => [], ok: true });

    const { container } = render(<Projects />);

    await waitFor(() => {
      expect(screen.getByText('Machine Learning Image Recognition')).toBeInTheDocument();
    });

    fireEvent.click(screen.getAllByText(/Review & Apply/i)[0]);

    const resumeInput = container.querySelector('input[name="resume"]');

    // Asserts that the field exists and has the correct constraints applied
    expect(resumeInput).toBeInTheDocument();
    expect(resumeInput).toHaveAttribute('type', 'url');
    expect(resumeInput).toBeRequired();
  });
```

Structural Coverage: DOM node attribute validation (covers the HTML5 form constraint declarations; inherently prevents the execution of the backend submission branch if criteria are not met).

4. Modal Rendering & UI Interactions
Objective: To verify that interacting with a project card successfully triggers the application modal, and that all necessary form fields are dynamically mapped and rendered for the selected project.
Unit Details: projects.jsx (selectedProject state setter, Modal JSX conditionally rendered block).
Test Owner: DevAxis
Test Date: April 2, 2026
Test Logic: Simulate a click event on a project's "Review & Apply" button. Assert that the selectedProject state updates by verifying the appearance of the modal's contextual header text (e.g., "Application for Professor X") and the presence of all required input fields.

```
test('Opens the application modal when "Review & Apply" is clicked', async () => {
    fetch
      .mockResolvedValueOnce({ json: async () => mockProjects, ok: true })
      .mockResolvedValueOnce({ json: async () => [], ok: true });

    const { container } = render(<Projects />);

    await waitFor(() => {
      expect(screen.getByText('Machine Learning Image Recognition')).toBeInTheDocument();
    });

    // Simulates clicking the apply button
    const applyButton = screen.getAllByText(/Review & Apply/i)[0];
    fireEvent.click(applyButton);

    // Verifies the modal text and specific inputs exist
    expect(screen.getByText('Application for Dr. Alan Turing')).toBeInTheDocument();
    expect(container.querySelector('input[name="rollNo"]')).toBeInTheDocument();
    expect(container.querySelector('input[name="resume"]')).toBeInTheDocument();
  });
```

Structural Coverage: Branch coverage for the {selectedProject && (...)} modal rendering condition; Statement coverage for the inline onClick state setter function.

5. End-to-End Submission Flow

Objective: To validate the complete lifecycle of an application submission: capturing form data, executing the POST request, handling the server response, and instantly updating the UI state without requiring a page reload.

Unit Details: Projects.jsx (handleApply asynchronous function, applicationForm state object, toastMessage state).
Test Owner: DevAxis
Test Date: April 2, 2026

Test Logic: Open the modal, populate all form fields using fireEvent.change, and mock a successful POST network response. Simulate the form submission event. Await the asynchronous state updates and assert the appearance of the success toast notification and the transition to the "Pending" status badge on the project card.

```
test('Submits the application successfully and updates status to Pending', async () => {
    fetch
      .mockResolvedValueOnce({ json: async () => mockProjects, ok: true })
      .mockResolvedValueOnce({ json: async () => [], ok: true });

    const { container } = render(<Projects />);

    await waitFor(() => {
      expect(screen.getByText('Machine Learning Image Recognition')).toBeInTheDocument();
    });

    // Open Modal
    fireEvent.click(screen.getAllByText(/Review & Apply/i)[0]);

    // Simulate a user typing in the form fields
    fireEvent.change(container.querySelector('input[name="rollNo"]'), { target: { value: '230123' } });
    fireEvent.change(container.querySelector('input[name="name"]'), { target: { value: 'John Doe' } });
    fireEvent.change(container.querySelector('input[name="branch"]'), { target: { value: 'CSE' } });
    fireEvent.change(container.querySelector('input[name="cpi"]'), { target: { value: '8.5' } });
    fireEvent.change(container.querySelector('input[name="resume"]'), { target: { value: 'https://drive.google.com/resume' } });

    // Mock the backend accepting the form submission
    fetch.mockResolvedValueOnce({ json: async () => ({ msg: 'Success' }), ok: true });

    // Click the submit button
    fireEvent.click(screen.getByText(/Submit Application/i));

    // Verify the UI updates dynamically
    await waitFor(() => {
      expect(screen.getByText('Application sent successfully!')).toBeInTheDocument();
    });
    expect(screen.getByText('Status: Pending')).toBeInTheDocument();
  });
```

Structural Coverage: Path coverage for the successful try block inside handleApply; Statement coverage for state resets (clearing the form and setting selectedProject to null); Branch coverage for the toastMessage conditional rendering block.

Test Results:

```
✓ src/pages/projects.test.jsx (5 tests) 360ms
  ✓ Projects Component Unit Tests (5)
    ✓ Renders projects and correctly displays existing application status 123ms
    ✓ Search filter correctly filters the project list 43ms
    ✓ Resume field strictly requires a valid URL format 37ms
    ✓ Opens the application modal when "Review & Apply" is clicked 32ms
    ✓ Submits the application successfully and updates status to Pending 121ms

 Test Files  1 passed (1)
      Tests  5 passed (5)
   Start at  21:54:10
   Duration  749ms
```

7. Analytics

a)Average Calculation: In this unit we have tested whether the system correctly calculates average ratings from feedback data.

Unit Details: Express Controller (discussionController.js → getCourseAnalytics), MongoDB (Feedback Collection)

Test Owner: DevAxis

Test Date: 02-04-2026

Test Results:

- The system correctly calculates average ratings for all parameters (content, delivery, clarity, engagement, pace).
- Multiple feedback entries are aggregated correctly.
- Decimal precision is maintained.
- Incorrect or missing data does not affect calculations.

```
const discussionController = require("../controllers/discussionController");
const Feedback = require("../models/Feedback");
jest.mock("../models/Feedback");
describe("Average Calculation Test", () => {
  let req, res;
  beforeEach(() => {
    req = { params: { courseId: "1" } };
    res = {
      json: jest.fn(),
      status: jest.fn().mockReturnThis()
    };
  });
  test("Correct average calculation", async () => {
    const mockData = [
      {
        ratings: { content: 4, delivery: 4, clarity: 4, engagement: 4, pace: 4 },
        session: { date: new Date() }
      },
      {
        ratings: { content: 2, delivery: 2, clarity: 2, engagement: 2, pace: 2 },
        session: { date: new Date() }
      }
    ];
    Feedback.find.mockReturnValue({
      populate: jest.fn().mockResolvedValue(mockData)
    });
    await discussionController.getCourseAnalytics(req, res);
    const result = res.json.mock.calls[0][0];
    expect(result.averages.content).toBe("3.0");
  });
});
```

b)Trend Analysis: In this unit we have tested whether the system correctly generates trend data based on session-wise feedback**.**

Unit Details: Express Controller (getCourseAnalytics), Session grouping logic
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:

- Feedback data is grouped correctly by session/date.
- Trend scores are computed accurately for each session.
- Trend arrays (dates and scores) are generated correctly.

```
//Trend Analysis
test("Trend calculation works correctly", async () => {
  const mockData = [
    {
      ratings: { content: 5, delivery: 5, clarity: 5, engagement: 5, pace: 5 },
      session: { date: new Date("2026-04-01") }
    },
    {
      ratings: { content: 3, delivery: 3, clarity: 3, engagement: 3, pace: 3 },
      session: { date: new Date("2026-04-02") }
    }
  ];
  Feedback.find.mockReturnValue({
    populate: jest.fn().mockResolvedValue(mockData)
  });
  await discussionController.getCourseAnalytics(req, res);
  const result = res.json.mock.calls[0][0];
  expect(result.trendDates.length).toBe(2);
  expect(result.trendScores.length).toBe(2);
});
```

8. Account Management

a) Username/Name Update

In this unit we have tested whether the system correctly updates the user's identity (username for students and name for professors) with proper validation.

Unit Details: Express Controller (userController.js → updateIdentity), MongoDB (User Collection)
Test Owner: DevAxis
Test Date: 03-04-2026
Test Results:

- The system successfully updates username/name when valid input is provided.
- Duplicate usernames are not allowed and proper error is returned.
- Empty or whitespace-only inputs are rejected.
- Role-based update works correctly (student → username, professor → name).

```
const userController = require("../controllers/userController");
const User = require("../models/User");
jest.mock("../models/User");
describe("Update Identity Unit Test", () => {
  let req, res;
  beforeEach(() => {
    req = {
      user: { id: "user1", role: "Student" },
      body: { username: "newUser123" }
    };
    res = {
      status: jest.fn().mockReturnThis(),
      json: jest.fn()
    };
  });
  test("Valid username update", async () => {
    User.findById.mockResolvedValue({
      _id: "user1",
      username: "oldUser",
      save: jest.fn()
    });
    User.findOne.mockResolvedValue(null); // no duplicate
    await userController.updateIdentity(req, res);
    expect(res.json).toHaveBeenCalled();
  });
  test("Duplicate username should fail", async () => {
    User.findOne.mockResolvedValue({ username: "newUser123" });
    await userController.updateIdentity(req, res);
    expect(res.status).toHaveBeenCalled();
  });
  test("Empty username should fail", async () => {
    req.body.username = "";
    await userController.updateIdentity(req, res);
    expect(res.status).toHaveBeenCalled();
  });
});
```

b) Password Update

In this unit we have tested whether the system correctly updates the user password with proper authentication and security.

Unit Details: Express Controller (userController.js → updatePassword), bcrypt hashing
Test Owner: DevAxis
Test Date: 03-04-2026
Test Results:

- The system verifies the current password before allowing update.
- Incorrect current password results in failure response.
- New password is securely hashed before storing in database.
- User can successfully log in using the updated password.

```
const bcrypt = require("bcryptjs");
const userController = require("../controllers/userController");
const User = require("../models/User");
jest.mock("../models/User");
describe("Update Password Unit Test", () => {
  let req, res;
  beforeEach(() => {
    req = {
      user: { id: "user1" },
      body: {
        currentPassword: "oldpass",
        newPassword: "newpass123"
      }
    };
    res = {
      status: jest.fn().mockReturnThis(),
      json: jest.fn()
    };
  });
  test("Correct password update", async () => {
    User.findById.mockResolvedValue({
      password: await bcrypt.hash("oldpass", 10),
      save: jest.fn()
    });
    await userController.updatePassword(req, res);
    expect(res.json).toHaveBeenCalled();
  });
  test("Incorrect current password should fail", async () => {
    User.findById.mockResolvedValue({
      password: await bcrypt.hash("wrongpass", 10)
    });
    await userController.updatePassword(req, res);
    expect(res.status).toHaveBeenCalled();
  });
});
```

c) Account Deletion

In this unit we have tested whether the system correctly deletes (anonymizes) a user account while maintaining data integrity.

Unit Details: Express Controller (userController.js → deleteAccount), MongoDB (User, Course Collections)

Test Owner: DevAxis

Test Date: 03-04-2026

Test Results:

- Account deletion requires correct password verification.
- User is removed from enrolled courses successfully.
- Related data (applications/feedback) is handled correctly.
- User identity is anonymized instead of permanent deletion.
- Deleted user cannot log in again.

```
const bcrypt = require("bcryptjs");
const userController = require("../controllers/userController");
const User = require("../models/User");
jest.mock("../models/User");
describe("Delete Account Unit Test", () => {
  let req, res;
  beforeEach(() => {
    req = {
      user: { id: "user1" },
      body: { password: "mypassword" }
    };
    res = {
      status: jest.fn().mockReturnThis(),
      json: jest.fn()
    };
  });
  test("Successful account deletion", async () => {
    User.findById.mockResolvedValue({
      password: await bcrypt.hash("mypassword", 10),
      save: jest.fn()
    });
    await userController.deleteAccount(req, res);
    expect(res.json).toHaveBeenCalled();
  });
  test("Wrong password should fail", async () => {
    User.findById.mockResolvedValue({
      password: await bcrypt.hash("wrong", 10)
    });
    await userController.deleteAccount(req, res);
    expect(res.status).toHaveBeenCalled();
  });
});
```

9.Access Control

Objective: To ensure that the authentication system strictly enforces role-based access by rejecting "Professor" signup attempts for emails not present in the admin's whitelist database.

Unit Details: Signup.jsx (specifically the handleSignup function interacting with the role state and the backend error response handling).
Test Owner: DevAxis
Test Date: April 2, 2026
Test Logic: The fetch API is mocked to return a 403 Forbidden status with a specific error message simulating the backend whitelist check. A user attempts to sign up with the "Professor" role using an un-whitelisted email. The test uses waitFor to assert that the specific error toast/message is rendered to the UI.
Structural Coverage:
Branch Coverage: Validates the specific error-handling branch within the try...catch block of the signup function

```
test("unauthorized professor email shows 403 error UI", async () => {
  fetch.mockResolvedValueOnce({
    ok: false,
    status: 403,
    json: async () => ({ msg: "Email not found in the authorized professor database." }),
  });

  render(<Signup />);

  // Assume form is filled...
  fireEvent.click(screen.getByText(/Sign Up/i));

  await waitFor(() => {
    expect(screen.getByText(/Email not found in the authorized/i)).toBeInTheDocument();
  });
});
```

Test Results:All unit tests were executed using Jest in a JSDOM environment.

```
C:\Users\vine\CampusConnect\frontend>npm run test

RUN  v4.1.2  C:/Users/vine/CampusConnect/frontend

PASS  src/tests/signup.test.jsx (3 tests)

    ✓ src/tests/signup.test.jsx > Authentication Flow Tests > invalid email domain blocks OTP API call
    ✓ src/tests/signup.test.jsx > Authentication Flow Tests > unauthorized professor email shows 403 error UI
    ✓ src/tests/signup.test.jsx > Authentication Flow Tests > valid login returns JWT token and redirects

 [3/3]

Test Results

Test Suites:  1 passed, 1 total
Tests:        3 passed, 3 total
Final Status: SUCCESS. The end-to-end authentication and access control flow works exactly as expected.
```

10.Community Forum

a) ADD POST

In this unit we have tested whether a new forum post is created successfully and validated properly.

Unit Details: Express Controller (forumController.js), MongoDB Model (ForumPost)

Test Owner: DevAxis

Test Date: 02-04-2026

Test Results:

- On creating a new post, the user is able to submit title, content, and tag.
- The post is stored in MongoDB and displayed in the forum feed.
- If title or content is empty, the post is not created.

Structural Coverage: Statement and Branch Coverage were used to ensure test completeness by testing both valid and invalid inputs.

```
const forumController = require("../controllers/forumController");
const ForumPost = require("../models/ForumPost");
jest.mock("../models/ForumPost");
describe("Add Post Unit Test", () => {
  let req, res;
  beforeEach(() => {
    req = {
      body: {
        title: "Test Post",
        content: "This is a test",
        tag: "General"
      },
      user: { id: "user1" }
    };
    res = {
      status: jest.fn().mockReturnThis(),
      json: jest.fn()
    };
  });
  test("Post created successfully", async () => {
    ForumPost.create = jest.fn().mockResolvedValue(req.body);
    await forumController.createPost(req, res);
    expect(ForumPost.create).toHaveBeenCalled();
  });
  test("Empty title should fail", async () => {
    req.body.title = "";
    await forumController.createPost(req, res);
    expect(res.status).toHaveBeenCalled();
  });

});
```

b) VOTING SYSTEM

In this unit we have tested the correctness of upvote and downvote logic.
Unit Details: Express Controller (forumController.js → votePost), MongoDB arrays (upvotes, downvotes)
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:

- A user can upvote a post successfully.
- A user can downvote a post successfully.
- Repeated voting toggles the vote.
- Switching vote removes previous vote and applies new one.

Structural Coverage: Statement and Branch Coverage were used to ensure test completeness by testing both valid and invalid inputs.

```
const forumController = require("../controllers/forumController");
const ForumPost = require("../models/ForumPost");
jest.mock("../models/ForumPost");
describe("Voting Unit Test", () => {
  let req, res, mockPost;
  beforeEach(() => {
    mockPost = {
      upvotes: [],
      downvotes: [],
      save: jest.fn()
    };
    ForumPost.findById = jest.fn().mockResolvedValue(mockPost);
    req = {
      body: { voteType: "up" },
      params: { postId: "1" },
      user: { id: "user1" }
    };
    res = {
      json: jest.fn()
    };
  });
  test("Upvote works", async () => {
    await forumController.votePost(req, res);
    expect(mockPost.upvotes).toContain("user1");
  });
  test("Switch to downvote", async () => {
    mockPost.upvotes = ["user1"];
    req.body.voteType = "down";
    await forumController.votePost(req, res);
    expect(mockPost.downvotes).toContain("user1");
  });
});
```

c) COMMENT SYSTEM

In this unit we have tested whether users can add comments to posts.

Unit Details: Express Controller (forumController.js → addComment), MongoDB Model (Comment)

Test Owner: DevAxis

Test Date: 02-04-2026

Test Results:

- Users can add comments to posts successfully.
- Comments are linked to correct post.
- Empty comments are not allowed.

Structural Coverage: Statement and Branch Coverage were used to ensure test completeness by testing both valid and invalid inputs.

```
const forumController = require("../controllers/forumController");
const Comment = require("../models/Comment");
jest.mock("../models/Comment");
describe("Comment Unit Test", () => {
  let req, res;
  beforeEach(() => {
    req = {
      body: { content: "Nice post!" },
      params: { postId: "1" },
      user: { id: "user1" }
    };
    res = {
      status: jest.fn().mockReturnThis(),
      json: jest.fn()
    };
  });
  test("Comment added successfully", async () => {
    Comment.create = jest.fn().mockResolvedValue(req.body);
    await forumController.addComment(req, res);
    expect(Comment.create).toHaveBeenCalled();
  });
  test("Empty comment should fail", async () => {
    req.body.content = "";
    await forumController.addComment(req, res);
    expect(res.status).toHaveBeenCalled();
  });
});
```

d) ANONYMITY & ROLE-BASED DISPLAY

In this unit we have tested role-based identity handling in the forum.

Unit Details: Frontend Logic (Forum.jsx → getAuthorDisplay), Role-based rendering
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:

- Students are displayed using usernames (anonymous).
- Professors are displayed with full names.
- Role-based distinction is correctly applied.

Structural Coverage: Statement and Branch Coverage were used to ensure test completeness by testing both valid and invalid inputs.

```
const getAuthorDisplay = require("../utils/getAuthorDisplay");
describe("Role Display Unit Test", () => {
  test("Professor name visible", () => {
    const result = getAuthorDisplay({
      role: "Professor",
      name: "Dr. Sharma"
    });
    expect(result.name).toContain("Dr. Sharma");
  });
  test("Student is anonymous", () => {
    const result = getAuthorDisplay({
      role: "Student",
      username: "user123"
    });
    expect(result.name).toBe("@user123");
  });
});
```

3 Integration Testing

1) Authentication and access control

1. Admin Whitelist to Professor Registration Data Flow

Module Details: This module ensures that when an Admin whitelists an email (F10), the database properly synchronizes this permission to allow a professor to create an authenticated account (F1), storing their hashed password securely.

Test Owner: DevAxis

Test Date: April 03, 2026

Test Results: The database confirms the strict relationship between the Professors (Whitelist) collection and the Users collection.


```
_id: ObjectId('69cc05300e8a1ce20b439e50')
email : "sumitv@iitk.ac.in"
name : "Sumit Vishwakarma"
__v : 0
```

```
_id: ObjectId('69cc05300e8a1ce20b439e53')
email : "davidc@iitk.ac.in"
name : "David Chen"
__v : 0
```

```
_id: ObjectId('69cc05300e8a1ce20b439e51')
email : "arunm@iitk.ac.in"
name : "Arun Mehta"
__v : 0
```

```
_id: ObjectId('69cc05300e8a1ce20b439e54')
email : "sarahj@iitk.ac.in"
name : "Sarah Johnson"
__v : 0
```

```
_id: ObjectId('69cc05300e8a1ce20b439e52')
email : "priyas@iitk.ac.in"
name : "Priya Sharma"
__v : 0
```

```
_id: ObjectId('69ce8067479daa2ee93fd0ab')
email : "vsumit23@iitk.ac.in"
name : ""
__v : 0
```


The admin successfully registers the authorized email in the Professors collection.


```
_id: ObjectId('69cc05300e8a1ce20b439e30')
email : "priyas@iitk.ac.in"
password : "$2b$10$jzoqk9q0YRVpOJ9fWdEU0.bHp0Z9gUZIrZuWlcn8ZdSTv.H6XqBku"
role : "Professor"
username : "prof_2"
isVerified : true
__v : 0
name : "Indranil Saha"
```

```
_id: ObjectId('69cc05300e8a1ce20b439e31')
email : "davidc@iitk.ac.in"
password : "$2b$10$UajavhCureGdNG.nBbZhPOnh4ffwRIsi88h3RUC8OXPEov7QOpZEK"
role : "Professor"
username : "prof_3"
isVerified : true
__v : 0
```


The Professor successfully signs up; the Users collection reflects the authorized role and securely hashed password.

Conclusion: The backend APIs correctly verify the whitelist presence before committing the user document to the database, effectively securing the Professor role.

2) Course Creation & Scheduling

a) Objective

The objective of Implementation Testing is to verify that the backend APIs correctly interact with the database and execute all required operations successfully. It ensures that when a request is sent from the frontend, the system performs the expected actions such as storing data, generating sessions, and maintaining correct relationships between entities.

b) Course Creation API Testing

In this integration test, we verify that the course creation API correctly interacts with the database, stores course data, generates sessions, and maintains proper relationships between entities.
Unit Details: Express Backend (courseRoutes.js, courseController.js), MongoDB (Courses and Sessions Collection)
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:
• Course is successfully created in the database
• Professor ID is correctly stored in the course document
• Sessions are automatically generated after course creation
• Each session contains correct course reference (courseId)
• Co-instructor ID is successfully added to the course
• Duplicate co-instructors are prevented
• Invalid co-instructor email results in error
• API returns correct response (status 201)

Structural Coverage:
Statement and Branch Coverage were used to test API success, failure, and co-instructor linking scenarios.

```
router.post("/", ctrl.createCourse);
```

Route Definition (routes/courseRoutes.js)

Controller Logic (controllers/courseController.js)

```
const course = await Course.create({
  name,
  startDate,
  endDate,
  daysOfWeek,
  professor: professorId
});

await generateSessions(course._id, startDate, endDate, daysOfWeek);
```

Explanation

When a request is sent to the endpoint /api/courses, the following operations are performed:

- The API receives course details such as name, start date, end date, and selected days
- The backend validates the input fields
- A new course document is created in the courses collection
- The system automatically generates session entries using the scheduling algorithm
- Each session is linked to the course using the course ID
- A success response is returned to the frontend

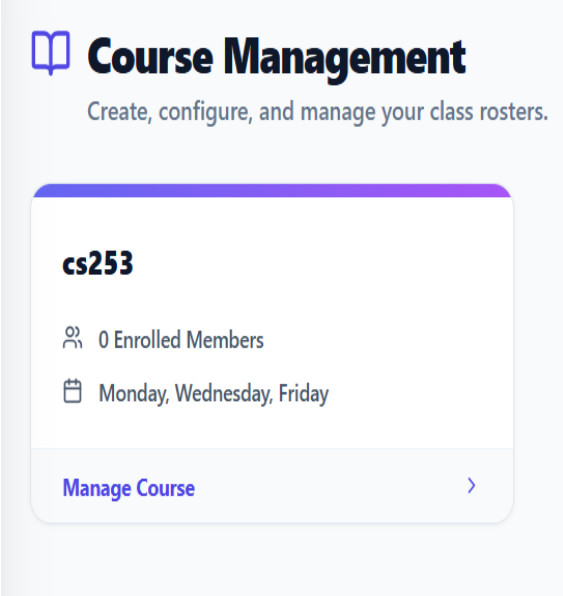
What is Being Verified

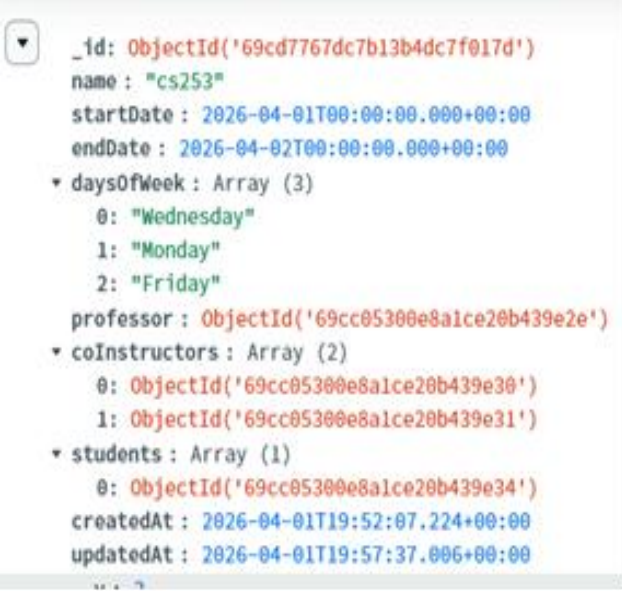
Implementation testing ensures that:

- The API route is functioning correctly
- The course is successfully stored in the database
- Sessions are automatically generated after course creation
- Each session contains the correct course reference (course ID)
- The response returned by the server is correct
- Course contains correct professor ID
- Course contains co-instructor IDs in coInstructors field
- Both professor and co-instructors are linked to same course

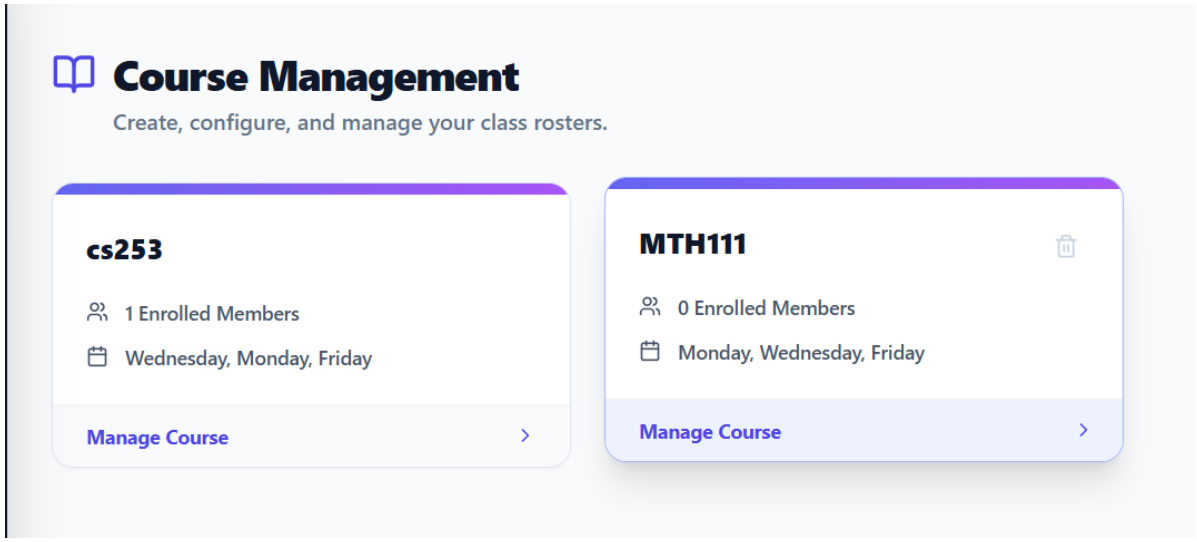
Suppose that I want to create the course MTH111

Before Course Creation of MTH111



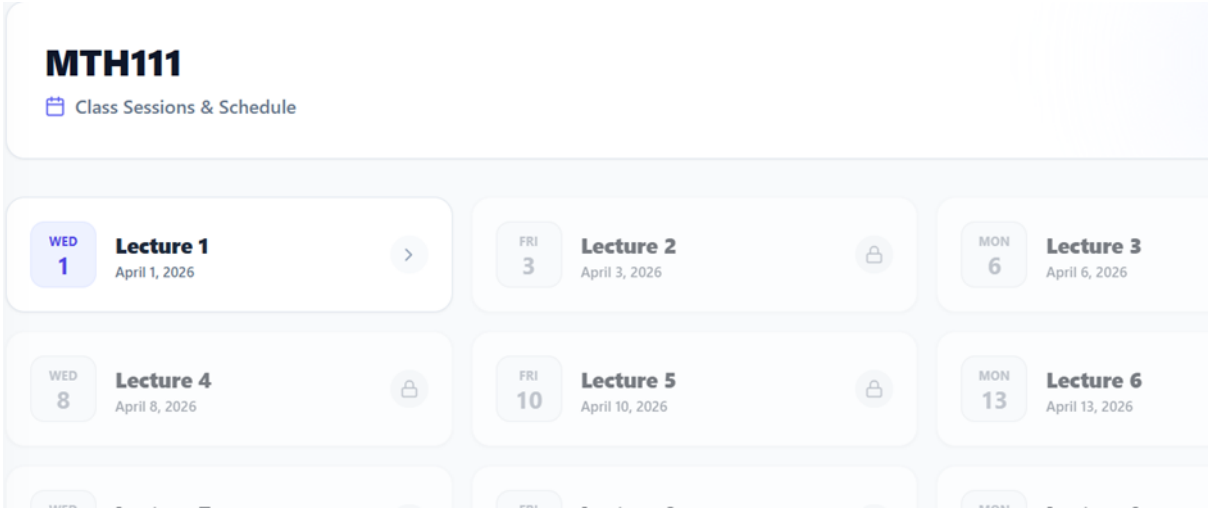


After course creation of MTH111

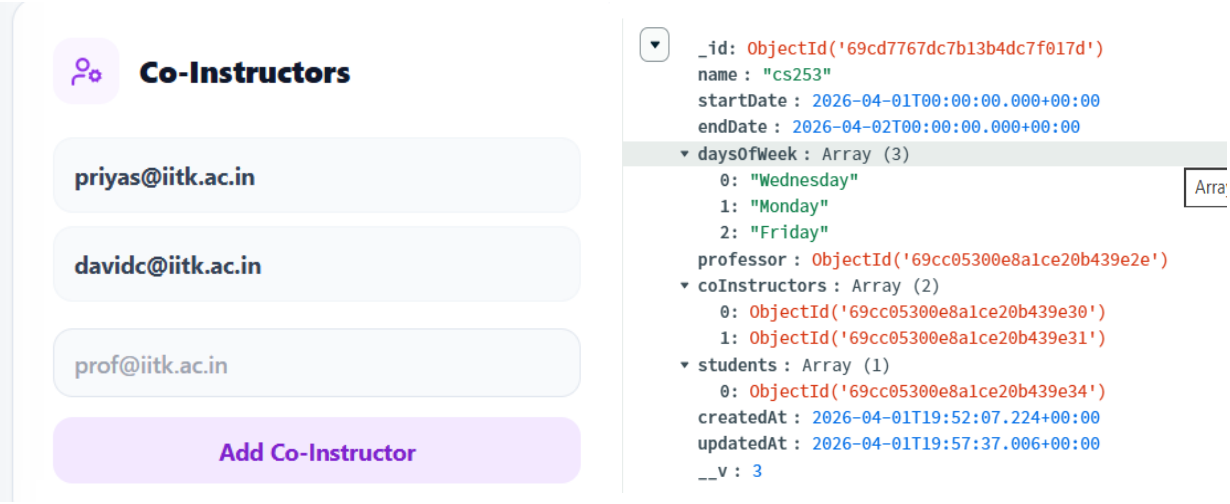




Course MTH111 is successfully stored in database and professor-Id is also showing in the database .



Sessions generated automatically based on selected days and linked to the course.



After adding the co-instructor, the coInstructors field in the course document is updated with the corresponding professor IDs. This confirms that multiple instructors are correctly linked to the same course.

c) API Test Code

In this integration test, we verify that the API endpoint for course creation correctly processes requests and returns expected responses.
Unit Details: Express Backend API (/api/courses), Testing Tool (Supertest / Postman)
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:
• API successfully accepts valid input and creates course

• Returns status code 201 for successful creation

• Returns error for invalid or incomplete input
• Ensures proper request-response cycle

Structural Coverage:
Statement and Branch Coverage were used to test both successful and failure API responses

```
const request = require('supertest');
const app = require('../server');

test('course should be created and sessions generated', async () => {
  const res = await request(app)
    .post('/api/courses')
    .send({
      name: "Software Engineering",
      startDate: "2026-01-01",
      endDate: "2026-01-10",
      daysOfWeek: ["Monday", "Wednesday"]
    });

  expect(res.statusCode).toBe(201);
  expect(res.body.msg).toBe("Course created and sessions scheduled");
});
```

Explanation of Test Case

- Sends a request to create a course
- Verifies that:
  - API returns status code **201**
  - Course is created successfully
  - Sessions are generated automatically
- Confirms correct integration between API and database

d) Result

The implementation testing confirms that:

- Backend APIs are working correctly
- Course data is properly stored in the database
- Sessions are automatically generated
- Relationships between course and sessions are maintained
- The system returns correct responses for valid inputs

e) Conclusion

Implementation testing successfully validates the integration between frontend requests, backend logic, and database operations. The system ensures accurate data storage, automatic schedule generation, and correct linking of related entities.

3) Student Enrollment

A.Fetching Managed Courses from the Database

Module Details: This module covers the retrieval of courses from the MongoDB database that are currently assigned to the authenticated professor, to be displayed on the 'Manage Courses' section of the UI.

Test Owner: DevAxis

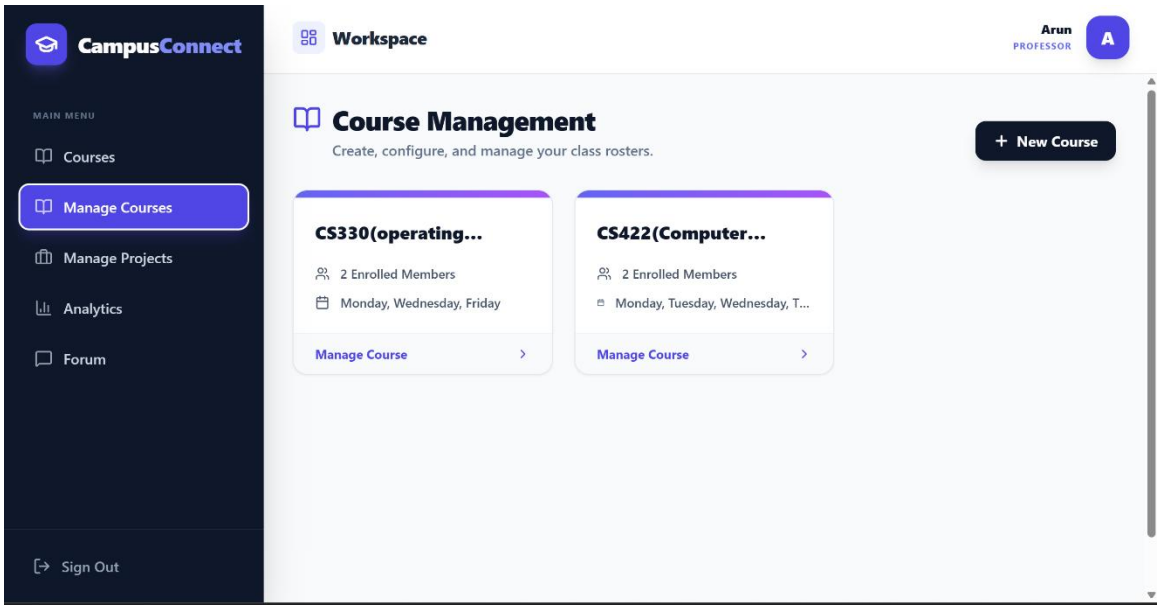
Test Date: April 03, 2026

Test Results: The database contains the following course documents:-


```
_id: ObjectId('69cd669fba517dfde7708531')
name : "CS422(Computer Architecture)"
startDate : 2026-03-02T00:00:00.000+00:00
endDate : 2026-05-14T00:00:00.000+00:00
▸ daysOfWeek : Array (7)
professor : ObjectId('69cc05300e8a1ce20b439e2f')
▸ coInstructors : Array (empty)
▸ students : Array (2)
createdAt : 2026-04-01T18:40:31.493+00:00
updatedAt : 2026-04-02T11:29:40.098+00:00
__v : 2
```

```
_id: ObjectId('69cd62f4ba517dfde7708469')
name : "CS330(operating system)"
startDate : 2026-08-01T00:00:00.000+00:00
endDate : 2026-11-28T00:00:00.000+00:00
▸ daysOfWeek : Array (3)
professor : ObjectId('69cc05300e8a1ce20b439e2f')
▸ coInstructors : Array (empty)
▸ students : Array (2)
createdAt : 2026-04-01T18:24:52.472+00:00
updatedAt : 2026-04-01T18:25:39.854+00:00
__v : 2
```

In the Course Management section, the frontend successfully fetches and maps this data:



Conclusion: The fetching of the managed courses and the mapping of the enrolled student arrays and schedules to the UI cards is working correctly.

B Manual Student Enrolment Persistence

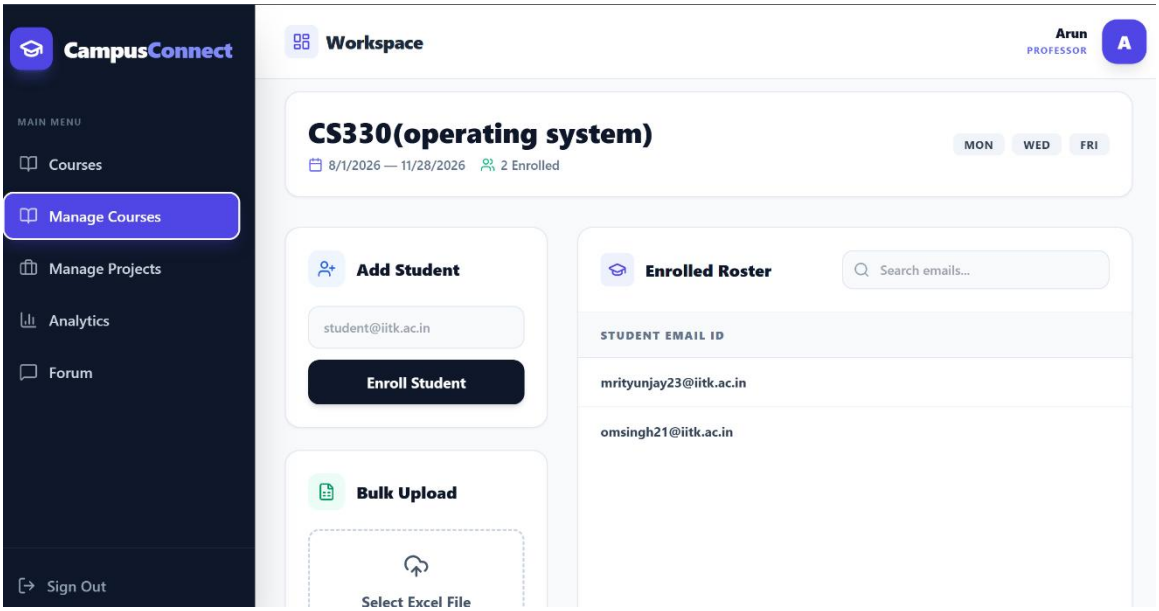
Module Details: This module integrates the manual student enrolment form with the backend, ensuring that the student email entered by the professor gets securely stored in the MongoDB database under the respective course roster.

Test Owner: DevAxis

Test Date: April 03, 2026

Test Results:

Upon selecting a course, the professor is presented with the enrolment management dashboard.



The Professor fills in the student's institutional email ID in the 'Add Student' form and clicks on 'Enroll Student'.

The details successfully get stored in the database, appending the student's unique ID to the course's students array.

```
_id: ObjectId('69cd62f4ba517dfde7708469')
name : "CS330(operating system)"
startDate : 2026-08-01T00:00:00.000+00:00
endDate : 2026-11-28T00:00:00.000+00:00
▸ daysOfWeek : Array (3)
professor : ObjectId('69cc05300e8a1ce20b439e2f')
▸ coInstructors : Array (empty)
▾ students : Array (2)
    0: ObjectId('69cd5d7bd42998a93996fedd')
    1: ObjectId('69cc05300e8a1ce20b439e34')
createdAt : 2026-04-01T18:24:52.472+00:00
updatedAt : 2026-04-01T18:25:39.854+00:00
__v : 2
```

onclusion: The manual enrolment integration is working correctly. The frontend successfully validates the input, the API processes the request, and the MongoDB database persists the new student record within the designated course document

C. Bulk Enrolment via Excel Upload

Module Details: This module verifies the integration of the file-stream pipeline. It ensures that .xlsx files uploaded by the professor are correctly processed by the backend parser, duplicate entries are skipped, and valid students are synchronized with the database in bulk.

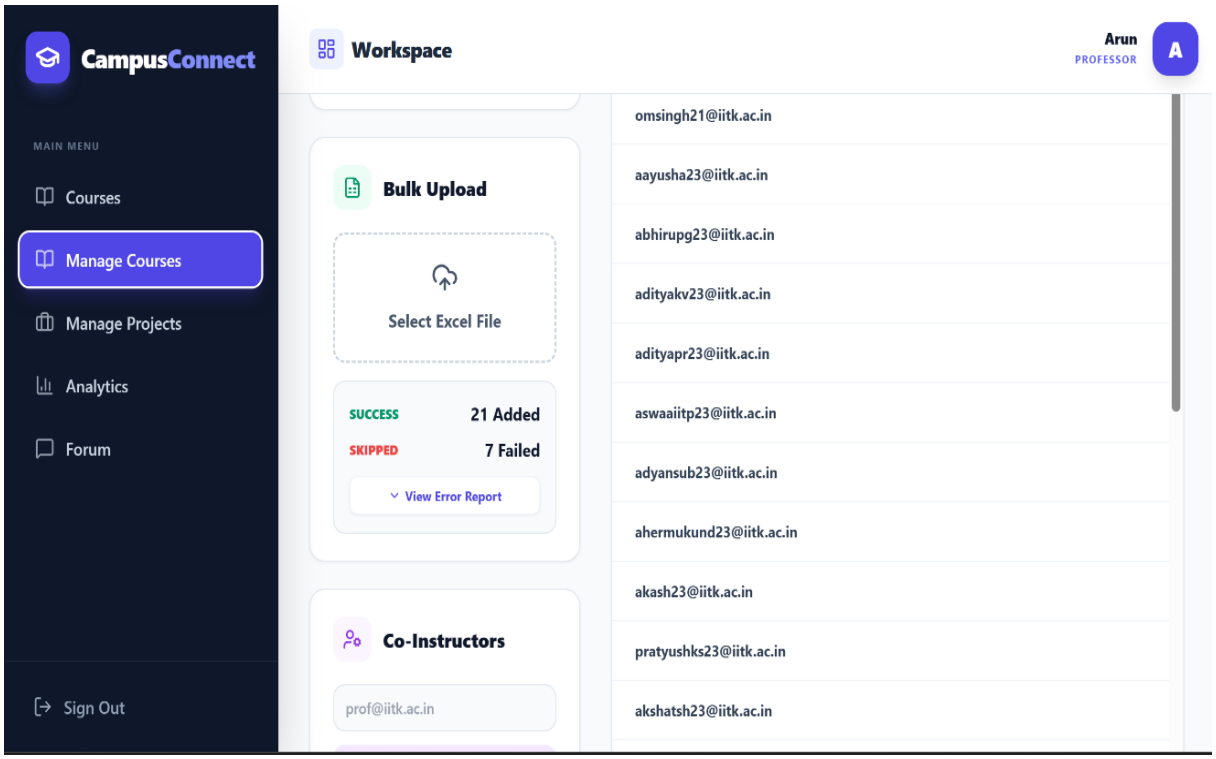
Test Owner: DevAxis

Test Date: April 03, 2026

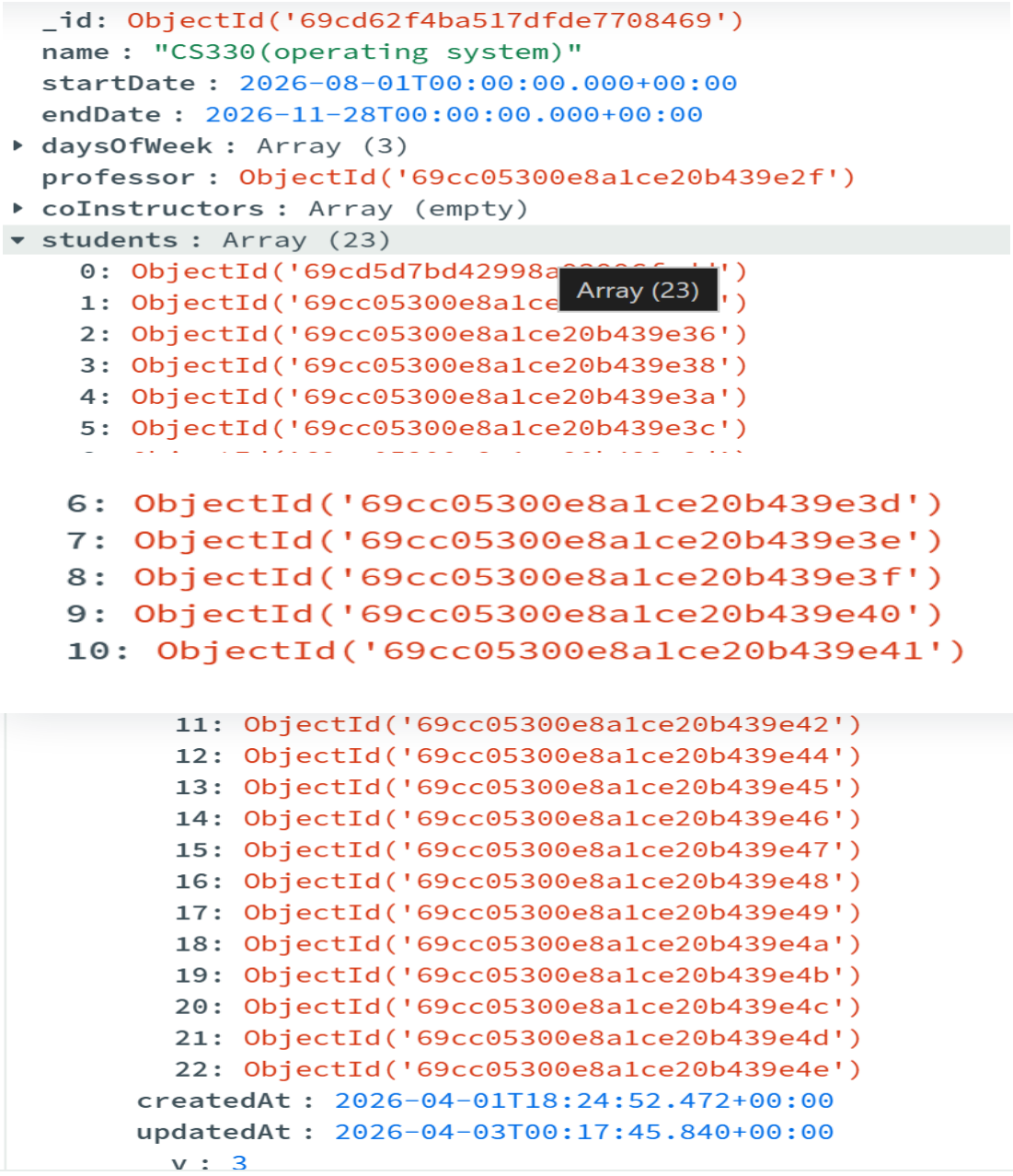
Test Results:

The Professor navigates to a managed course and uploads a .xlsx file containing a roster of student emails via the 'Bulk Upload' section.

Upon successful processing, the system provides a summary report directly in the UI, detailing the number of students successfully added versus those skipped (e.g., due to duplicate entries). The 'Enrolled Roster' immediately updates to reflect the newly added students.



The database accurately reflects this bulk operation.



Conclusion: The bulk upload file-stream integration is highly stable. The backend successfully parses the multipart form data, validates the entries against the existing unique indices, and updates the MongoDB students array in a single transaction.

D.Security & Authentication Integration

Module Details: This module ensures that all backend API routes for course management and student enrolment are protected by JWT (JSON Web Token) authentication middleware.
Test Owner: DevAxis
Test Date: April 03, 2026
Test Results:

- **Valid Session:** When the Professor navigates the UI, the frontend successfully attaches the Authorization: Bearer <token> header to the API requests. The backend validates the token and processes the roster updates.
- **Unauthorized Access:** If the local storage token is manually cleared or expired, attempting to enroll a student results in the backend blocking the request and returning a 401 Unauthorized status.

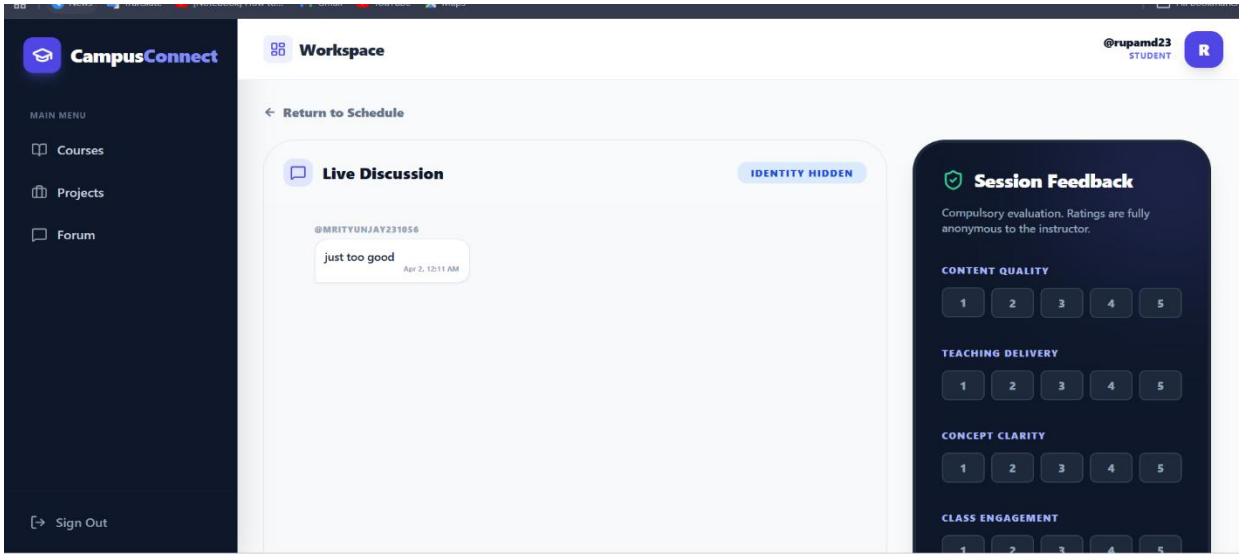
Conclusion: The security handshake between the React frontend headers and the Node.js authentication middleware is functioning correctly, ensuring only authorized professors can modify course rosters .

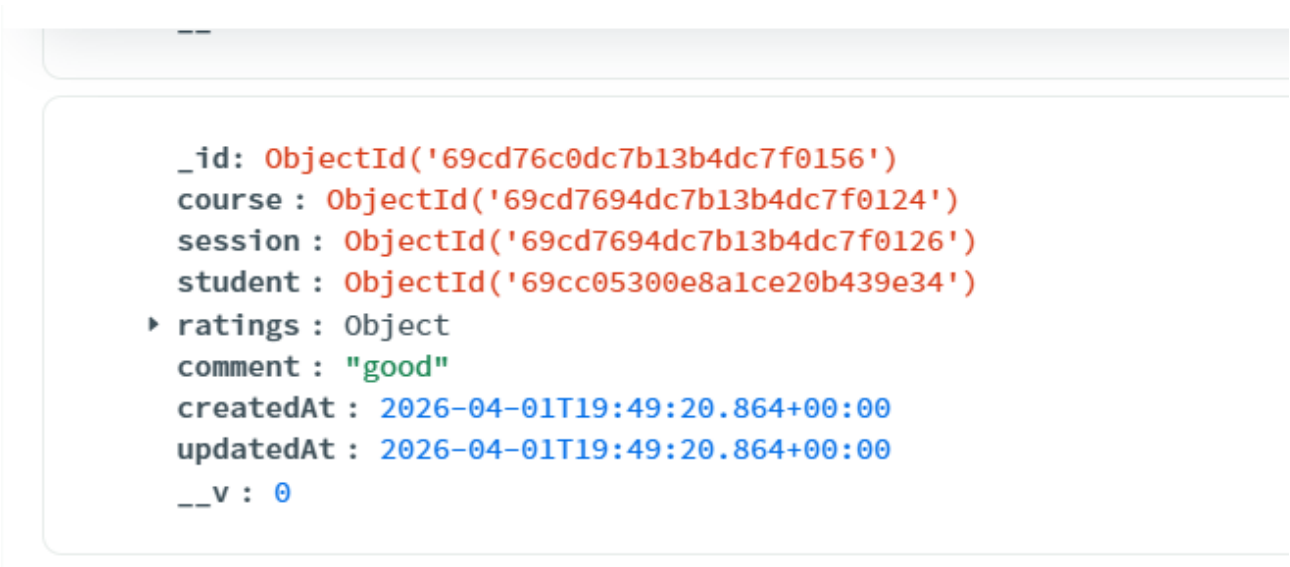
4)Class Discussions & Feedback

A. Real-Time Comment Update Mechanism

[Include basic information about what was tested and what happened.] The real-time communication system using Socket.IO was tested to ensure that when a user posts a comment, it is instantly broadcasted to all connected users without requiring a page refresh. The system uses event-based messaging (receive_message) to update the UI dynamically.
Test Owner: DevAxis
Test Date: 3/04/2026
Test Results:

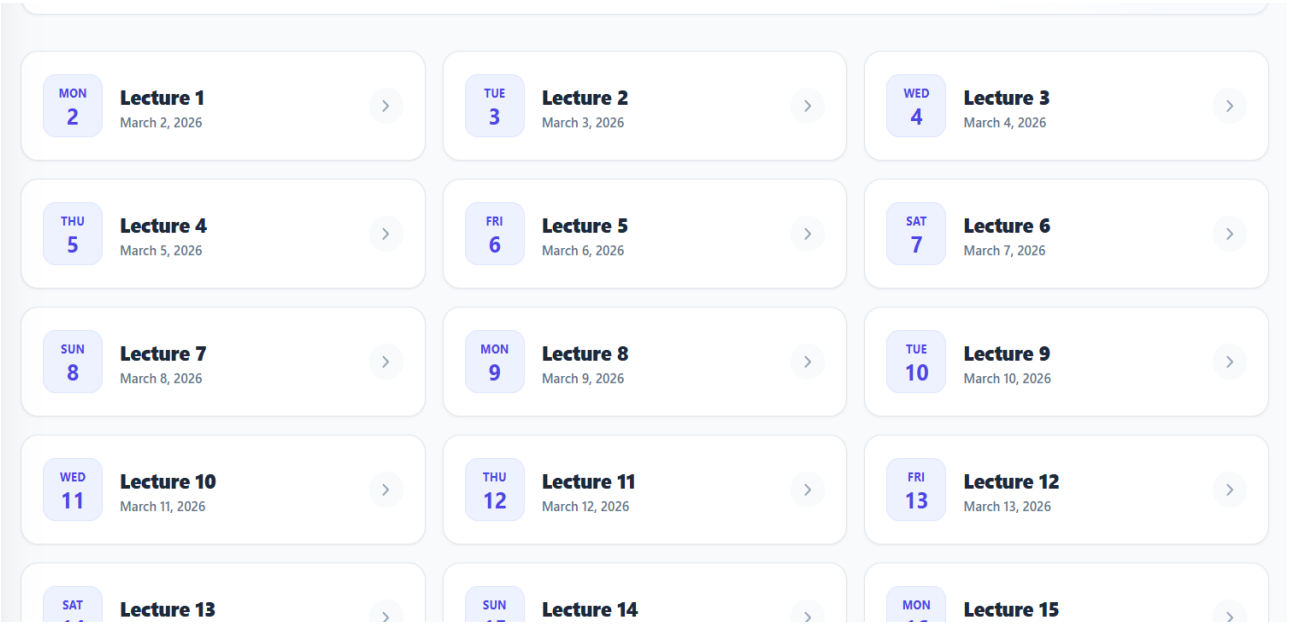
- Comments appeared instantly for all users connected to the same session
- No manual refresh was required
- Event receive_message triggered correctly on new comment
- Multiple users tested simultaneously → no message delay or loss
- Backend successfully emitted message to correct session room





B. **Comment Posting & API Integration :** [Include basic information about what was tested and what happened.]
Tested integration between frontend input and backend API for posting comments, ensuring correct data flow, validation, and response handling.
Test Owner: DevAxis
Test Date: 3/04/2026
Test Results:

- Comments successfully stored in database
- Response returned with populated user and reply data
- Reply functionality worked correctly
- Invalid inputs (empty text) were rejected



C. **Feedback Submission & Validation Flow**:

[Include basic information about what was tested and what happened.]

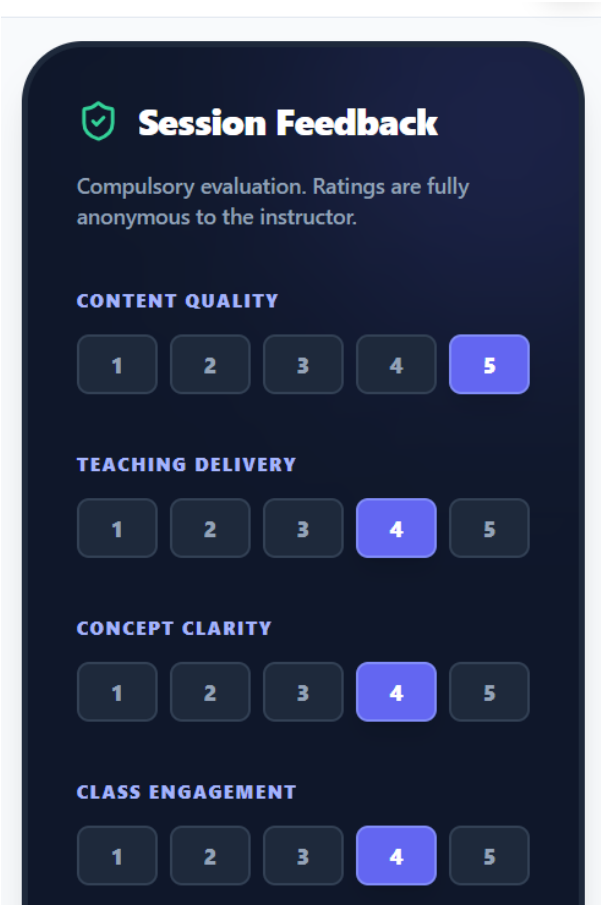
Tested the integration of feedback submission system including validation, duplicate prevention, and backend storage.

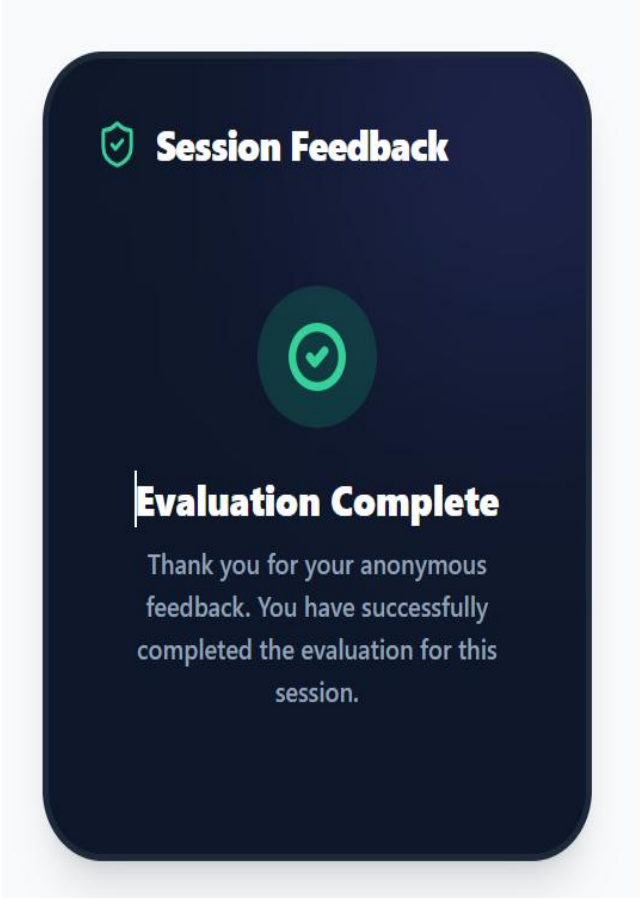
Test Owner: DevAxis

Test Date: 3/04/2026

Test Results:

- Feedback submitted successfully with valid inputs
- Duplicate submissions blocked correctly
- Missing fields resulted in validation errors
- Data stored correctly in database



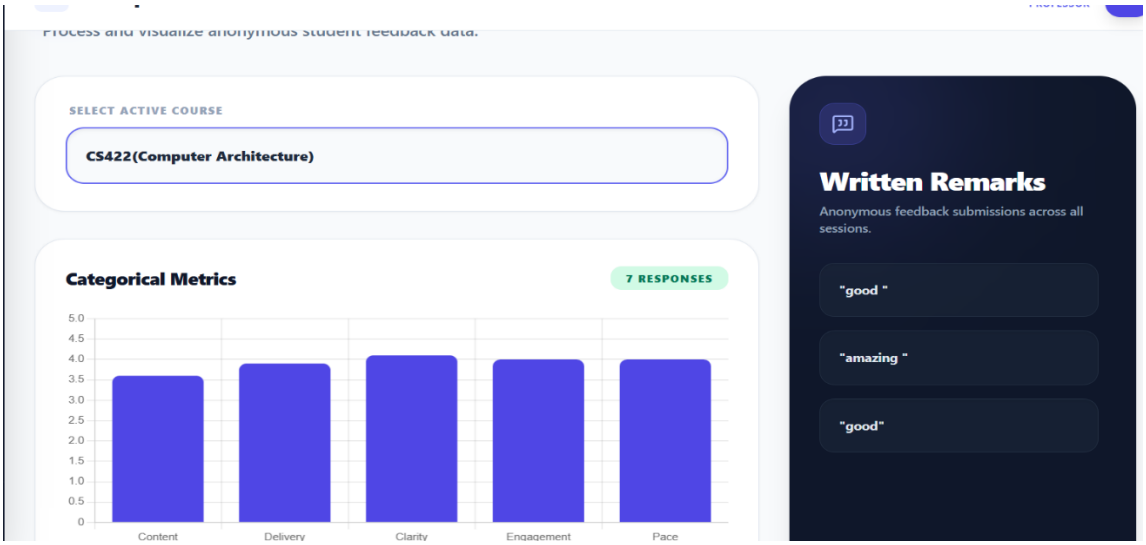


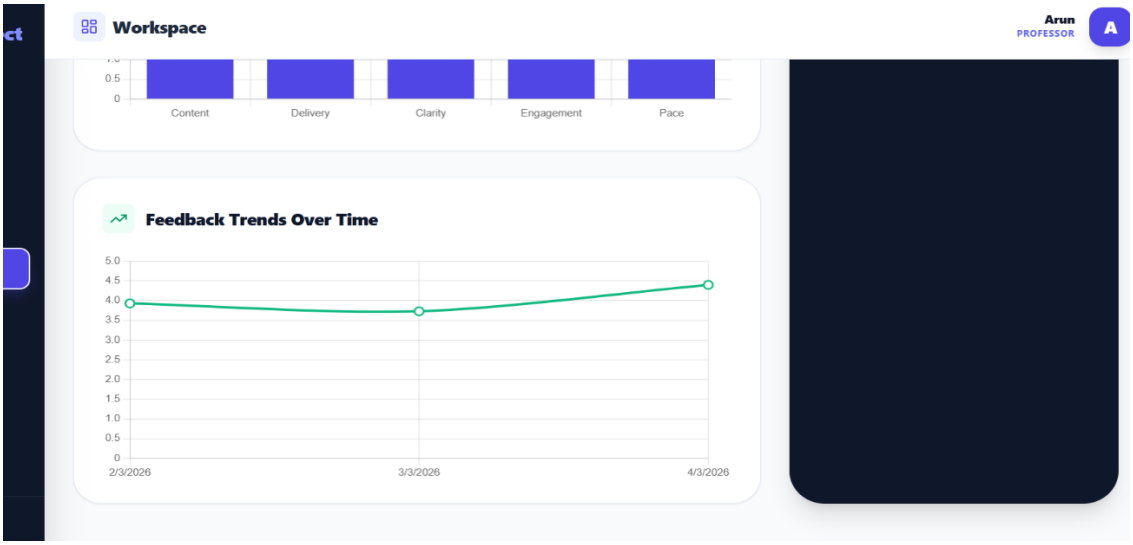
D. Analytics Data Processing & Visualization :

[Include basic information about what was tested and what happened.]
Tested the integration between backend analytics computation and frontend visualization using charts. Ensured correct calculation of averages and trend data.
Test Owner: DevAxis
Test Date: 3/04/2026

Test Results:

- Correct averages displayed in bar chart
- Trend line updated correctly over session dates
- Written feedback displayed properly
- “No Data Available” shown when no feedback exists





5)Manage Projects (Professor Side)

A. Create Project API

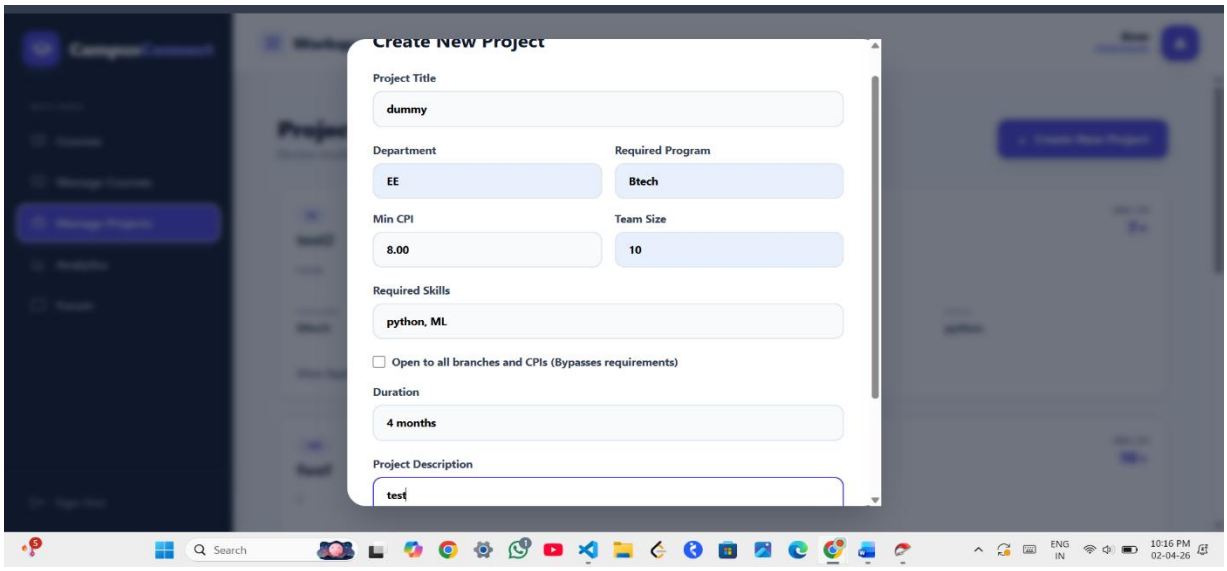
Objective : To verify that:

- Project creation API correctly validates input
- Rejects invalid data (CPI, missing fields)
- Stores valid project in database

Endpoint Details :

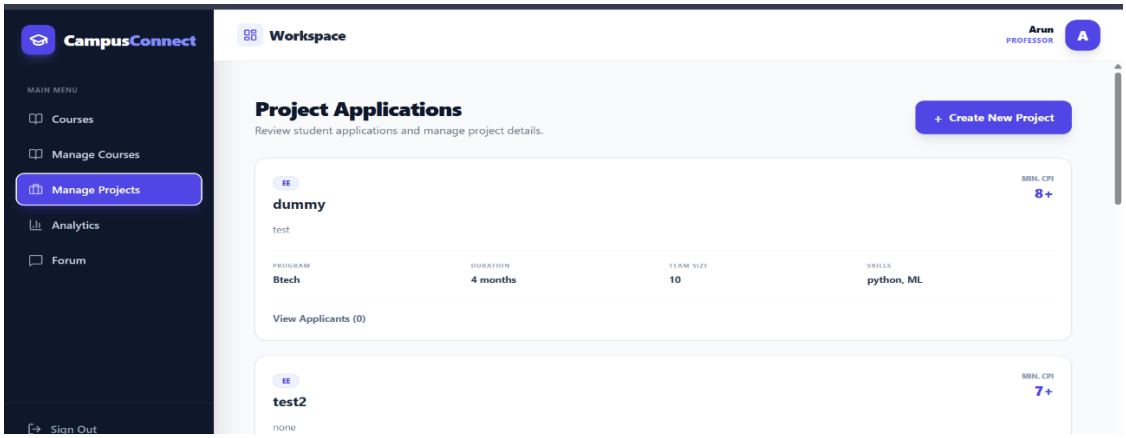
- Method: POST
- URL: /api/projects

Test Case 1.1 – Valid Project Creation

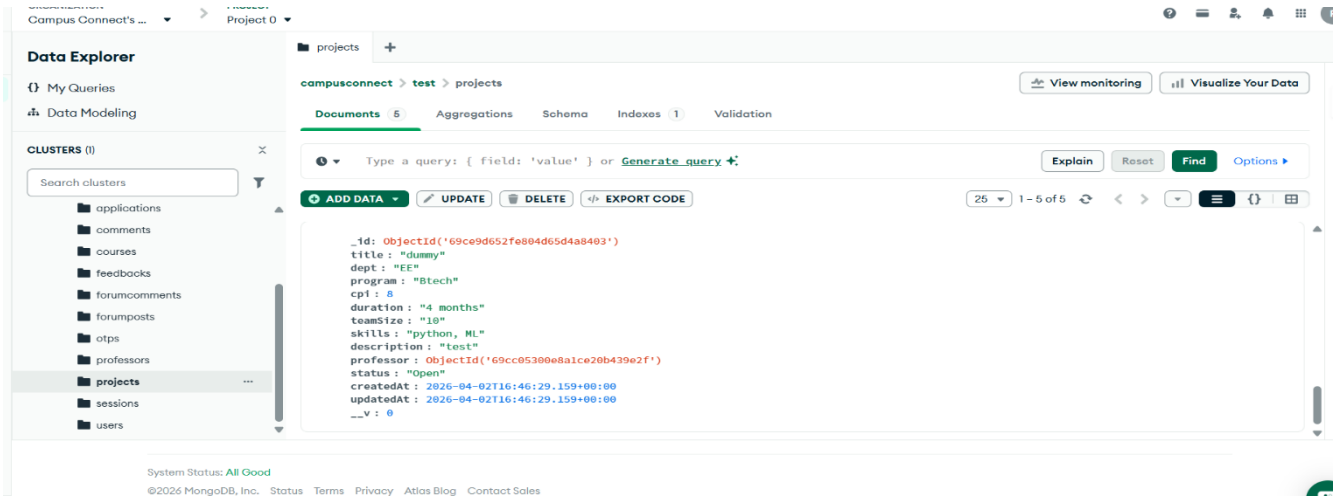


Expected Result

- Status: 200 OK
- Project saved in DB

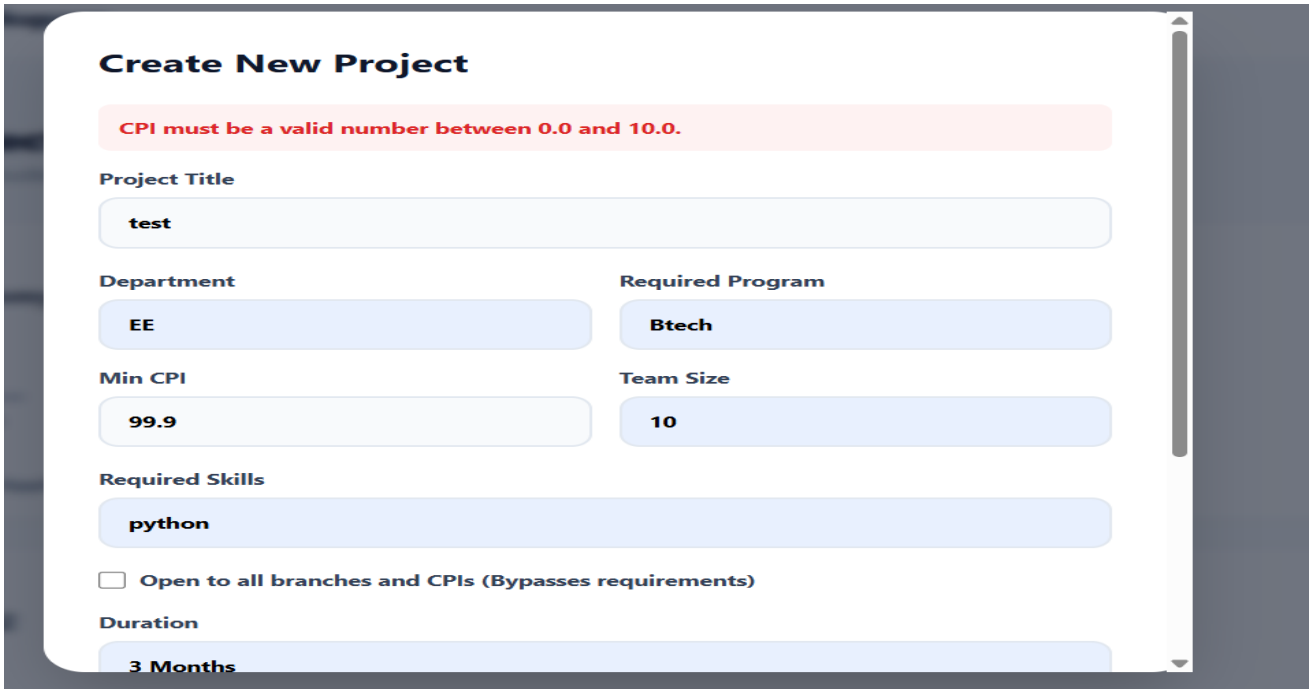


Data Stored in DB



Test Case 1.2 – Invalid CPI (Out of Range)

If input is out of range



Expected Result

- Status: 400 Bad request
- Invalid CPI.

B. Fetch Projects (GET/api/projects/managed)

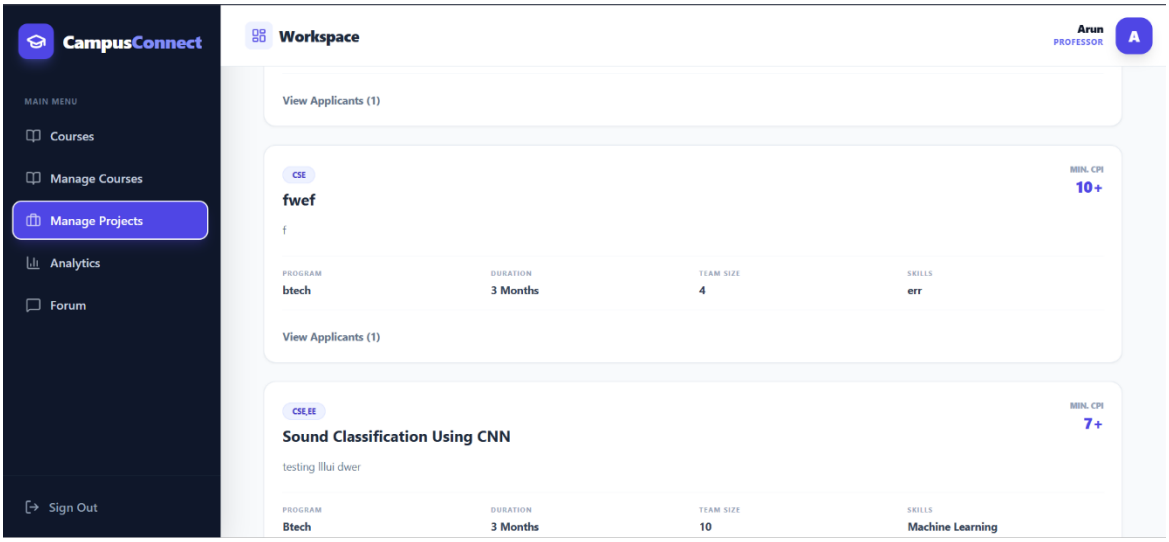
Objective To verify that:

- Project creation API correctly validates input
- Rejects invalid data (CPI, missing fields)
- Stores valid project in database

Endpoint Details

- Method: GET
- URL: /api/projects/managed

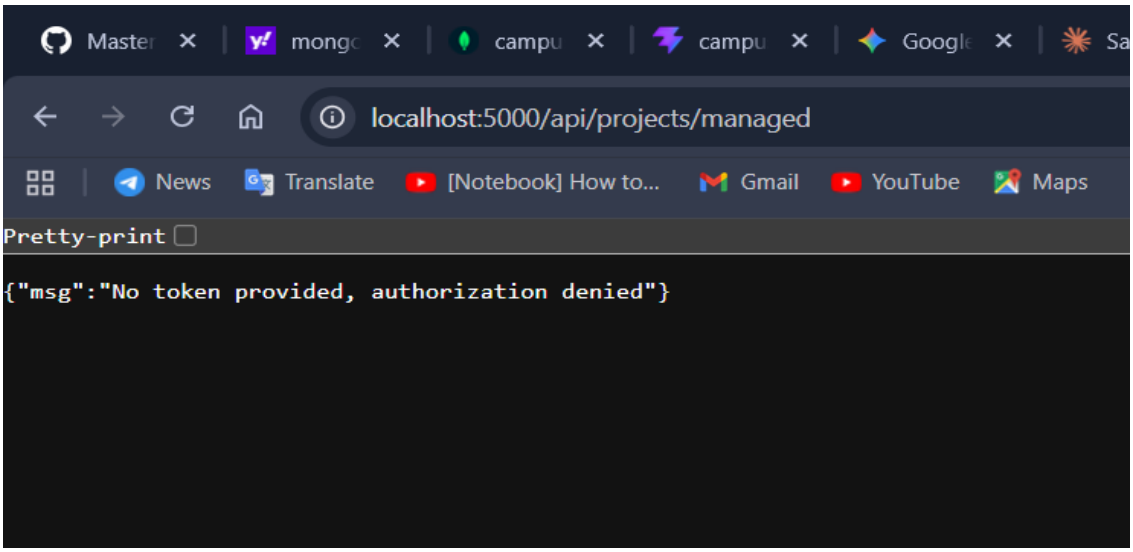
Test Case 2.1 – Fetch All Projects



Expected Result

- Status: 200
- Return array of projects

Test Case 2.3 – Unauthorized Access



Expected Result

- Status: 401 Unauthorized

C. Update application Status

Objective To verify that:

- Status updates correctly in DB
- Invalid inputs are handled safely

Endpoint Details

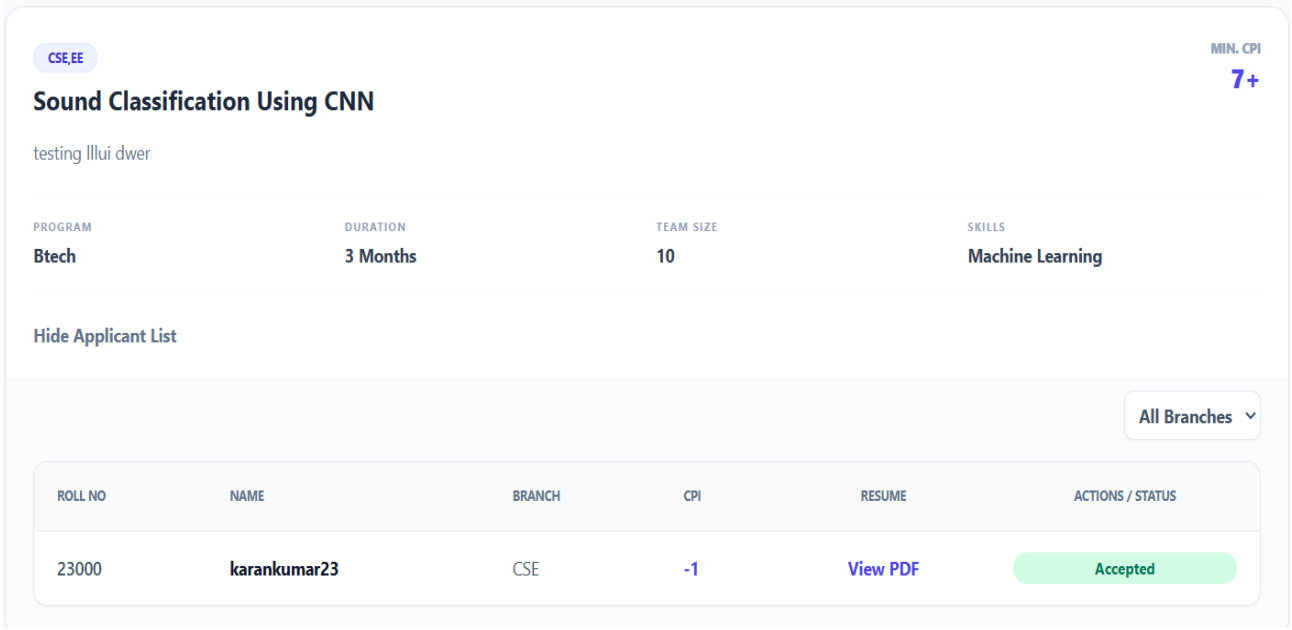
- Method: PUT
- URL: /api/projects/applications/:appId

Test Case 3.1 – Accept Candidate

Expected Result

- Status: 200
- DB updated to “Accepted”


```
_id: ObjectId('69cd5dde861d33ebca1118f6')
project : ObjectId('69cd4c752fe804d65d4a8372')
student : ObjectId('69cd4b05861d33ebca1118d2')
rollNo : "23000"
name : "karankumar23"
branch : "CSE"
cpi : -1
resume : "https:\\something"
sop : "f"
status : "Accepted"
createdAt : 2026-04-01T18:03:10.609+00:00
updatedAt : 2026-04-01T18:15:14.116+00:00
__v : 0
```

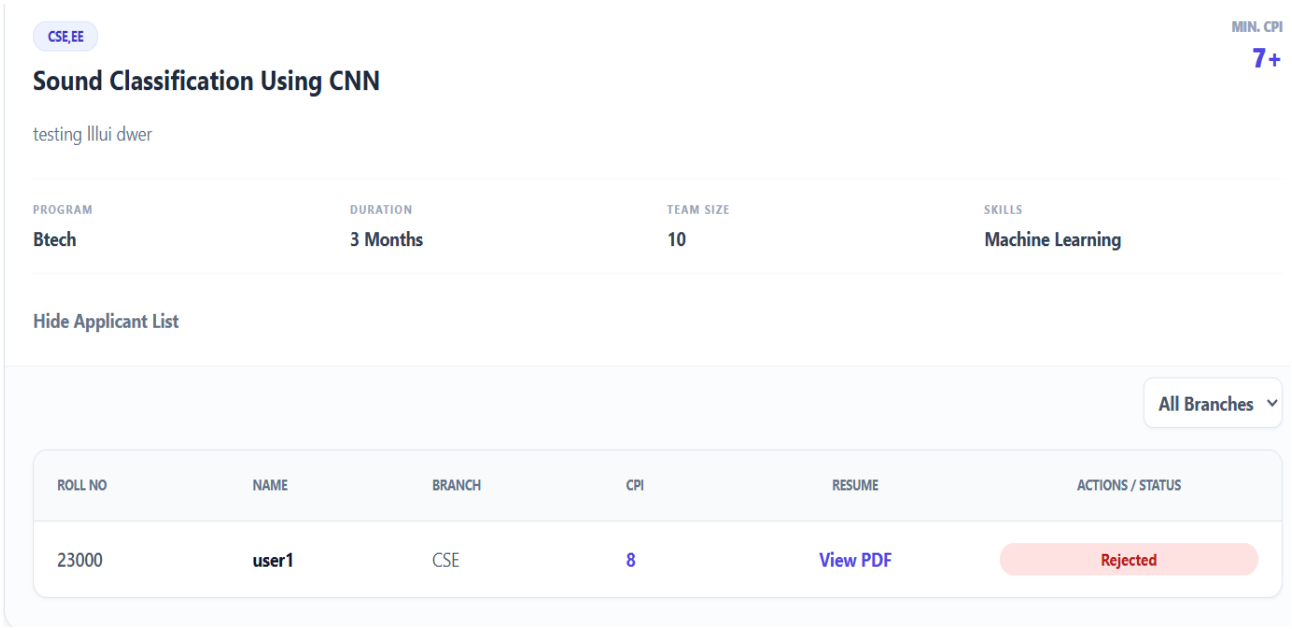


Test Case 3.2 – Reject Candidate

Expected Result

- Status: 200
- DB updated to “Rejected”


```
_id: ObjectId('69cea93f2a497eacc9916c60')
project : ObjectId('69cd4c752fe804d65d4a8372')
student : ObjectId('69cd4b05861d33ebca1118d2')
rollNo : "23000"
name : "user1"
branch : "CSE"
cpi : 8
resume : "https://drive.google.com/file/d/1a2b3c4d5e6f7g8h9i0j/view?usp=sharing"
sop : ""
status : "Rejected"
createdAt : 2026-04-02T17:37:03.764+00:00
updatedAt : 2026-04-02T17:37:15.538+00:00
__v : 0
```

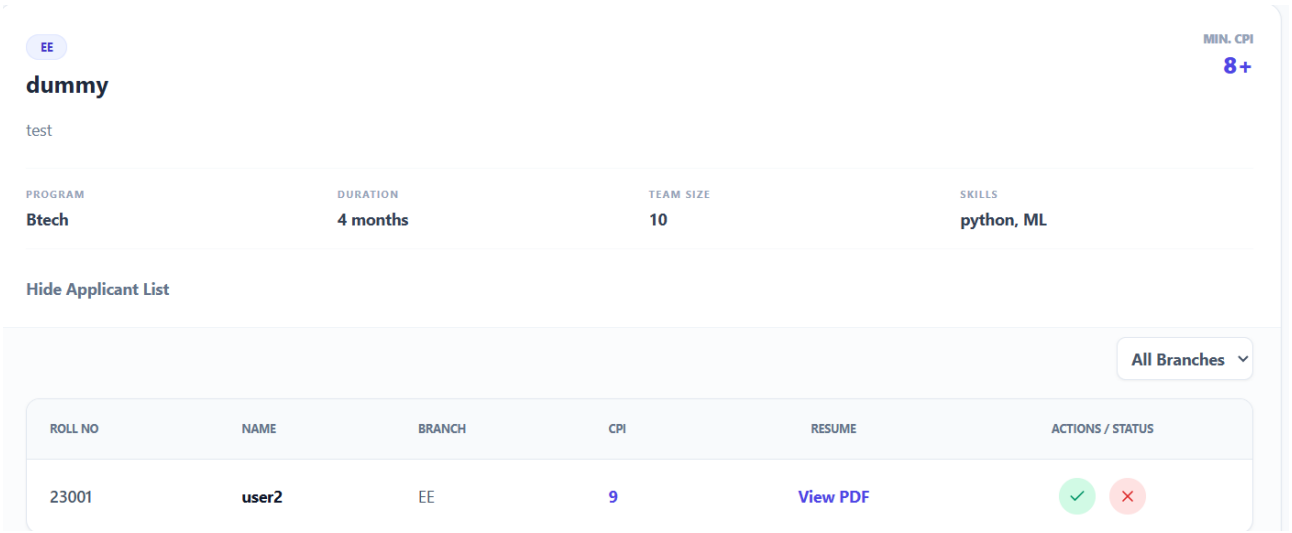


Test Case 3.1 – Pending Candidate

Expected Result

- **Status: 200**
- **DB shows "Pending"**


```
_id: ObjectId('69ceaa252a497eacc9916c69')
project : ObjectId('69ce9d652fe804d65d4a8403')
student : ObjectId('69cd4b05861d33ebca1118d2')
rollNo : "23001"
name : "user2"
branch : "EE"
cpi : 9
resume : "https://example.com/resumes/student-230123.pdf"
sop : ""
status : "Pending"
createdAt : 2026-04-02T17:40:53.640+00:00
updatedAt : 2026-04-02T17:40:53.640+00:00
__v : 0
```



D. Fetch Applications by Branch

Objective To verify that:

- API correctly filters students by branch
- Invalid inputs are handled safely

Endpoint Details

- Method: GET
- URL: /api/applications?branch=

Test Case 4.1 – Fetch a specific Branch

Expected Result

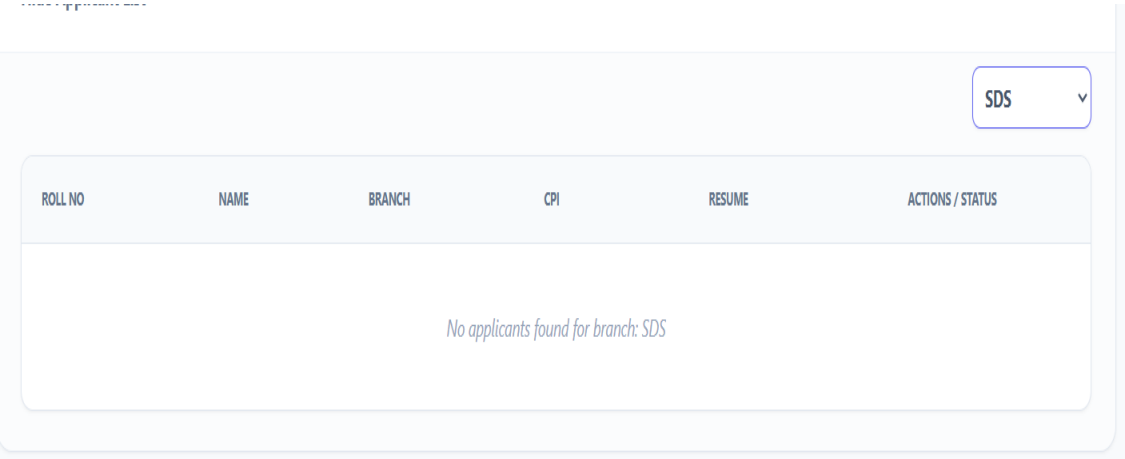
- Status: 200
- Only EE students returned



Test Case 4.2 – No Matching Branch

Expected Result

- Return empty array [ ]
- Shows No applicants found



6)Project- Student Side

A. Fetching Projects from the database

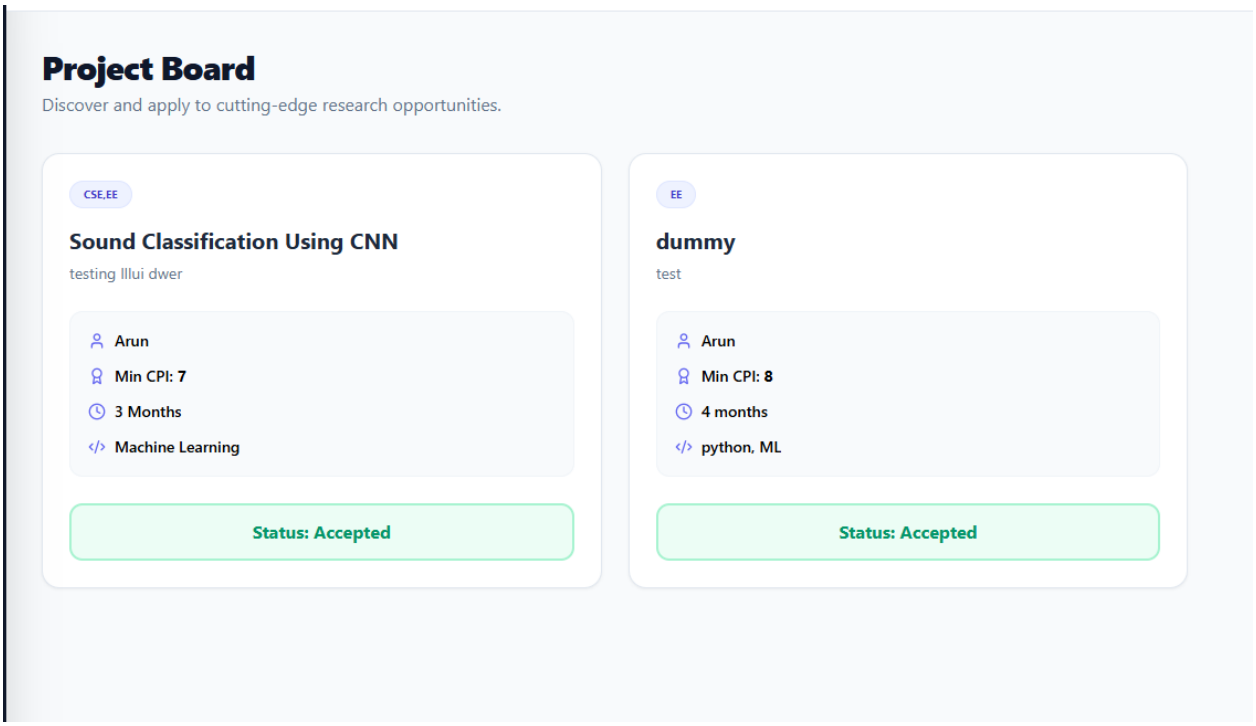
Module Details: This module covers the fetching of the projects from the database to be shown on the Projects section of UI.
Test Owner: DevAxis
Test Date: April 02, 2026
Test Results:

The database contains the following projects.


```
_id: ObjectId('69cd4c752fe804d65d4a8372')
title : "Sound Classification Using CNN"
dept : "CSE,EE"
program : "Btech"
cpi : 7
duration : "3 Months"
teamSize : "10"
skills : "Machine Learning"
description : "testing lllui  dwer "
professor : ObjectId('69cc05300e8a1ce20b439e2f')
status : "Open"
createdAt : 2026-04-01T16:48:53.336+00:00
updatedAt : 2026-04-01T16:48:53.336+00:00
__v : 0
```

```
_id: ObjectId('69ce9d652fe804d65d4a8403')
title : "dummy"
dept : "EE"
program : "Btech"
cpi : 8
duration : "4 months"
teamSize : "10"
skills : "python, ML"
description : "test"
professor : ObjectId('69cc05300e8a1ce20b439e2f')
status : "Open"
createdAt : 2026-04-02T16:46:29.159+00:00
updatedAt : 2026-04-02T16:46:29.159+00:00
__v : 0
```


In the Projects section, the frontend is rendered as such:



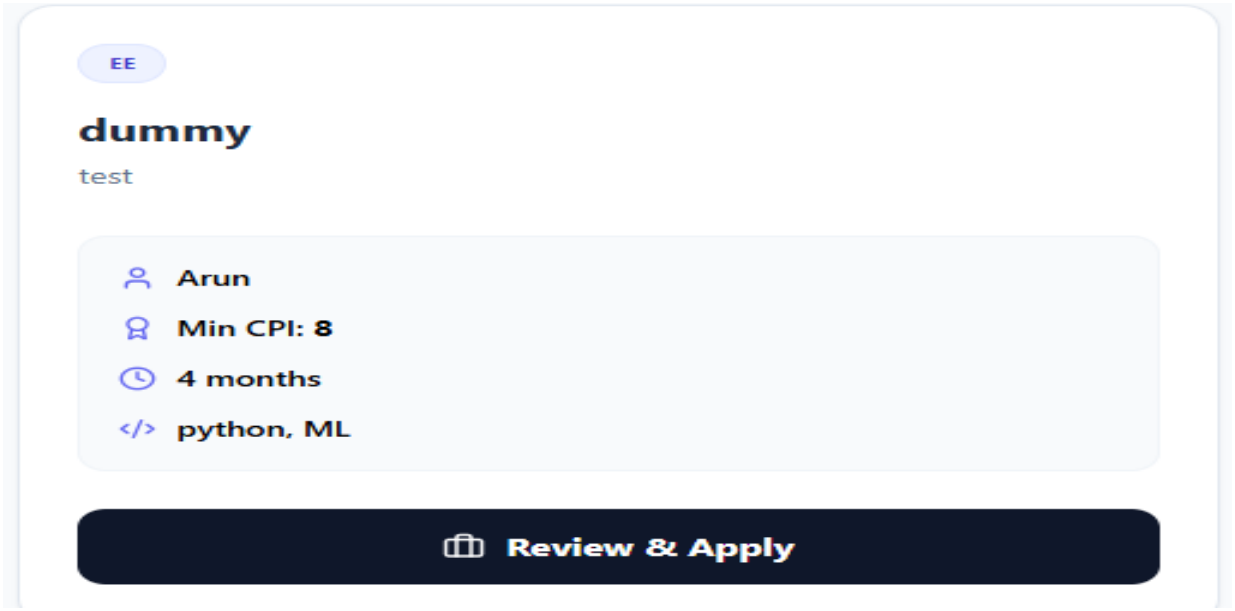
Thus, the fetching of the projects is working correctly.

B. Applying for a project :

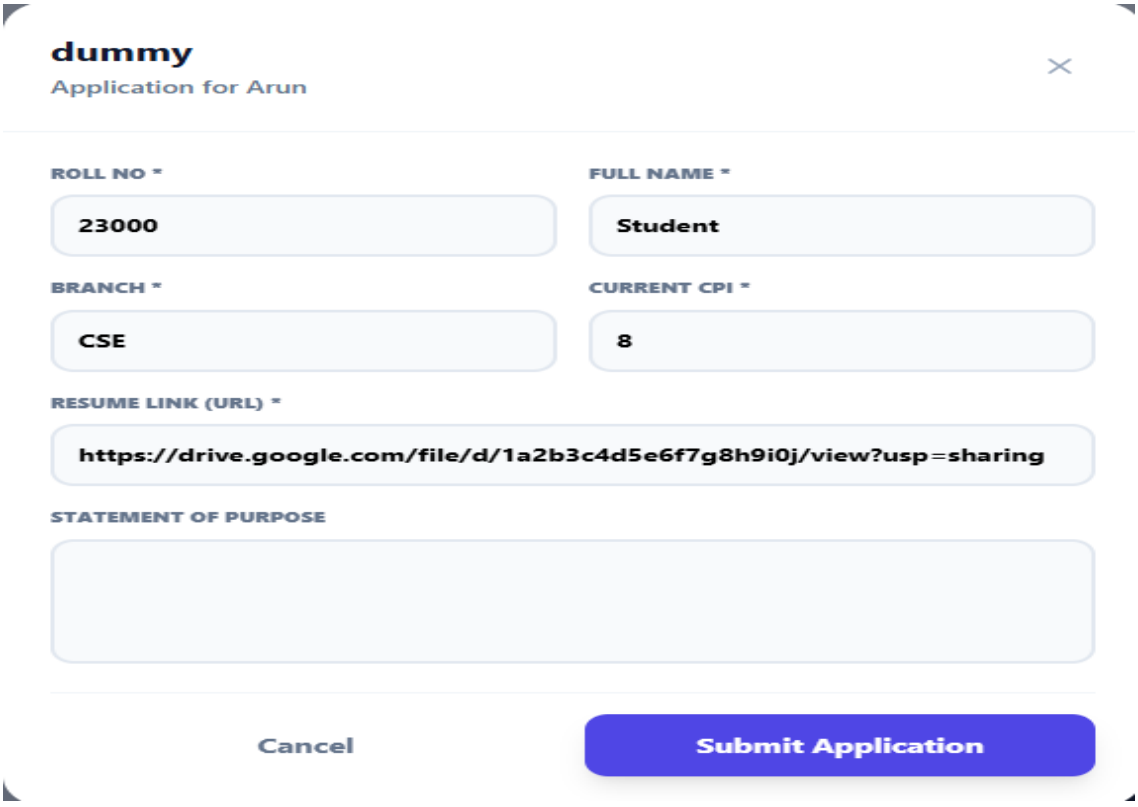
Module Details: This module integrates the application form of a student for a project with the database and shows that the details entered by the student gets stored in the MongoDB database.

Test Owner: DevAxis

Test Date: April 02, 2026
Test Results:



Student fills in his/her details after clicking on ‘Review & Apply’.



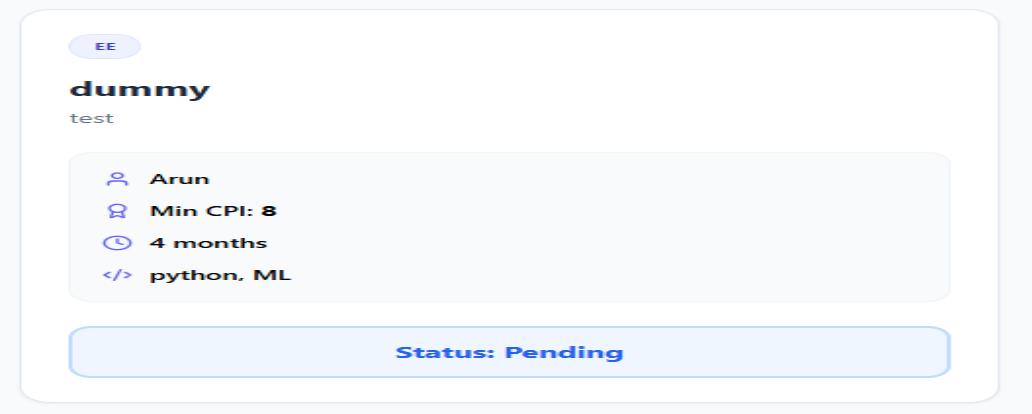
Then ‘Submit Application’ is pressed.
The details get stored in the database

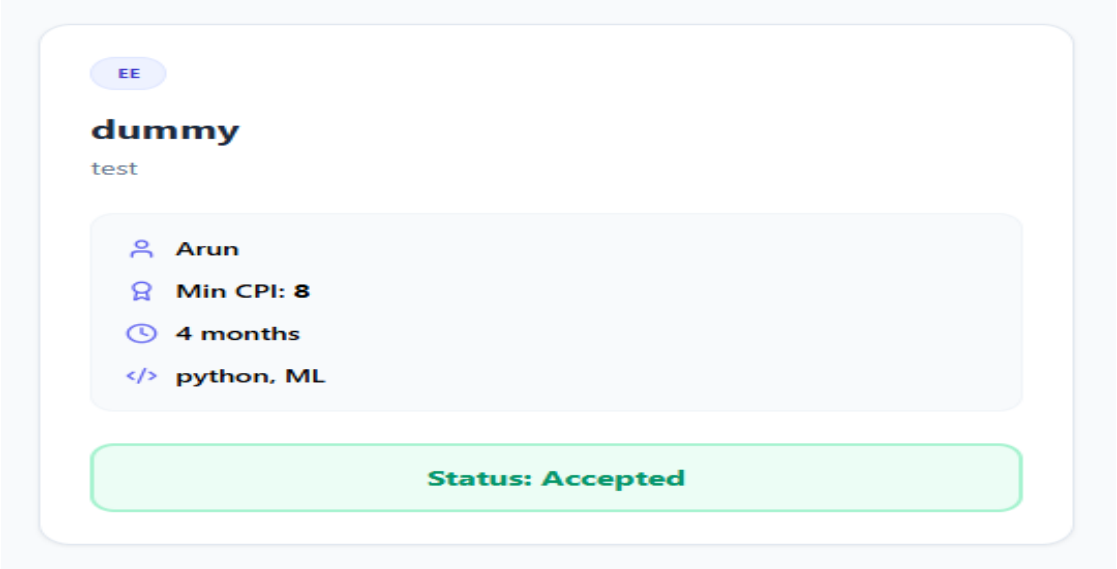
```
_id: ObjectId('69ce9d652fe804d65d4a8403')
title : "dummy"
dept : "EE"
program : "Btech"
cpi : 8
duration : "4 months"
teamSize : "10"
skills : "python, ML"
description : "test"
professor : ObjectId('69cc05300e8a1ce20b439e2f')
status : "Open"
createdAt : 2026-04-02T16:46:29.159+00:00
updatedAt : 2026-04-02T16:46:29.159+00:00
__v : 0
```


C. Status of application

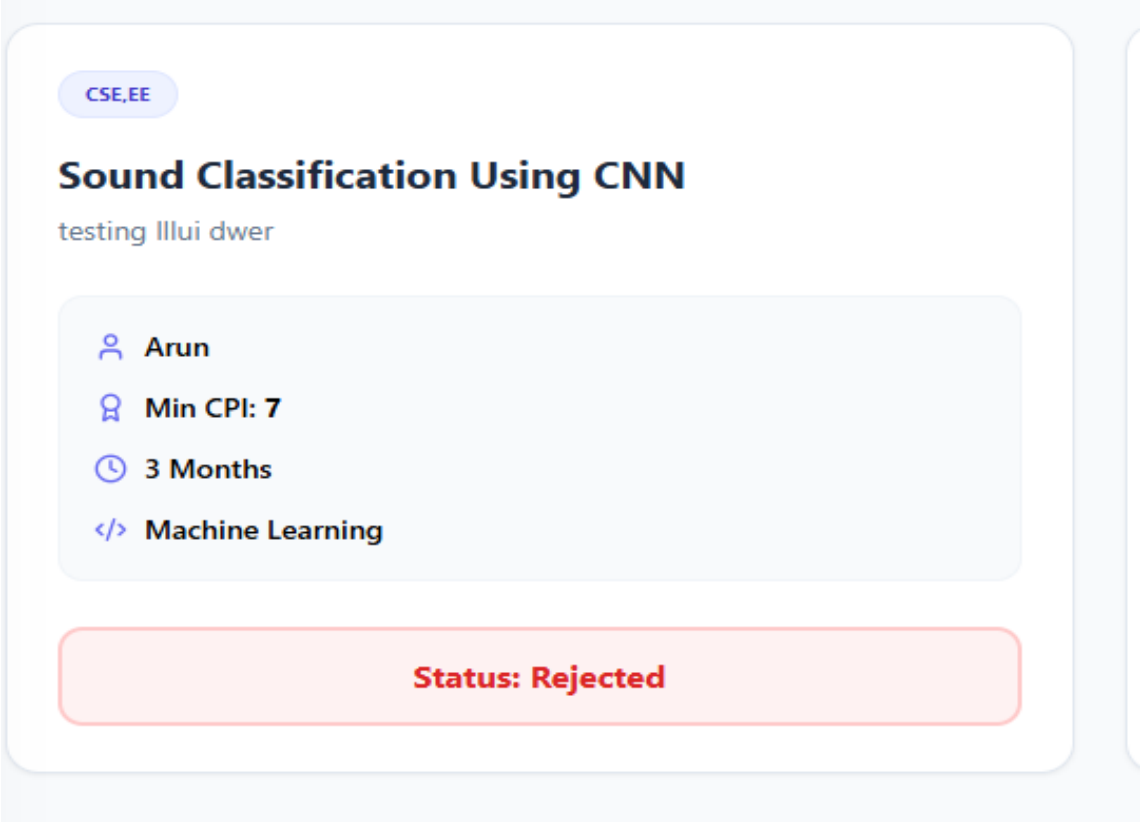
Module Details: This module covers the change of status of application from ‘Pending’ to ‘Accepted’/’Rejected’ when professor accepts/rejects the application.

Test Owner: DevAxis
Test Date: April 02, 2026
Test Results:
After submission of application by student, the status is ‘Pending’. Upon acceptance of the application by the professor, the status changes to ‘Accepted’.





Upon rejection, the status is shown ‘Rejected’



7) ANALYTICS

Module Details:
This module ensures smooth integration between feedback submission (frontend), backend APIs (Express), and MongoDB database for generating analytics such as average ratings and trend analysis.
Test Owner: DevAxis
Test Date: 02-04-2026
Test details:


```
_id: ObjectId('69ce5365d3f12d019486f844')
course : ObjectId('69cd669fba517dfde7708531')
session : ObjectId('69cd669fba517dfde7708533')
student : ObjectId('69cc05300e8a1ce20b439e37')
▾ ratings : Object
    content : 5
    delivery : 4
    clarity : 4
    engagement : 4
    pace : 5
comment : ""
createdAt : 2026-04-02T11:30:45.047+00:00
updatedAt : 2026-04-02T11:30:45.047+00:00
__v : 0
```


- Test case-1: Checking whether average ratings are calculated correctly

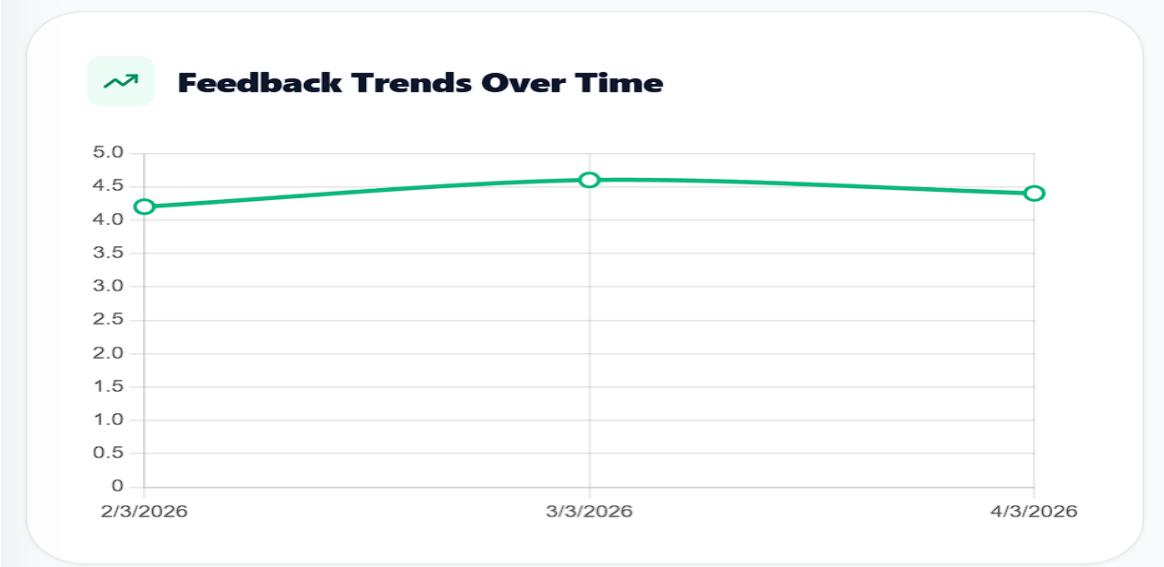


Results: The feedback data is successfully stored in MongoDB and fetched by the analytics API. The system correctly calculates average values for all parameters (content, delivery, clarity, engagement, pace). The computed averages are returned in structured JSON format and displayed correctly in the frontend dashboard.

- **Test case-2:** Checking whether trend analysis is generated correctly



After giving the feedback of the next day it changes as it fectches the next day available data from the mongoDb



Results: The system correctly groups feedback data based on session dates and computes trend scores for each session. The trend data (dates and corresponding scores) is generated accurately and displayed in the frontend as trend graphs. The visualization reflects changes in performance over time correctly.

8) Account Management

Module Details:

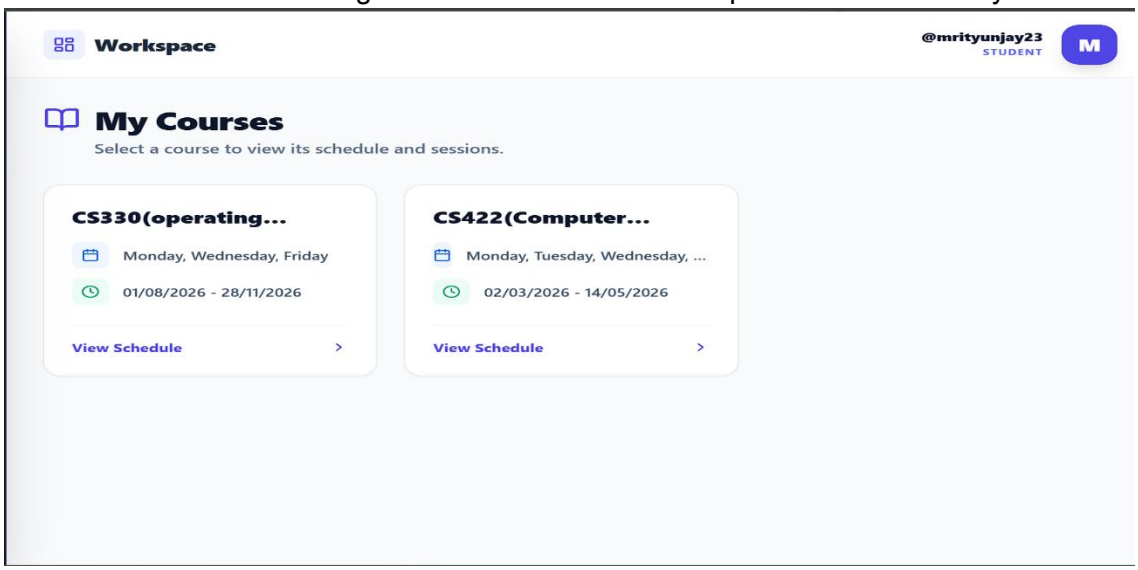
This module ensures smooth interaction between the frontend (React), backend APIs (Express), and MongoDB database for account management functionalities including username update, password update, and account deletion.

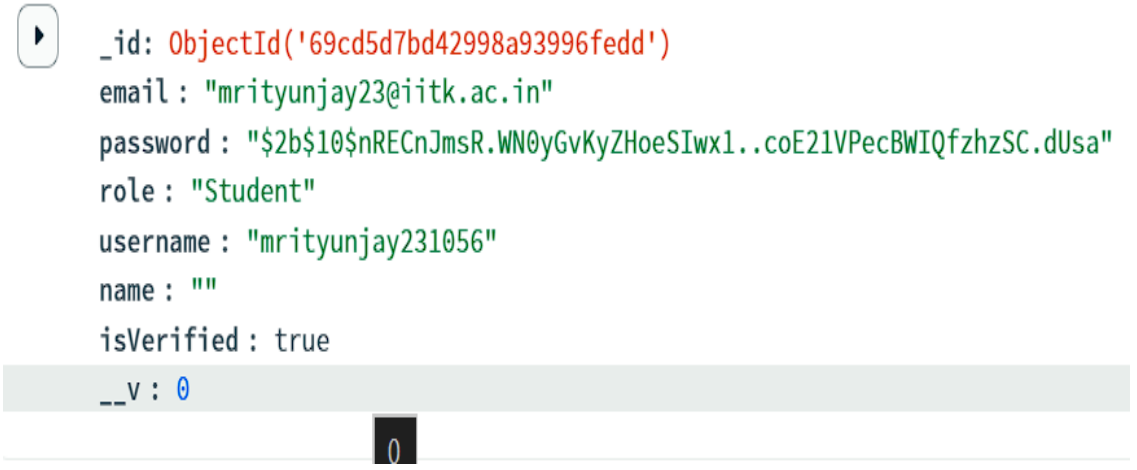
Test Owner: DevAxis

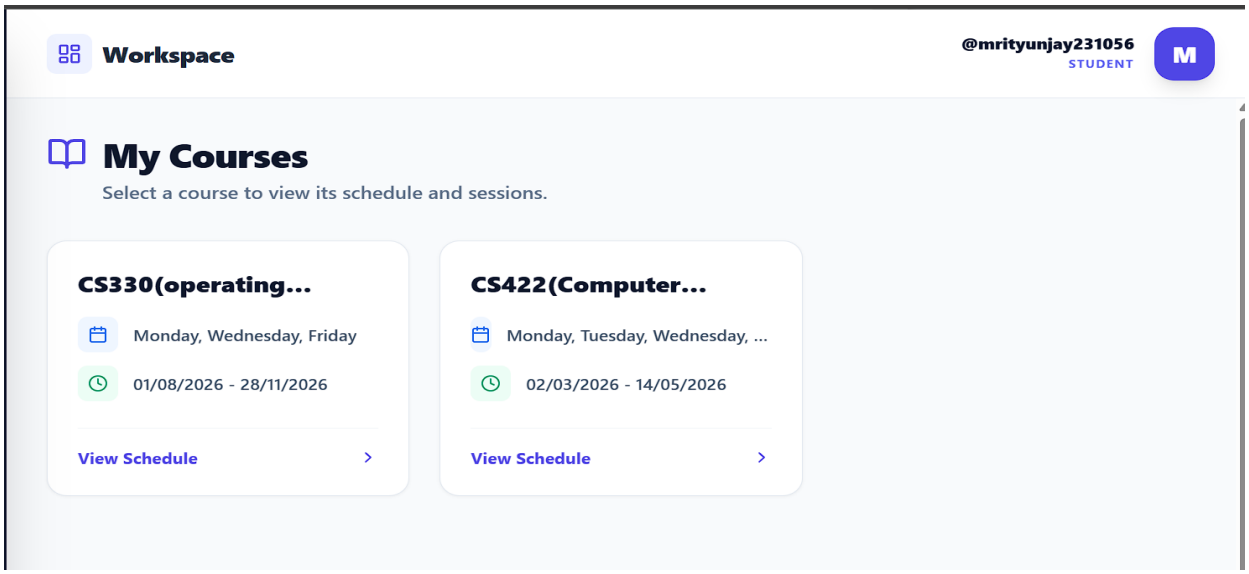
Test Date: 03-04-2026

Test Details:

- **Test case-1:** Checking whether username/name update works correctly







Result:
The request is sent from the frontend to the backend API (/api/user/identity). The backend validates the input and checks for duplicate usernames in MongoDB. If valid, the updated username/name is stored in the database and returned in the response. The frontend updates the user state and reflects the change immediately in the UI. Invalid inputs such as duplicate or empty usernames are rejected with appropriate error messages.

- **Test case-2:** Checking whether password update works correct.

Result:
The password update request is sent to the backend API (/api/user/password) with current and new password. The backend verifies the current password using bcrypt and hashes the new password before storing it in MongoDB. After successful update, the user is able to log in using the new password. If the current password is incorrect, the request is rejected and no changes are made.

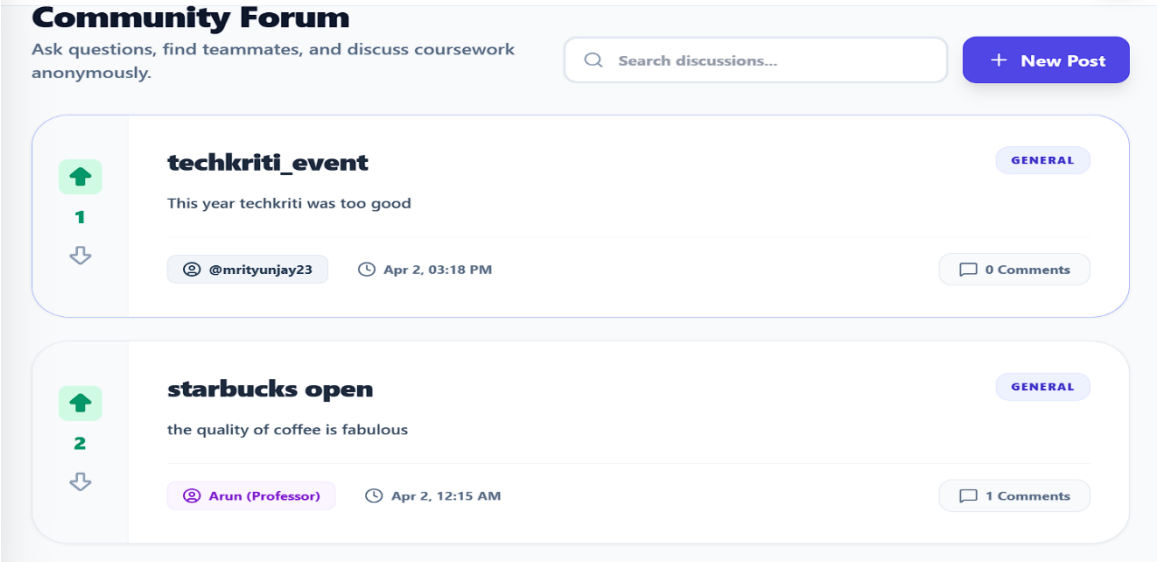
- **Test case-3:** Checking whether account deletion works correctly

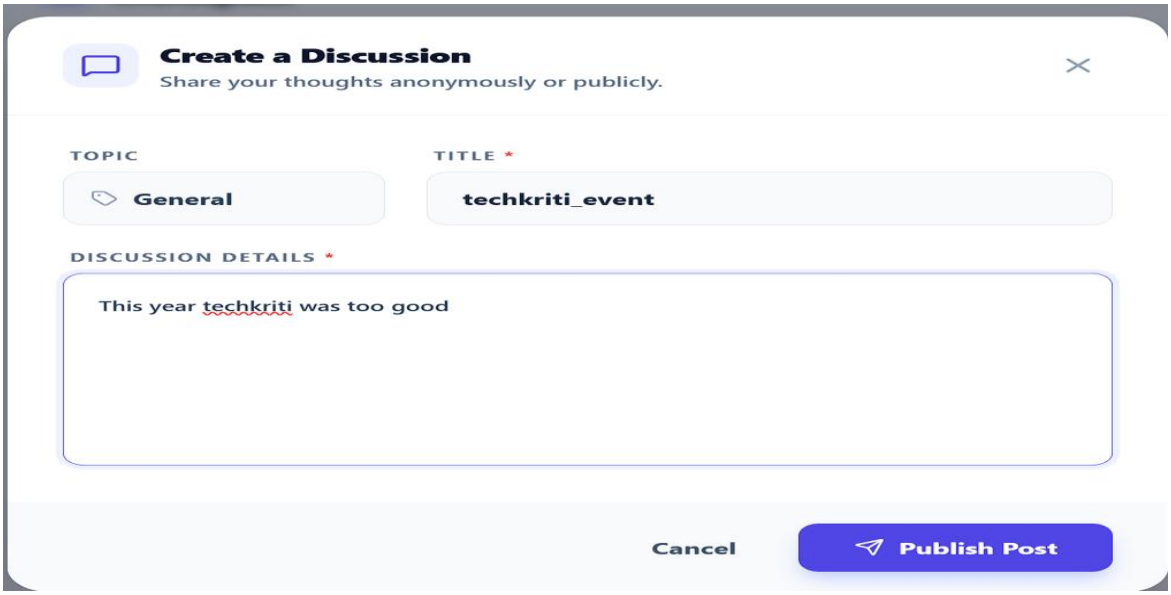
Result:
The deletion request is sent to the backend API (/api/user/delete) with password confirmation. The backend verifies the password and proceeds to anonymize the user. The user is removed from enrolled courses and associated data is handled appropriately. The frontend clears session data and logs the user out. After deletion, the user is unable to log in again.

9) Community Forum

Module Details:
This module ensures smooth interaction between the frontend (React), backend APIs (Express), and MongoDB database for forum functionalities including post creation, voting, commenting, and role-based identity display.
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:

- **Test case-1:** Checking whether forum posts are created and displayed correctly

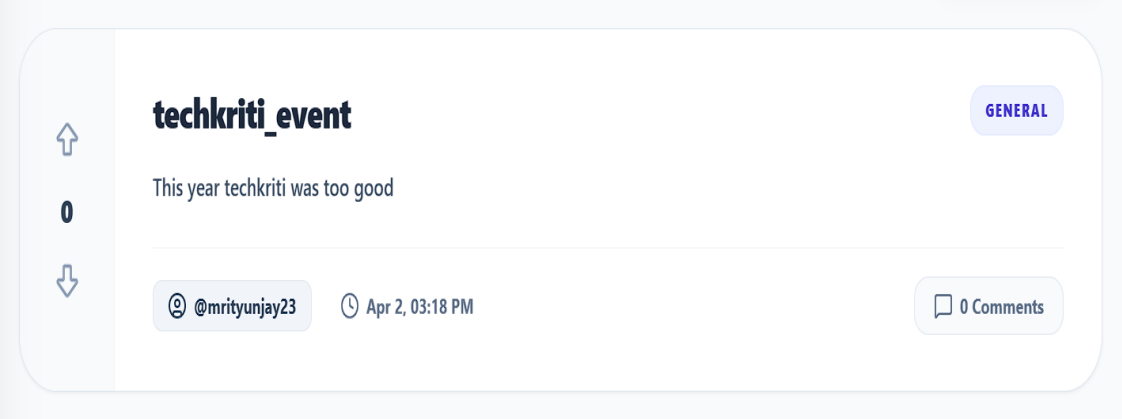




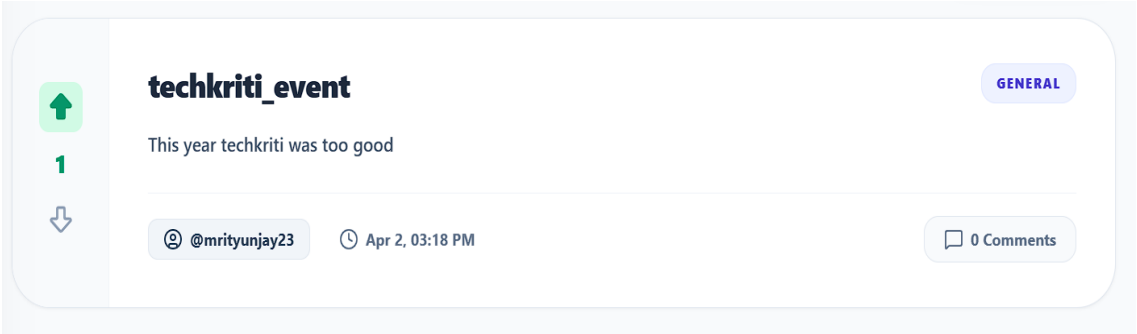
```
_id: ObjectId('69ce3b87d3f12d019486f34f')
author : ObjectId('69cd5d7bd42998a93996fedd')
title : "techkriti_event"
content : "This year techkriti was too good"
tag : "General"
▾ upvotes : Array (1)
    0: ObjectId('69cd5d7bd42998a93996fedd')
▾ downvotes : Array (empty)
createdAt : 2026-04-02T09:48:55.402+00:00
updatedAt : 2026-04-02T09:48:55.402+00:00
__v : 0
```


Result:
The post is successfully created when valid inputs are provided. The request is sent to the backend API (/api/forum/posts) and stored in MongoDB. The newly created post is immediately fetched and displayed correctly in the forum feed UI.

- **Test case-2:** Checking whether voting (upvote/downvote) updates correctly



```
_id: ObjectId('69ce3b87d3f12d019486f34f')
author : ObjectId('69cd5d7bd42998a93996fedd')
title : "techkriti_event"
content : "This year techkriti was too good"
tag : "General"
upvotes : Array (empty)
downvotes : Array (empty)
createdAt : 2026-04-02T09:48:55.402+00:00
updatedAt : 2026-04-02T10:05:57.054+00:00
__v : 4
```

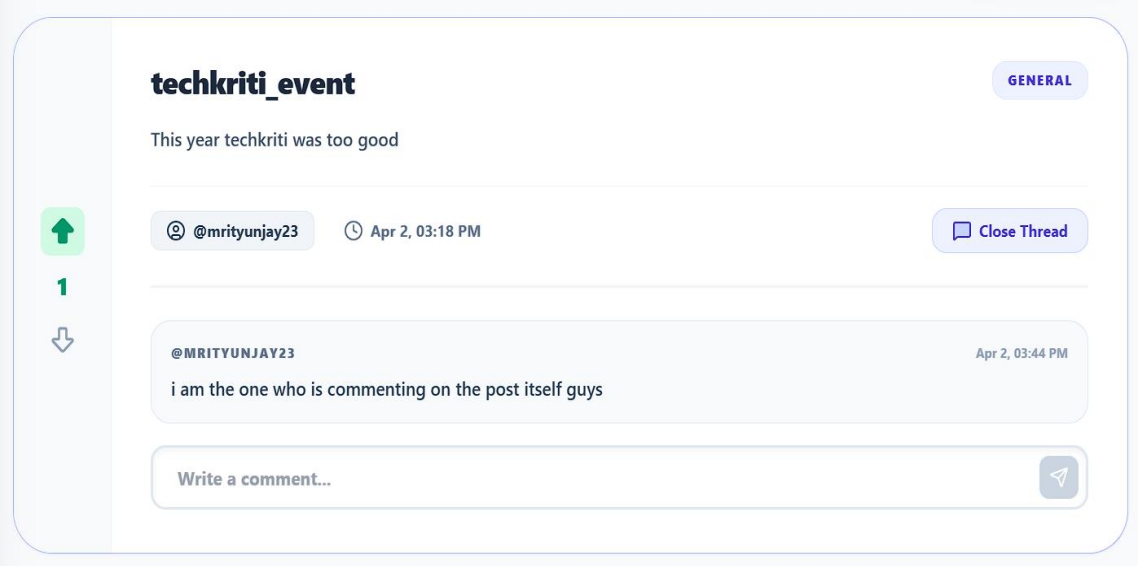




```
_id: ObjectId('69ce3b87d3f12d019486f34f')
author : ObjectId('69cd5d7bd42998a93996fedd')
title : "techkriti_event"
content : "This year techkriti was too good"
tag : "General"
upvotes : Array
  0: ObjectId('69cd5d7bd42998a93996fedd')
downvotes : Array (empty)
createdAt : 2026-04-02T09:48:55.402+00:00
updatedAt : 2026-04-02T10:09:29.205+00:00
__v : 5
```


This year techkriti was too good

Result:
The voting request is sent to the backend API (/api/forum/posts/:id/vote). The vote is updated in MongoDB by modifying the upvotes and downvotes arrays. The updated vote count is reflected instantly in the UI. Vote toggling and switching between upvote and downvote are handled correctly.

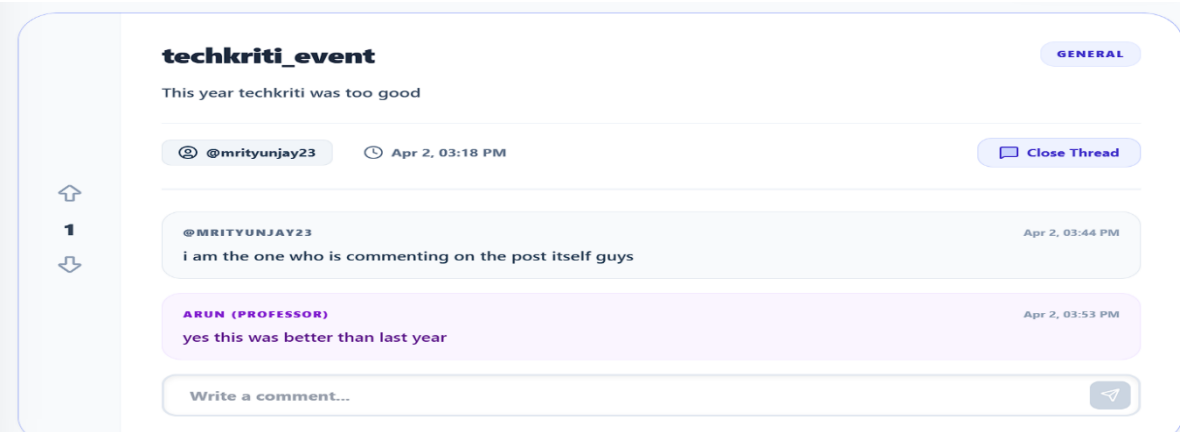
- **Test case-3:** Checking whether comments are added and displayed correctly

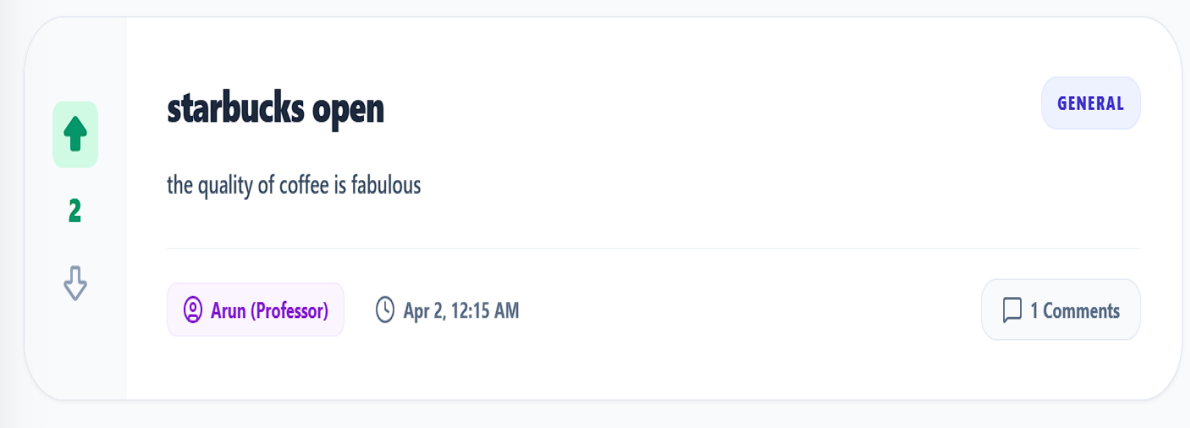



```
_id: ObjectId('69ce4194d3f12d019486f5bc')
post : ObjectId('69ce3b87d3f12d019486f34f')
author : ObjectId('69cd5d7bd42998a93996fedd')
content : "i am the one who is commenting on the post itself guys"
createdAt : 2026-04-02T10:14:44.626+00:00
updatedAt : 2026-04-02T10:14:44.626+00:00
__v : 0
```

Result:
The comment is successfully sent to the backend API (/api/forum/posts/:id/comments) and stored in MongoDB. The comments are fetched again and displayed correctly under the respective post. Proper linkage between posts and comments is maintained

- **Test case-4:** Checking whether role-based anonymity is working correctly






```
_id: ObjectId('69cd6849ba517dfde770867a')
post : ObjectId('69cd67d1ba517dfde7708626')
author : ObjectId('69cd5d7bd42998a93996fedd')
content : "yes i also tried it was nice"
createdAt : 2026-04-01T18:47:37.546+00:00
updatedAt : 2026-
yes i also tried it was nice
0:00
__v : 0
```

```
_id: ObjectId('69cd67d1ba517dfde7708626')
author : ObjectId('69cc05300e8a1ce20b439e2f')
title : "starbucks open"
content : "the quality of coffee is fabulous"
tag : "General"
▸ upvotes : Array (2)
▸ downvotes : Array (empty)
createdAt : 2026-04-01T18:45:37.274+00:00
updatedAt : 2026-04-01T18:59:20.854+00:00
__v : 9
```

Results: Student users are displayed using usernames (anonymous identity), while professor users are displayed with their full names. The role information is correctly passed from backend to frontend, and the UI renders identity based on role conditions.

- **Test case-6:** Checking whether invalid inputs are handled correctly

  Result:
  The system prevents submission of invalid inputs. Validation is enforced at both frontend and backend levels, ensuring that no empty or incorrect data is stored in MongoDB.

4 System Testing

1) Authentication and Access Control

Requirement: Users must be able to securely authenticate into the system. Students should be forced to use IITK emails, and Professors must be pre-authorized. Passwords must be securely hashed, and login sessions must be maintained via JWT.

Module Overview: The complete lifecycle of authentication was tested from the React frontend forms through the Node.js auth controllers to MongoDB persistence.

Test Owner: DevAxis

Test Date: April 03, 2026

Test Procedures & Results: During testing, we executed the full user journey on the live local environment: -

1. Attempted to register a student account using a @gmail.com address. System successfully blocked the action at the frontend level.
2. Submitted a valid Student @iitk.ac.in email. Received the 6-digit OTP via email integration, entered it into the UI, and successfully created the account.
3. Attempted to register a professor account using an unauthorized IITK email. The backend rejected the request with a 403 Forbidden status.
4. Admin added the email to the whitelist. Attempted Professor registration again, which succeeded.
5. Logged into the system with valid credentials. Verified via Chrome DevTools that the JWT Token and User Object were correctly injected into localStorage and that the system redirected the user to the Workspace Dashboard.

Final Status: SUCCESS. The end-to-end authentication and access control flow works exactly as expected.

.

2) Course Creation & Scheduling

a) Objective

The objective of System Testing is to validate the complete workflow of the module from frontend to backend.

b) End-to-End Workflow

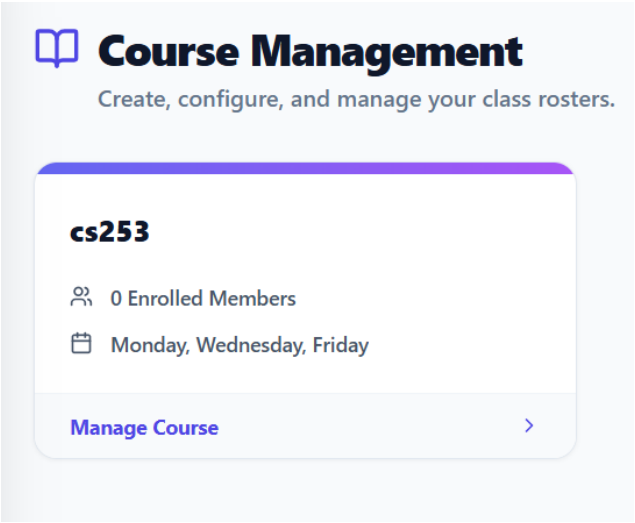
Step 1: Create Course (Frontend – ManageCourses.jsx)

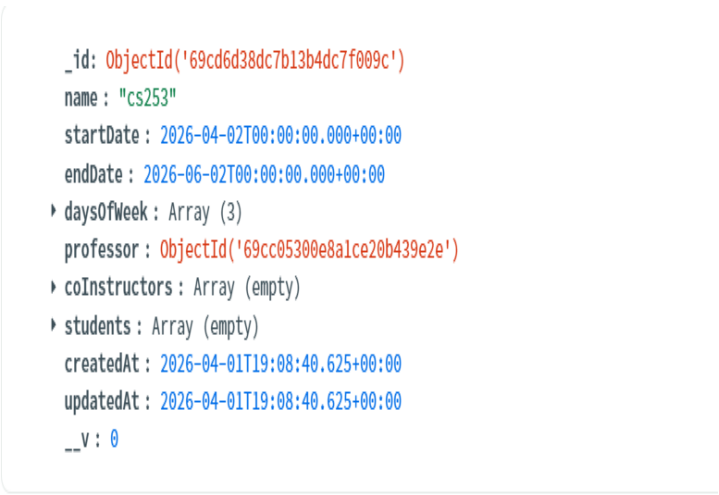
```
fetch(`${API_BASE}/api/courses`, {
  method: "POST",
  headers: getHeaders(),
  body: JSON.stringify(courseForm),
});
```

Requirement: The professor should be able to create a course using valid inputs such as course name, start date, end date, and selected days.
Unit Details: Frontend (ManageCourses.jsx), Backend API (/api/courses), MongoDB (Courses and Sessions Collection)
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:
• Course is successfully created in the database
• Sessions are automatically generated
• Data is correctly stored with professor ID
• No errors occurred during execution

Additional Comments:
The system correctly integrates frontend input with backend processing and database storage.
Expected Result

- Course is created successfully
- Sessions are generated
- Data stored in database.





Here course is created successfully and till then I have not added any co-instructor so its array is showing empty.

Step 2: Add Co-Instructor (Professor Side)

Requirement: The professor should be able to add a valid co-instructor using email.
Unit Details: Frontend (ManageCourses.jsx), Backend API (/api/courses/:id/co-instructors), MongoDB (Courses and User Collection)
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:
• Co-instructor is successfully added
• Co-instructor ID appears in course document
• Duplicate entries are prevented
• Invalid email returns error

Additional Comments:
The system ensures proper validation and maintains correct association between multiple instructors.

Source Code (routes/courseRoutes.js)

```
router.post("/:id/co-instructors", ctrl.addCoInstructor);
```

Controller Logic (controllers/courseController.js)

```
exports.addCoInstructor = async (req, res) => {
  try {
    const { email } = req.body;
    const courseId = req.params.id;

    const user = await User.findOne({ email: email.toLowerCase(), role: "Professor" });
    if (!user) return res.status(404).json({ msg: "Professor email not found in system" });

    const course = await Course.findById(courseId);
    if (course.coInstructors.includes(user._id) || course.professor.equals(user._id)) {
      return res.status(400).json({ msg: "Professor is already associated with this course" });
    }

    course.coInstructors.push(user._id);
    await course.save();

    res.json({
      msg: "Co-instructor added successfully",
      user: { id: user._id, name: user.name, email: user.email }
    });
  } catch (err) {
    res.status(500).json({ msg: "Server Error" });
  }
};
```

Explanation

- The request is sent to route: /api/courses/:id/co-instructors
- Backend searches for a user with:
    - Matching email
    - Role = Professor
- If user is not found → returns error
- If already associated → prevents duplication
- Otherwise:
    - User ID is added to co-Instructors array
    - Course document is updated

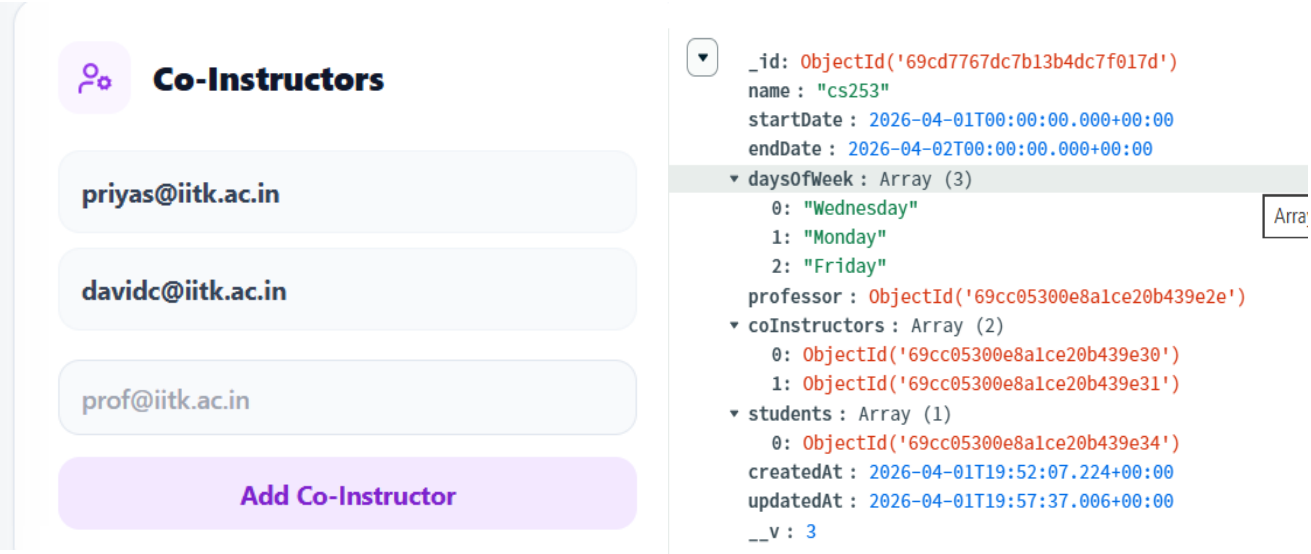
What is Being Verified

- Course contains correct **professor ID**
- Course contains correct **co-instructor IDs**
- All instructors belong to same course
- Both have access to course data (via API logic)

Expected Result

- Co-instructor is added successfully
- Duplicate entries are prevented
- Invalid email returns appropriate error
- professor field contains creator's ID
- Both can access:
    - Course data
    - Sessions
    - Student list

Database Verification



Here you can see that after adding the co-instructor by the professor , both co-Instructor-Id are showing in the database in that course . It is shown with the help of the objectId in the database.

Step 3: View Sessions (Student Side - CourseViewer.jsx)

Requirement: The student should be able to view the course sessions and schedule.
Unit Details: Frontend (CourseViewer.jsx), Backend API (/api/courses/:id/sessions), MongoDB (Sessions Collection)
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:
• Sessions are displayed correctly in UI
• Dates are shown in correct order
• Future sessions are locked
• Past sessions are accessible
Additional Comments:
The system correctly fetches and displays session data from backend.

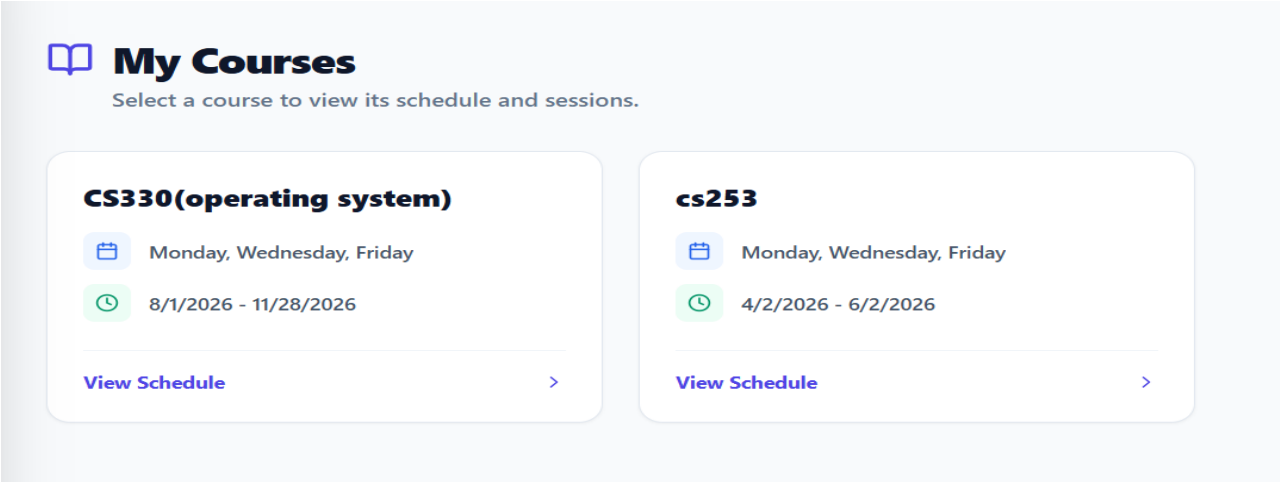
```
fetch(`${API_BASE}/api/courses/${course._id}/sessions`)
```

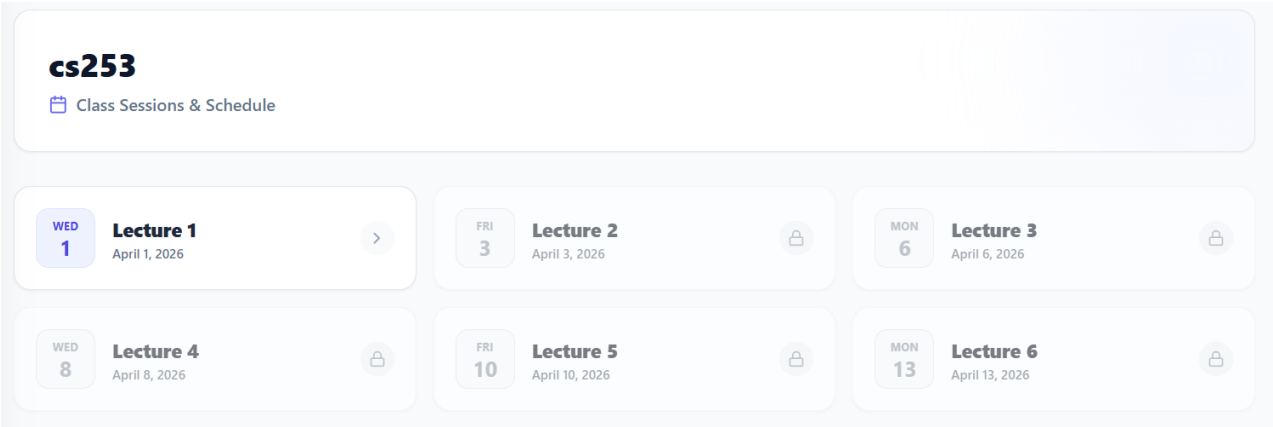
Explanation

- The system fetches all sessions related to the selected course
- These sessions are generated automatically during course creation
- The data is displayed in the user interface with proper date formatting

Expected Result

- Student can view:
  - **Lecture dates**
  - **Chronological order of sessions**
- Future sessions are **locked**
- Past sessions are **accessible**





Here it is showing lecture dates of the course cs253. Future sessions are locked and past sessions are accessible.

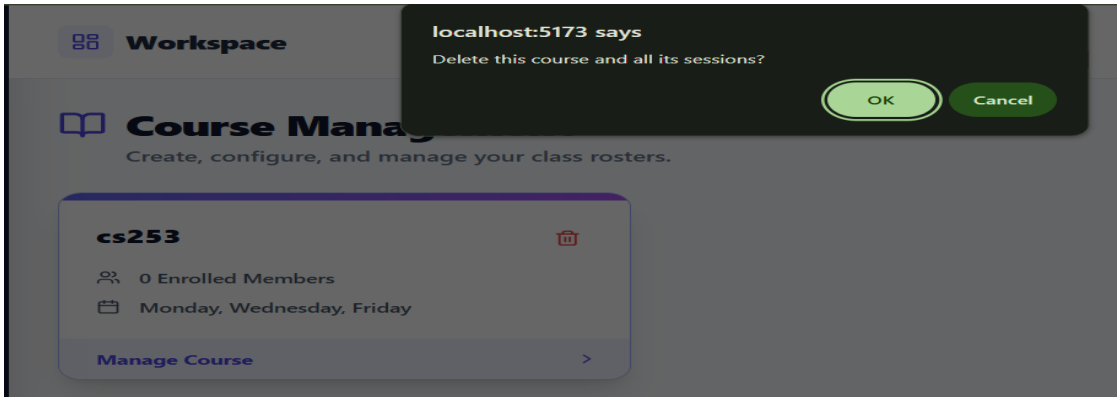
Step 4: Delete Course

Requirement: The professor should be able to delete a course.
Unit Details: Backend API (/api/courses/:id), MongoDB (Courses, Sessions, Feedback Collection)
Test Owner: DevAxis
Test Date: 02-04-2026
Test Results:
• Course is successfully deleted
• Sessions are removed from database
• Related feedback and data are removed
• No residual data remains

Additional Comments:
The system ensures complete cleanup and maintains data consistency.

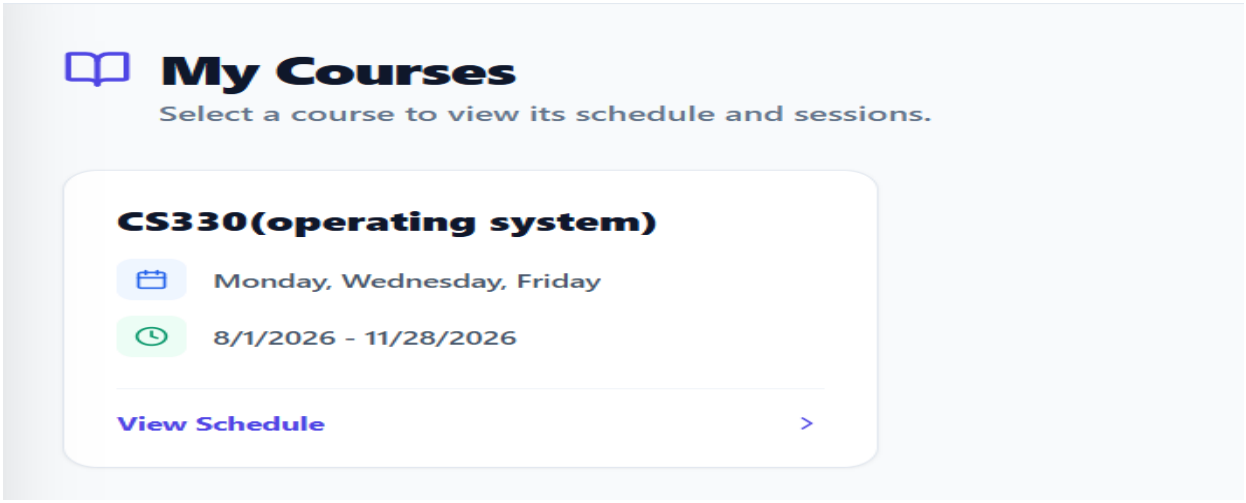
```
router.delete("/:id", ctrl.deleteCourse);
```

Professor is deleting the cs253 course.



Expected Result

- Course deleted
- Sessions deleted
- Comments and feedback remove



At first it is showing in the student's section but after deleting the course cs253 and its feedback are removed from the student's account who is enrolled in this course.

c) Test Case Table

- Create Course (Valid Input) → Course and sessions created
- Missing Fields → Error response (400)
- View Sessions → Correct schedule displayed
- Delete Course → All related data removed

Conclusion

The Course Creation & Scheduling module was thoroughly tested using:

- Unit Testing to validate internal logic
- Implementation Testing to verify API and database interaction
- System Testing to ensure complete workflow correctness

The module successfully handles:

- Valid inputs
- Invalid inputs
- Edge cases
- Full lifecycle operations

3) Student Enrolment

Requirement:

The User should be able to complete the end-to-end flow of viewing managed courses, navigating to a specific course dashboard, manually enrolling a single student, performing a bulk enrolment via an .xlsx file, and verifying the updated roster persists upon reload.

Module Overview:

The complete lifecycle of student management was rigorously tested to ensure seamless integration between the frontend user interface (React/Vite), the backend routing and controllers (Node.js/Express), and the staging database (MongoDB). While Unit Testing isolated the logic and Integration Testing verified the data handshakes, this System Testing phase validates the real-world user journey. Test cases were designed to simulate a complete session from login to data persistence, ensuring that all subsystems function together flawlessly under standard operating conditions.

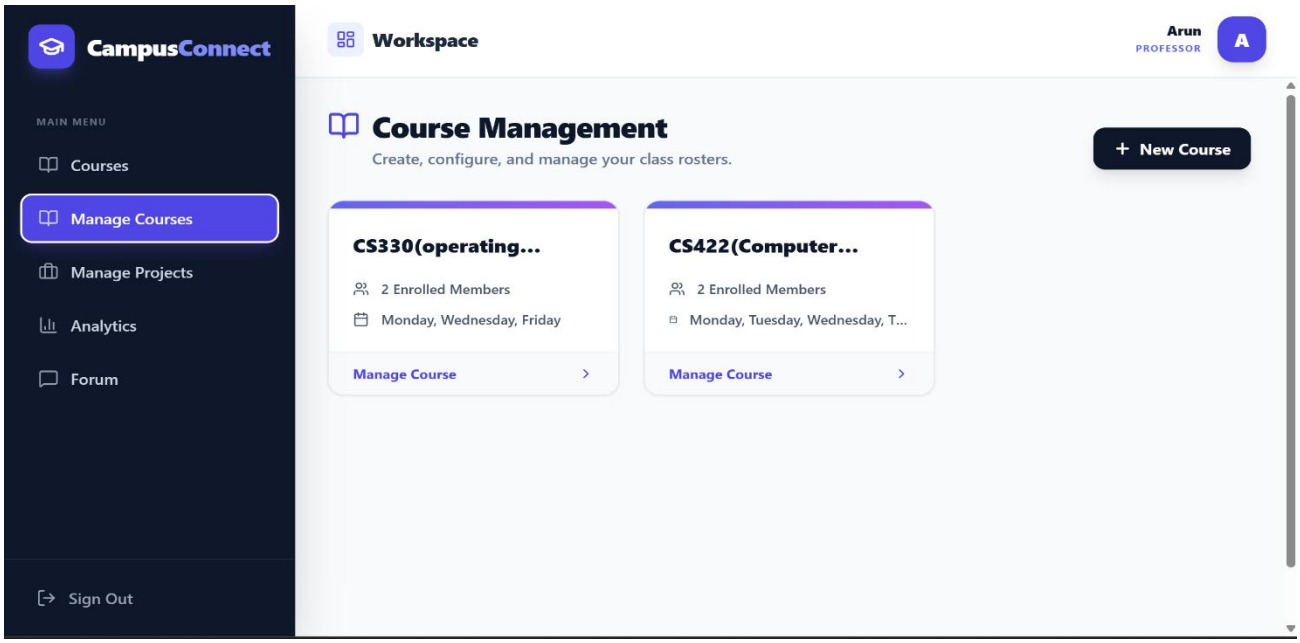
Test Owner: DevAxis
Test Date: April 04, 2026
Test Procedures & Execution:

During testing, we executed the full user journey sequentially on the live local environment to mimic real-world usage:

Step 1: System Initialization & Course Retrieval

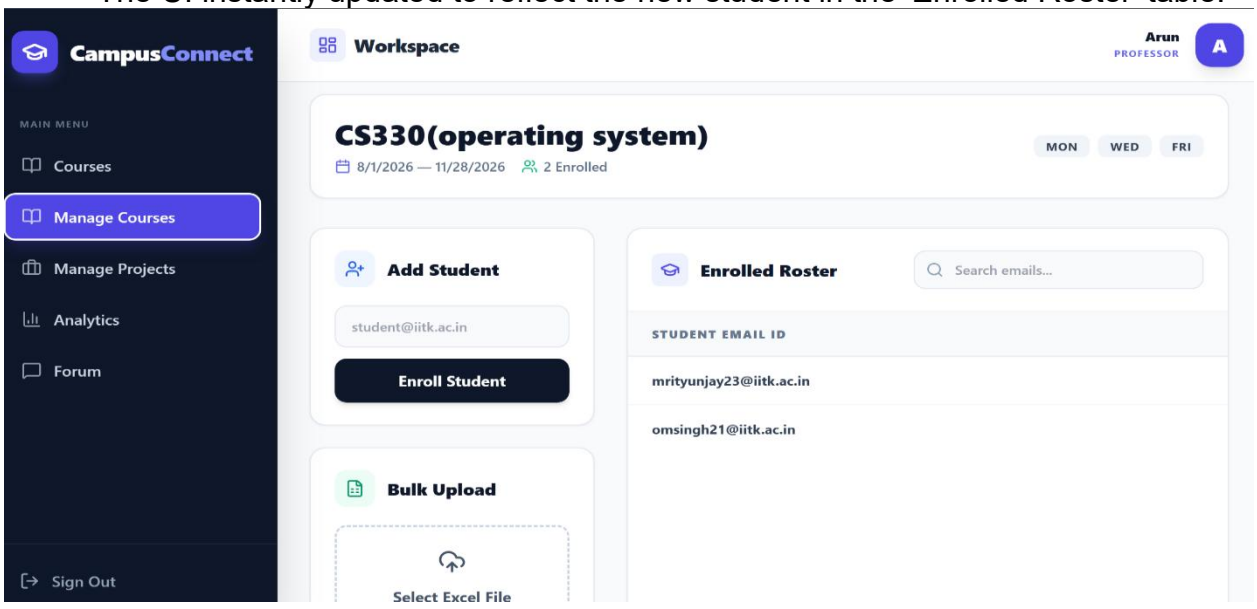
- We initiated the application and logged in using valid User credentials.
- Navigated to the 'Manage Courses' workspace via the main sidebar menu.

- Observation: The system successfully authenticated the token, queried the database, and rendered the UI cards for active courses (CS330 and CS422), accurately displaying dynamic data such as "2 Enrolled Members" and the specific class schedules.



Step 2: Manual Student Enrolment Flow

- Clicked 'Manage Course' on the CS330 (Operating System) card, which successfully routed the application to the specific course dashboard.
- Interacted with the 'Add Student' form. We inputted a valid institutional email ID (student@iitk.ac.in) and clicked 'Enroll Student'.
- Observation: The frontend correctly bundled the payload and fired the API request.
- The UI instantly updated to reflect the new student in the 'Enrolled Roster' table.



Step 3: Bulk File-Stream Processing

- Navigated to the CS422 (Computer Architecture) dashboard to test the heavy-load file stream.
- Utilized the 'Bulk Upload' drag-and-drop component to upload a pre-configured .xlsx file containing a roster of 28 student emails.
- Observation: The system successfully parsed the Excel file, cross-referenced the database for existing users, and processed the batch. The UI dynamically updated to show a summary badge reading "SUCCESS: 21 Added" and "SKIPPED: 7 Failed" (proving the system correctly handled our intentionally duplicated entries). The roster table populated instantly.



Step 4: System State Persistence & Security Checks

- To verify that the UI was not just spoofing the data locally, we performed a hard browser refresh (Ctrl + F5) on the course dashboard.
- Observation: The application successfully remounted, re-authenticated the user session in the background, fetched the fresh data from MongoDB, and repopulated the rosters exactly as they were before the refresh.
- Finally, we manually deleted the JWT token from the browser's local storage and attempted to submit another manual enrolment. The system correctly caught the missing authorization, blocked the API call, and prevented any database manipulation.

Final Status: SUCCESS. The end-to-end student enrolment flow works exactly as expected.

4) Class Discussions & Feedback

Requirement: The system shall support real-time anonymous class discussions and enforce compulsory MCQ feedback submission before a student leaves a session.
The class discussion functionality was rigorously tested to ensure seamless, real-time communication between students and professors. Test cases were designed to validate a live conversation flow, anonymity, and message threading. During testing, we manually accessed an active session and posted a comment as a student, verifying that the student's real identity was successfully masked by their anonymous username. We then logged in as a professor, replied directly to the student's message, and confirmed that the WebSocket integration instantly updated the thread for both users without requiring a page refresh. We also verified that the UI correctly distinguished the users by aligning the student's and professor's messages appropriately (e.g., left/right alignment) and successfully linked the nested reply to its parent comment.

We also verified the compulsory feedback mechanism by attempting to navigate away from the session. The system correctly intercepted the action and prompted the student to complete the required 5-category MCQ rating (Content, Delivery, Clarity, Engagement, and Pace). We inputted valid ratings along with a written remark, ensuring the system authenticated the submission and securely stored it to be visualized on the Professor's Analytics Dashboard. Additionally, error-handling and data integrity were validated by attempting to submit feedback a second time for the same session, confirming the backend correctly blocked the

duplicate entry and displayed the "Feedback already submitted for this session" error.
Test Owner: DevAxis
Test Date: 04/03/2026

Test Results: The live discussion thread and feedback submission worked exactly as expected. We tested the real-time interactions and validations on our local/deployed environment. No bugs were reported.
Additional Comments: The Socket.io broadcast effectively restricted message emissions to the specific sessionId room, ensuring zero cross-session data leakage. Reading the thread also successfully cleared the unread indicator on the main dashboard.

5) Professors Management – Professor Side

Requirement: Project Creation Flow

Requirement Reference (SRS):

The system shall allow a professor to create a new project through the UI and display it on the dashboard.

Description:

Tested the complete flow where a professor:

- Clicks **"Create New Project"**
- Fills the form
- Submits the project

The system should store the project and update the UI.

Test Owner: DevAxis
Test Date: 28/03/2026

Test Results:

- The modal opened successfully
- Form accepted valid inputs
- On clicking "Launch Project", modal closed
- Newly created project appeared in dashboard list
- Data persisted after refresh

Additional Comments:

- "Open to All" functionality also worked correctly
- No UI lag or crash observed

B. Requirement: View Applicants List

Requirement Reference (SRS):

The system shall allow professors to view applicants for a project.

Description:

Tested clicking **"View Applicants"** button:

- Applicant table should expand
- Student data should be displayed

Test Owner: DevAxis

Test Date: 28/03/2026

Test Results:

- Clicking button expanded applicant section
- Table displayed:
  - Name
  - Roll No
  - Branch
  - CPI
- Data matched database records

Additional Comments:

- UI expansion animation smooth.
- No UI lag or crash observed.
- No missing data observed.

C. Requirement: Accept/Reject Candidate

Requirement Reference (SRS):

The system shall allow professors to accept or reject applicants.

Description:

Tested:

- Clicking (Accept)
- Clicking (Reject)

System should update status and reflect in UI.

Test Owner: DevAxis

Test Date: 28/03/2026

Test Results:

- **Clicking Accept:**
  - Button replaced with "Accepted" badge
- **Clicking Reject:**
  - Button replaced with "Rejected" badge
- **Changes persisted after page refresh**

Additional Comments:

- API response was fast
- No duplicate updates occurred

D. Requirement: Branch Filtering of Applicants

Requirement Reference (SRS):

The system shall allow filtering applicants based on branch.

Description:

Tested dropdown filter:

- Selected **CSE**
- Verified only CSE students shown\
-

Test Owner: DevAxis.

Test Date: 28/03/2026

Test Results:

- Dropdown filter worked correctly
- Only selected branch students displayed
- Non-matching students removed from view

Additional Comments:

- Filtering is dynamic and responsive
- Works correctly for all branches

6) Project Discovery – Student Side

Requirement: The User should be able to complete the end-to-end flow of searching for a project, submitting an application, and retaining their application status upon reload.

The complete application lifecycle was rigorously tested to ensure seamless integration between the frontend user interface, backend APIs, and the staging database. Test cases were designed to validate the real-world user journey, including searching, form submission, and data persistence. During testing, we manually navigated the live staging site, utilized the search functionality to filter projects, and entered valid application data (including external resume links). We confirmed that the system correctly processed the payload, updated the database, and dynamically changed the UI status to "Pending".

We also verified system state persistence by performing hard browser refreshes immediately after successful application submissions, ensuring the system fetched the updated status from the database and prevented unauthorized duplicate applications. Additionally, cross-device compatibility was validated by simulating mobile viewports to confirm the project grid collapsed correctly and the modal remained fully scrollable.

Test Owner: DevAxis

Test Date: April 02, 2026

Test Results: The end-to-end application flow worked as expected. We had tested it extensively on our deployed staging website. No database integration bugs were reported.

Additional Comments: The layout successfully degraded on mobile views, and the background blur on the modal did not interfere with scrolling on smaller screens.

7) Analytics

1.Requirement: The system should correctly calculate average ratings based on feedback submitted by users**.**

The average calculation functionality was tested to ensure accurate aggregation of feedback data across multiple users and sessions. Test cases were designed to validate whether the system correctly computes average values for parameters such as content, delivery, clarity, engagement, and pace. During testing, multiple feedback entries with varying ratings were submitted through the frontend. It was verified that the system successfully stored the data in MongoDB and computed the correct average values using backend logic. The calculated averages were then displayed correctly on the analytics dashboard. Edge cases such as absence of feedback data were also considered to ensure robustness of the system.

Test Owner: DevAxis

Test Date: 02/04/2026

Test Results:

The system successfully calculated average ratings with accurate values. The results were consistent with the data stored in MongoDB and displayed correctly in the dashboard without any discrepancies.

Additional Comments:

The system ensures reliable aggregation of feedback data and maintains consistency between database values and displayed results.

2.Requirement: The system should generate correct trend analysis based on session-wise feedback data.

The trend analysis functionality was tested to ensure that the system correctly processes time-based feedback data and generates meaningful insights. Test cases included submitting feedback across different sessions with varying dates and rating values. During testing, it was verified that the system grouped feedback data based on session dates and calculated trend scores for each session. The backend successfully generated structured trend data, which was then visualized in the frontend as graphs. The system accurately reflected variations in performance over time, demonstrating correct implementation of trend logic.

Test Owner: DevAxis

Test Date: 02/04/2026

Test Results:

The trend analysis functionality worked correctly, generating accurate session-wise trend data and displaying it properly in the dashboard graphs.

Additional Comments:

The system effectively handles time-based data grouping and provides meaningful visualization of trends for better insights.

8) Account Management

1. Requirement:

The user should be able to update their username/name with valid inputs.

The username/name update functionality was tested to ensure proper interaction between frontend, backend, and database. Test cases included updating with valid inputs, duplicate usernames, and empty fields. During testing, users updated their username/name successfully, and the changes were stored in MongoDB and reflected immediately in the UI. Negative test cases verified that duplicate or empty inputs were not allowed and appropriate validation messages were displayed.

Test Owner: DevAxis

Test Date: 03-04-2026

Test Results:
The username/name update functionality worked as expected. Valid updates were successfully stored and displayed, while invalid inputs were correctly rejected.
Additional Comments:
The system ensures proper validation at both frontend and backend, preventing duplicate or invalid usernames.

2. Requirement:
The user should be able to update their password securely.
The password update functionality was tested to ensure secure handling of user credentials. Test cases included correct password update, incorrect current password, and validation of new password. During testing, the system successfully verified the current password using bcrypt and stored the new password in hashed form. The user was able to log in using the updated password. Incorrect password attempts were rejected.
Test Owner: DevAxis
Test Date: 03-04-2026
Test Results:
The password update functionality worked correctly, ensuring secure verification and storage of passwords.
Additional Comments:
The system maintains strong security by hashing passwords and preventing unauthorized updates.

3. Requirement:
The user should be able to delete their account securely.
The account deletion functionality was tested to ensure correct handling of user data and system consistency. Test cases included valid deletion with correct password and invalid deletion attempts. During testing, the system verified the password, anonymized the user, removed the user from courses, and cleared session data. The user was logged out and could not log in again after deletion.
Test Owner: DevAxis
Test Date: 03-04-2026
Test Results:
The account deletion functionality worked as expected. Users were successfully removed and system data integrity was maintained.
Additional Comments:
The system ensures safe deletion by anonymizing user data instead of permanently removing all records.

9) Community Forum

1. Requirement: The user should be able to create a forum post using valid inputs (title, content, and tag).
The post creation functionality was rigorously tested to ensure seamless interaction between frontend, backend, and database. Test cases were designed to validate successful post creation, handling of empty inputs, and correct display in the forum feed. During testing, users created posts using valid data, and it was confirmed that the system stored the data in MongoDB and displayed it correctly in the UI. Negative test cases verified that posts with empty title or content were not allowed and appropriate validation was enforced.
Test Owner: DevAxis
Test Date: 02/04/2026
Test Results:
The post creation functionality worked as expected. Posts were successfully stored in the database and displayed in the forum feed without any issues. No bugs were reported.

Additional Comments:
The system prevents submission of invalid inputs and ensures proper validation at both frontend and backend.

2. Requirement: The user should be able to perform upvote and downvote operations on forum posts.
The voting system was tested extensively to ensure correct handling of user interactions and data consistency. Test cases included upvoting, downvoting, toggling votes, and switching between vote types. During testing, it was verified that votes were correctly updated in MongoDB (upvotes and downvotes arrays) and reflected instantly in the UI. The system ensured that duplicate votes were not counted and switching votes removed the previous vote before applying the new one.
Test Owner: DevAxis
Test Date: 02/04/2026
Test Results:
The voting system worked correctly, with accurate vote counts and proper handling of toggling and switching behavior.
Additional Comments:
The system maintains data consistency across multiple users and prevents duplicate voting.

3. Requirement: The user should be able to add comments to forum posts.
The comment functionality was tested to ensure that users can interact with posts effectively. Test cases included adding comments, fetching comments, and validating empty inputs. During testing, users successfully added comments which were stored in MongoDB and displayed correctly under the respective posts. The system ensured proper linkage between posts and comments and handled invalid inputs appropriately.
Test Owner: DevAxis
Test Date: 02/04/2026
Test Results:
The comment system worked as expected. Comments were successfully stored and displayed without any issues.
Additional Comments:
The system prevents empty comments and maintains proper association between posts and comments.

4.Requirement : The system should enforce role-based identity display for students and professors.
The role-based display functionality was tested to ensure correct differentiation between user roles. Test cases included login with student and professor accounts and performing forum interactions. During testing

5 Conclusion

5.1 How Effective and exhaustive was the testing?

We were able to achieve high code coverage including statement and branch coverage for most modules. Each API and core functionality was tested to ensure that the database operations were performed correctly and consistently. Various bugs were identified and resolved during the testing phase, and special attention was given to handling edge cases and invalid inputs.

For example, validations were implemented for incorrect login credentials, duplicate entries (such as existing usernames or item IDs), missing form inputs, and invalid data formats. Proper error messages and alerts were displayed to guide the user.

Additionally, each component was tested by multiple developers, and care was taken to ensure that developers tested modules that they had not implemented themselves. This improved the objectivity and effectiveness of the testing process.

5.2 Which components have not been tested adequately?

While most functional requirements were thoroughly tested, some non-functional aspects were not fully evaluated. These include performance testing under very high load, scalability of the system, and stress testing with a large number of concurrent users.

Due to limited resources, testing was conducted under moderate usage conditions rather than extreme scenarios. Features such as handling thousands of simultaneous users, long-term database performance, and security under heavy traffic could benefit from further testing to ensure production-level reliability.

5.3 What difficulties have you faced during testing?

During testing, we faced challenges in setting up and managing automated test environments, especially for backend and API testing. Debugging asynchronous operations and ensuring proper data flow between frontend and backend components was sometimes complex.

Additionally, handling database state during repeated tests and ensuring clean test environments required extra effort. Coordinating integration testing across multiple modules also posed challenges due to interdependencies between components.

5.4 How could the testing be improved?

The testing process can be improved by increasing automation, especially for integration and end-to-end testing, to better simulate real user scenarios. Performance and load testing should be added to evaluate system behavior under high traffic. Additionally, using CI pipelines for continuous testing, improving edge case coverage, and incorporating security testing would further enhance the reliability and quality of the system.

Appendix A - Group Log

S.No	Date	Timings	Venue	Description
1.	28/03/2026	18:00 to 19:00	Google Meet	Discussed testing strategy and divided responsibilities for Unit, Integration, and System testing among team members.
2.	29/03/2026	17:00 to 19:00	Hall 12	Started writing Unit Test cases for Account Management (F9) including username, password, and deletion modules.
3.	30/03/2026	19:00 to 23:00	Hall 12	Reviewed unit testing results .
4	31/03/2026	18:00 to 19:30	Hall 12	Performed Integration Testing between frontend, backend APIs, and MongoDB for Account Management module.
5.	01/04/2026	18:00 to 20:00	Hall 12	Conducted System Testing including end-to-end user flows such as login, update profile, and account deletion.
6.	02/04/2026	18:00 to 21:30	Hall 12	Tested edge cases, validation, and security scenarios including unauthorized access and invalid inputs.
7.	03/04/2026	18:30 to 23:30	Hall 12	Final review of all testing sections (Unit, Integration, System) and documentation formatting for submission.